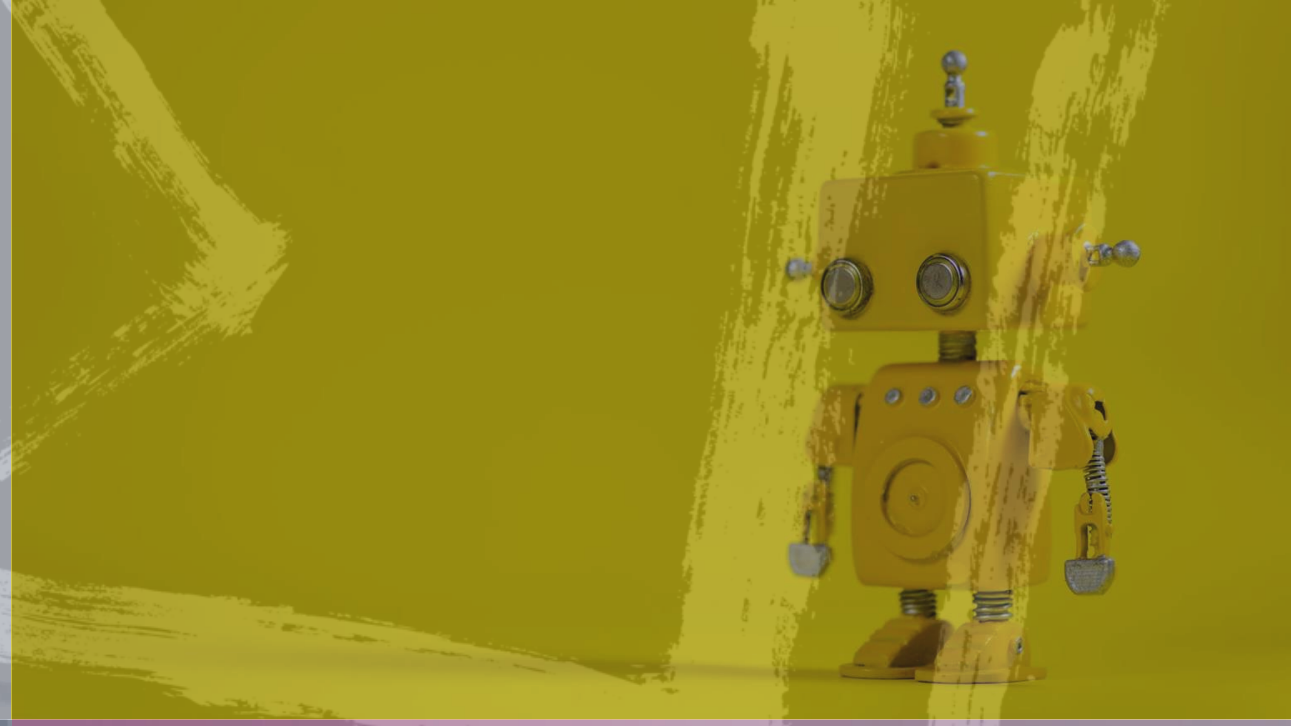





Decision Theory

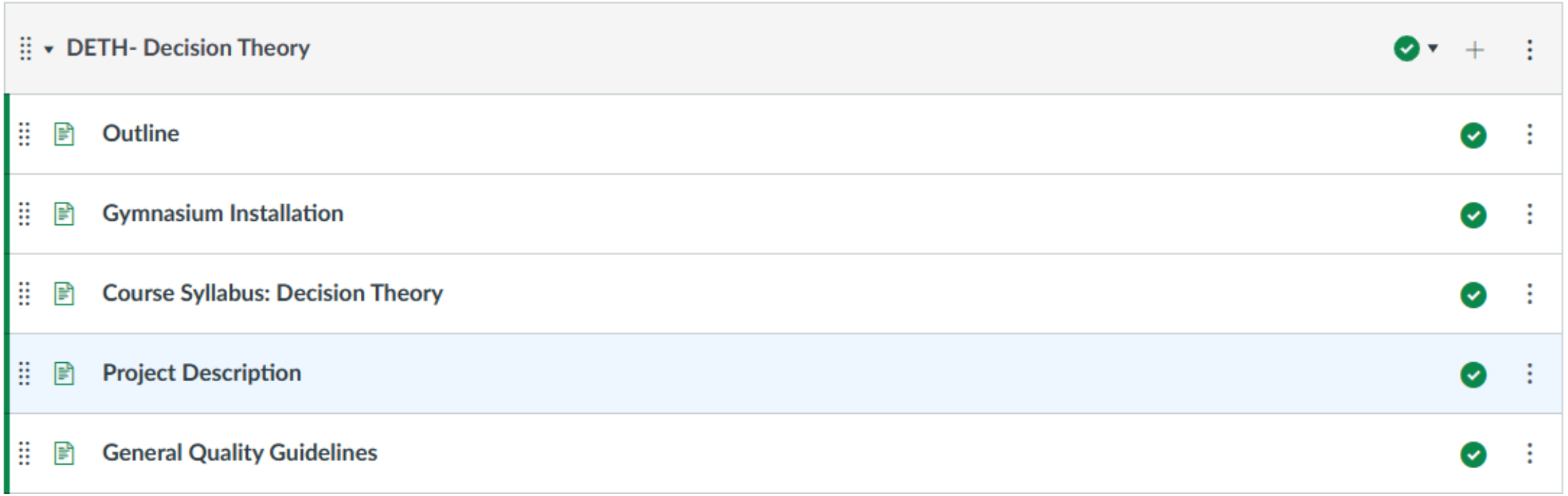
April 2024



iman.mossavat@fontys.nl

Please Familiarize Yourself

AS-APC4-CB-CMK (instructure.com)

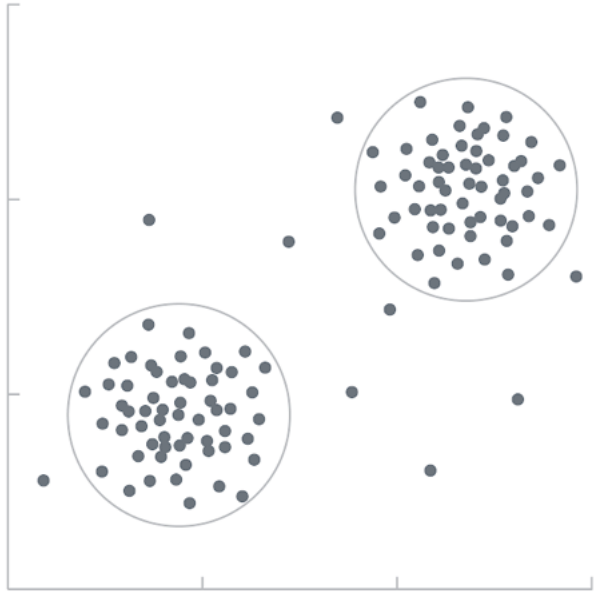


Supervised

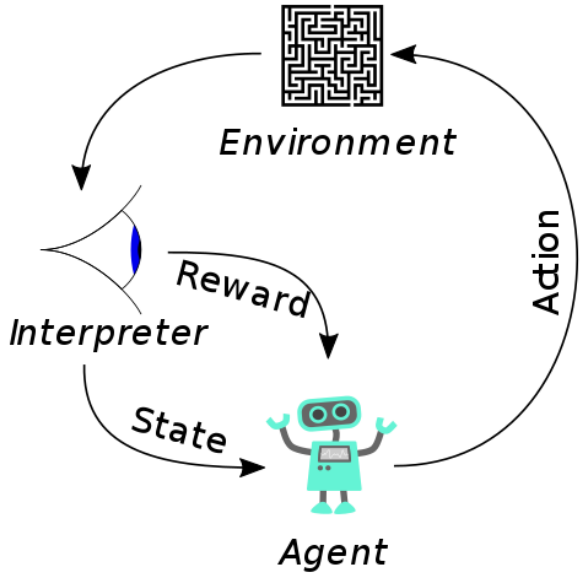


2.jpg
Label: dog

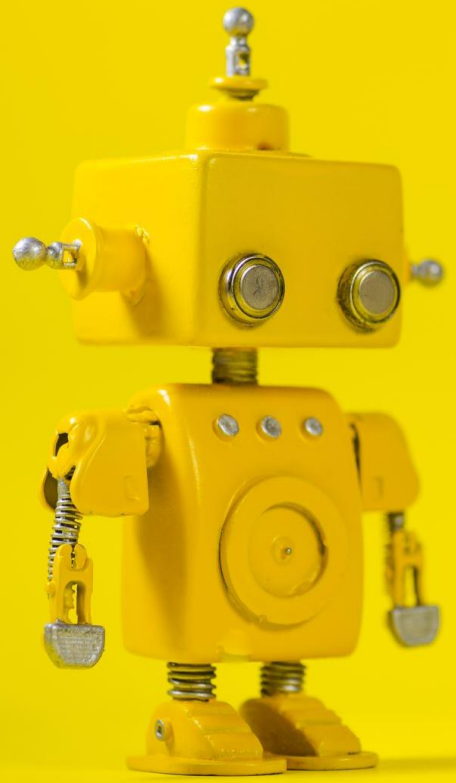
Unsupervised



Reinforcement



Agent (Action holder)



Policy





Environment

Uncertainty



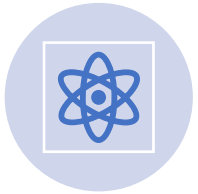
Reward Signal



An Explosion in RL Application



Reinforcement learning from human feedback (RLHF): use in ChatGPT

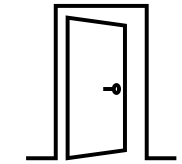


Accelerating fusion science through learned plasma control



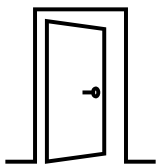
Alpha Tensor: *discovering new algorithms*

Multi-Arm Bandit (1/2)



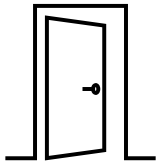


$\frac{1}{8} \times 240 = \mathbf{30}$





$\frac{1}{6} \times 60 = \mathbf{10}$





$\frac{1}{4} \times 80 = \mathbf{20}$

expected reward = chance * reward
(unknown-to be learned)



Multi-Arm Bandit (2/2)

There are three doors. (three actions)

When you **open** a door, you **may** get a **reward**

Each door has its own unique **probability** of giving rewards

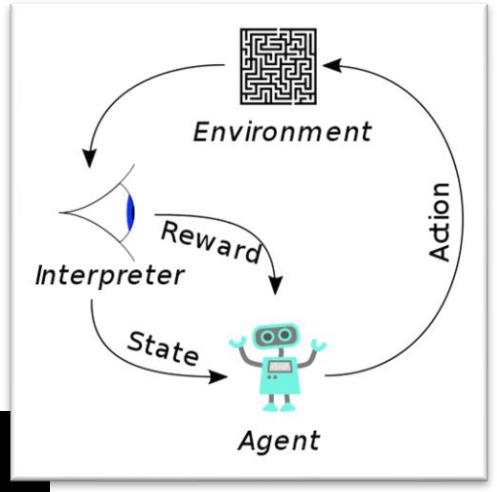
Some doors yield **greater rewards**, while others offer only **modest gains**

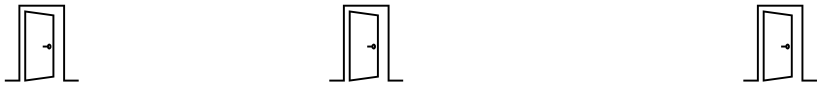
You are allowed to open doors of your choice say T= 100 times (episode length)

Reward size and probabilities do not change from day to day.

You **do not know** the probabilities and the rewards

Environment with no memory





		action 1	action 2	action 3
Time step 1	**State 1**	30	10	20
Time step 2	**State 1**	30	10	20
Time step 3	**State 1**	30	10	20
…	…	…	…	…

Expected reward

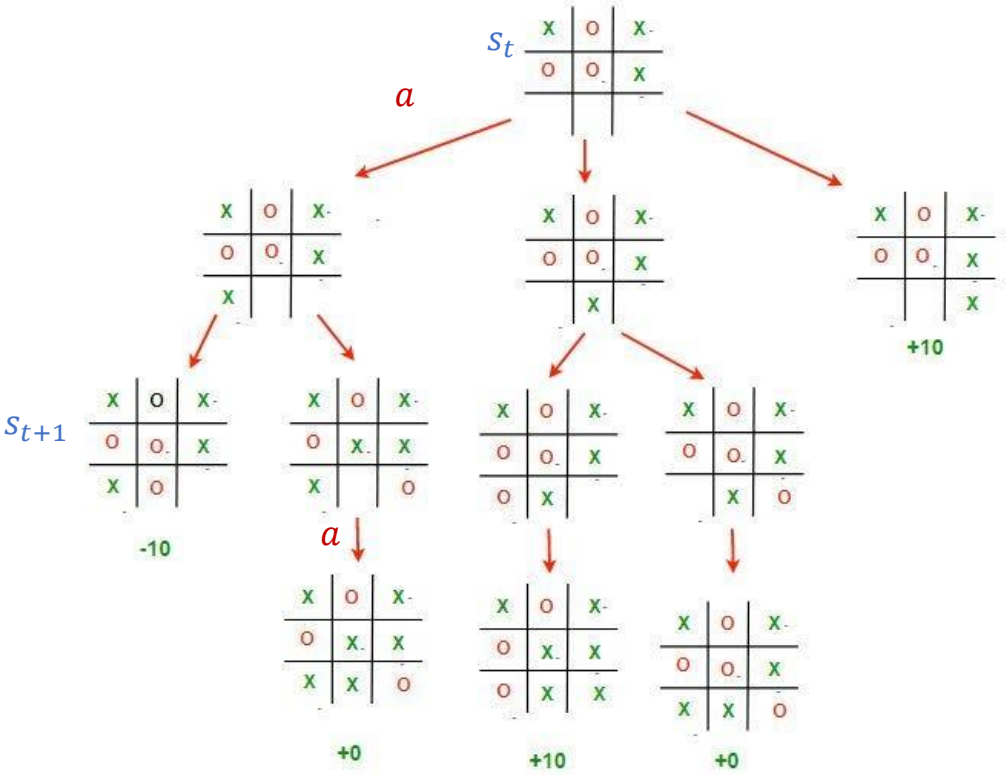
What if the environment has memory?

States s are used to model the memory.

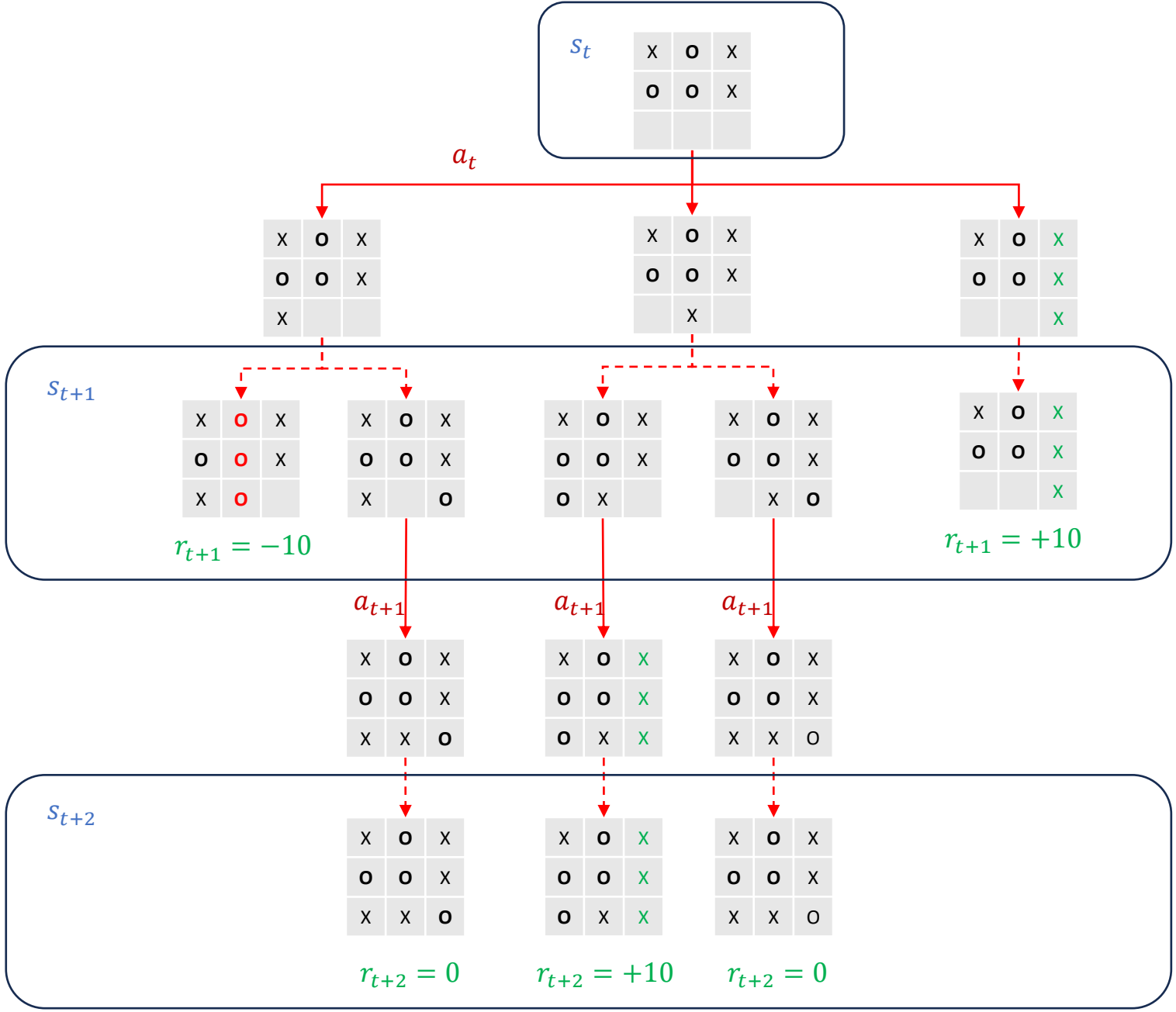
Actions a change the state are shown by arrows.

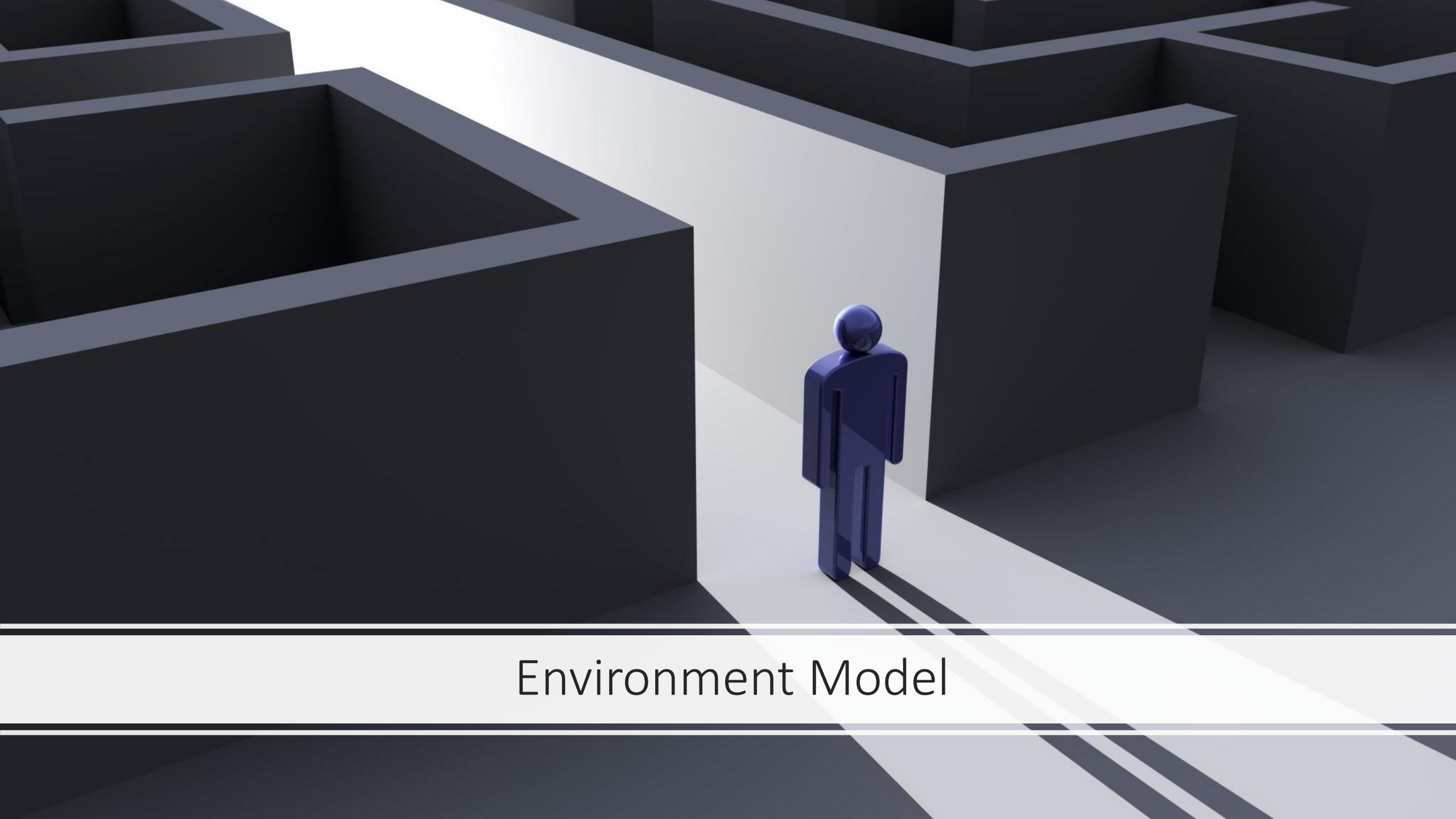
Rewards are attributes of the end leaf.

From rewards we can derive the **quality** $\boldsymbol{Q}$ (expected future reward) of the inner nodes



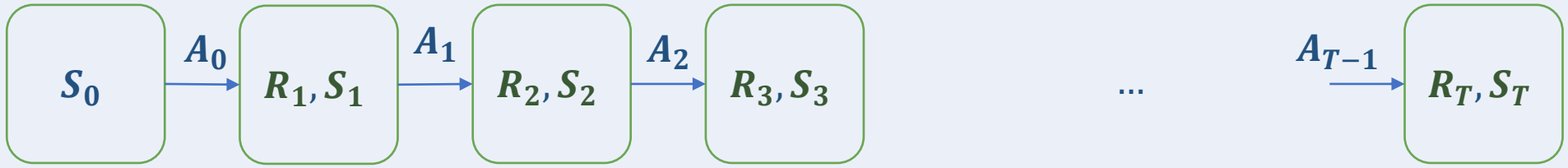
Introduction to Monte Carlo Tree Search: The Game-Changing Algorithm behind DeepMind's AlphaGo | by Ankit Choudhary | Analytics Vidhya | Medium


s_t
a_t
s_{t+1}
$r_{t+1} = -10$
$r_{t+1} = +10$
a_{t+1}
a_{t+1}
a_{t+1}
s_{t+2}
$r_{t+2} = 0$
$r_{t+2} = +10$
$r_{t+2} = 0$



Environment Model

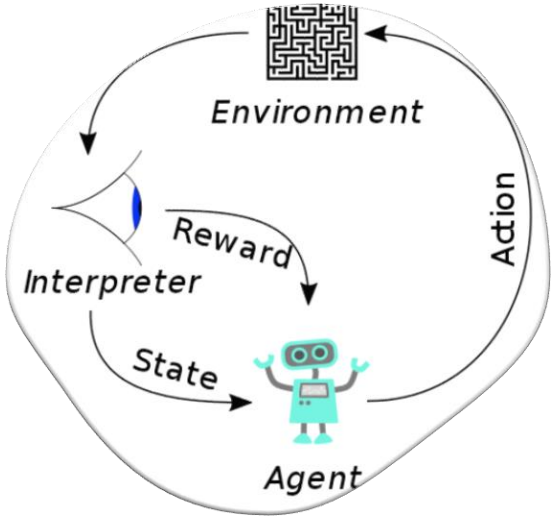
Markov Decision Process (MDP)



$\boldsymbol{S_0}$: State at time 0 (Initial State)
$\boldsymbol{A_0}$: Action at time 0
$\boldsymbol{R_1, S_1}$: Reward and State at time 1
$\boldsymbol{A_1}$: Action at time 1
…

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, \dots, R_T, S_T$$

Markov Decision Process (MDP)



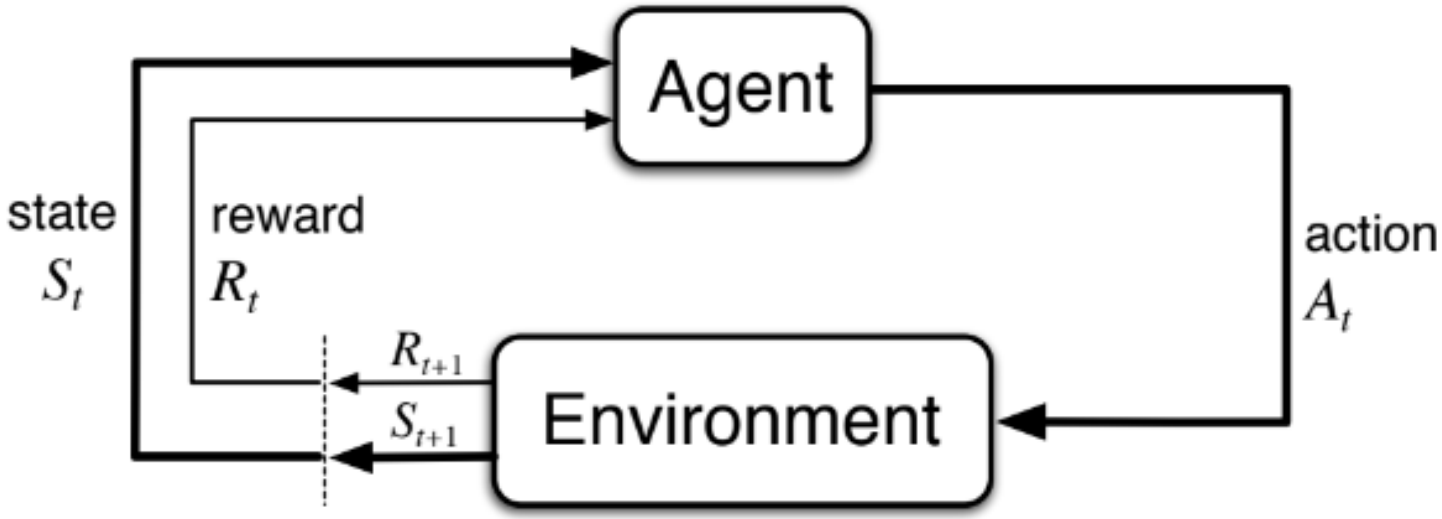
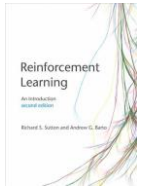


Figure 3.1: The agent–environment interaction in a Markov decision process.



[Sutton and Barto Book]

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, \dots, R_T, S_T$$



Transition Probabilities

- *The model specifies the probabilities:*

$$P(\boldsymbol{S_{t+1}}, \boldsymbol{R_{t+1}} | \boldsymbol{S_t}, \boldsymbol{A_t})$$

Probability of:

- landing in state $\boldsymbol{S_{t+1}}$ and receiving reward $\boldsymbol{R_{t+1}}$ at time $\boldsymbol{t+1}$

after

- taking on action A_t while being at state S_t at time step $\boldsymbol{t}$.

What does it take to define an MDP?



MDP (version 1 as in the videos)

A set of states $\mathcal{S}$: continuous or discrete

A set of actions $\mathcal{A}$: continuous or discrete

A set of rewards $\mathcal{R}$: continuous or discrete

For each state $s \in \mathcal{S}$ there are permitted actions $a(s) \in \mathcal{A}$

Transition Probabilities

$$P(\boldsymbol{S_{t+1}}, \boldsymbol{R_{t+1}} | \boldsymbol{S_t}, \boldsymbol{A_t})$$

MDP (version 2 as in some of the literature)

A set of states $\mathcal{S}$: continuous or discrete

A set of actions $\mathcal{A}$: continuous or discrete

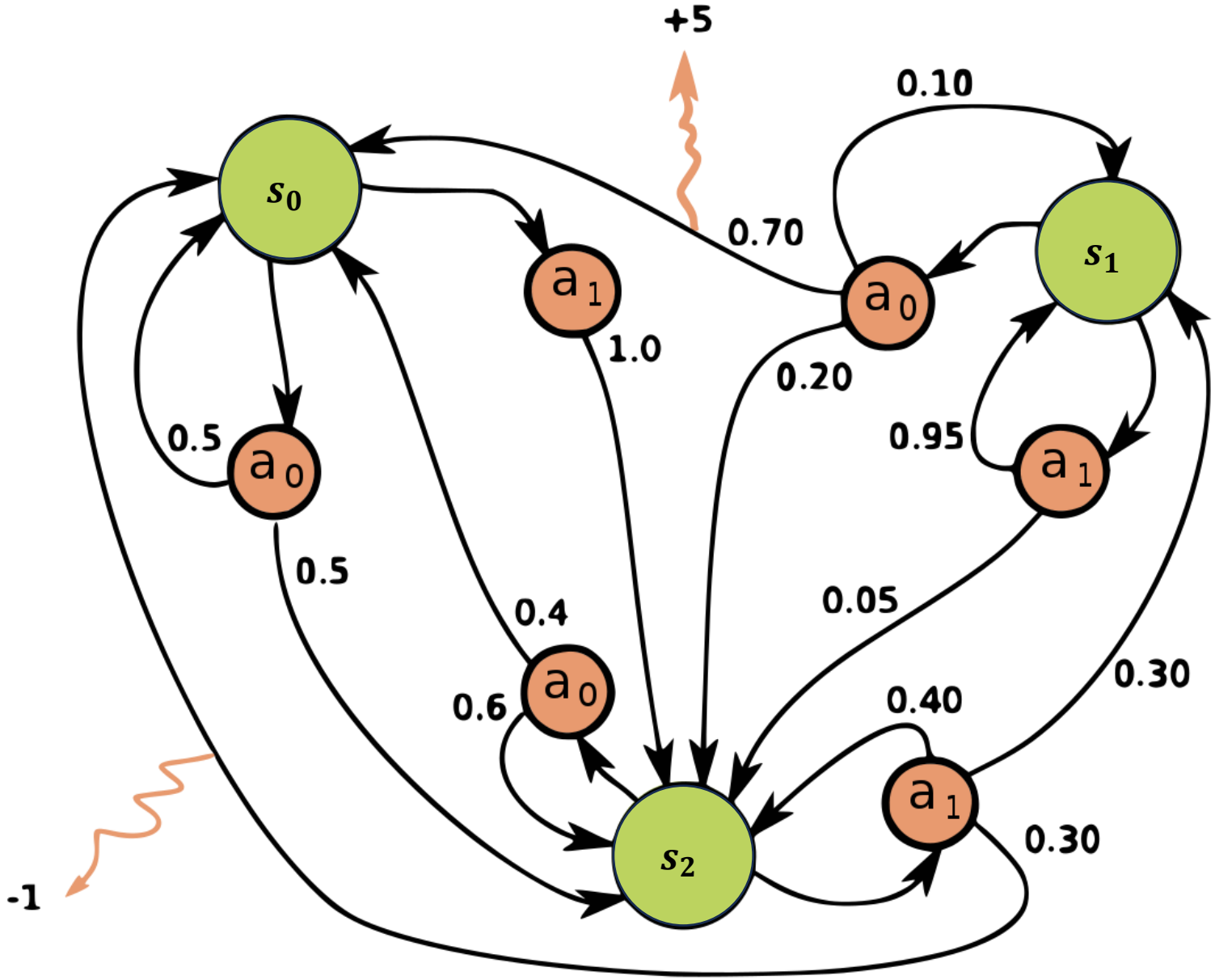
A set of rewards $\mathcal{R}$: continuous or discrete

For each state $s \in \mathcal{S}$ there are permitted actions $a(s) \in \mathcal{A}$

Sometimes transition probabilities are modeled as

$$P(\boldsymbol{S_{t+1}}, |\boldsymbol{S_t}, \boldsymbol{A_t})$$

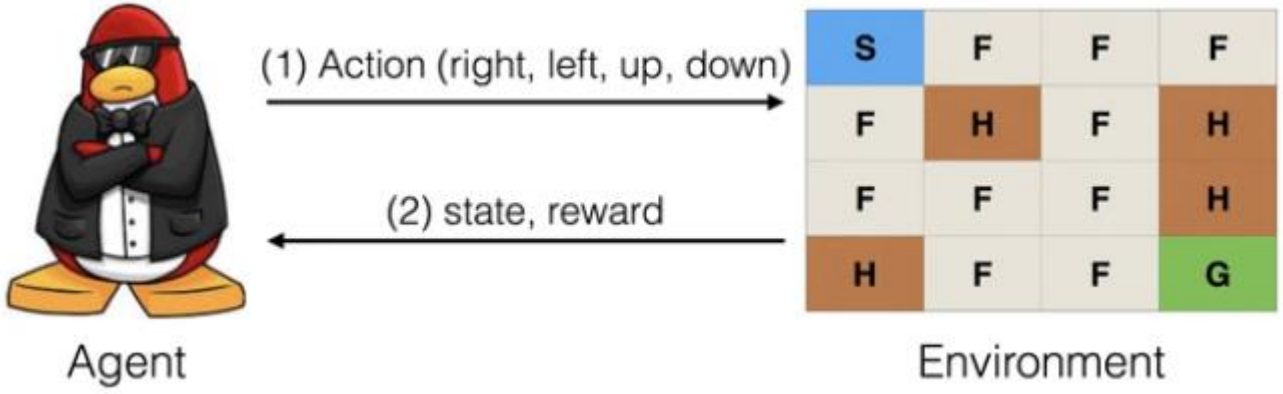
And the reward is a deterministic function of $R_t = f(S_t, S_{t+1}, A_t)$



[Markov decision process - Wikipedia]

Example: Slippery Frozen Lake

Frozen Lake - Gymnasium Documentation (farama.org)



The agent starts at **S** and receives a reward when reaching **G**. If it falls into hole **H** the episode ends (and it may start over)

The agent does NOT have this overview initially, it must figure out the map by experimenting.

The environment env manages the update of the state and calculation of the reward. It knows in which state we are.

```python
import gymnasium as gym
env = gym.make("LunarLander-v2", render_mode="human")
observation, info = env.reset(seed=42)
for _ in range(1000):
   action = env.action_space.sample()  # this is where you would insert your policy
   observation, reward, terminated, truncated, info = env.step(action)

   if terminated or truncated:
      observation, info = env.reset()

env.close()
```

[Gymnasium Documentation (farama.org)]

Initial state

```
import gymnasium as gym
env = gym.make("LunarLander-v2", render_mode="human")
observation, info = env.reset(seed=42)
for _ in range(1000):
   action = env.action_space.sample()  # this is where you would insert your policy
   observation, reward, terminated, truncated, info = env.step(action)

   if terminated or truncated:
      observation, info = env.reset()

env.close()
```

[Gymnasium Documentation (farama.org)]

$T = 1000$ in our slides

```
                                                mode="human")
observation, info = env.reset(seed=42)
for _ in range(1000):
   action = env.action_space.sample()  # this is where you would insert your policy
   observation, reward, terminated, truncated, info = env.step(action)

   if terminated or truncated:
      observation, info = env.reset()

env.close()
```

[Gymnasium Documentation (farama.org)]

env.action_space is the set of feasible actions (state-dependent)

```
import gymnasium a
env = gym.make("Lu        der-v2", render_mode="human")
observation, info =       v.reset(seed=42)
for _ in range(1000
   action = env.action_space.sample()  # this is where you would insert your policy
   observation, reward, terminated, truncated, info = env.step(action)

   if terminated or truncated:
      observation, info = env.reset()

env.close()
```

[Gymnasium Documentation (farama.org)]

Here we select a random move! (random policy, maximum exploration)

```
import gymnasium as gym
env = gym.make("LunarLander-v2"      der_mode="human")
observation, info = env.reset(s      42)
for _ in range(1000):
   action = env.action_space.sample()  # this is where you would insert your policy
   observation, reward, terminated, truncated, info = env.step(action)

   if terminated or truncated:
      observation, info = env.reset()

env.close()
```

[Gymnasium Documentation (farama.org)]

Here we ask the environment to execute the action give the state we are in

```python
import gymnasium as gym
env = gym.make("LunarLander-v2", render_mode="human")
observation, info = env.reset(seed=42)
for _ in range(1000):
   action = env.action_space.sample()  # this is where you would insert your policy
   observation, reward, terminated, truncated, info = env.step(action)

   if terminated or truncated:
      observation, info = env.reset()

env.close()
```

[Gymnasium Documentation (farama.org)]

Here we ask the environment to execute the action give the state we are in (state is an environment variable)

```python
import gymnasium as gym
env = gym.make("LunarLander-v2", render_mode="human")
observation, info = env.reset(seed=42)
for _ in range(1000):
   action = env.action_space.sample()  # this is where you would insert your policy
   observation, reward, terminated, truncated, info = env.step(action)

   if terminated or truncated:
      observation, info = env.reset()

env.close()
```

[Gymnasium Documentation (farama.org)]

New state

```
import gy    ium as gym
env = gym    ke("LunarLander-v2", render_mode="human")
observati    , info = env.reset(seed=42)
for _ in     ange(1000):
    actio    = env.action_space.sample()  # this is where you would insert your policy
    observation, reward, terminated, truncated, info = env.step(action)

    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

[Gymnasium Documentation (farama.org)]

More examples of MDPs



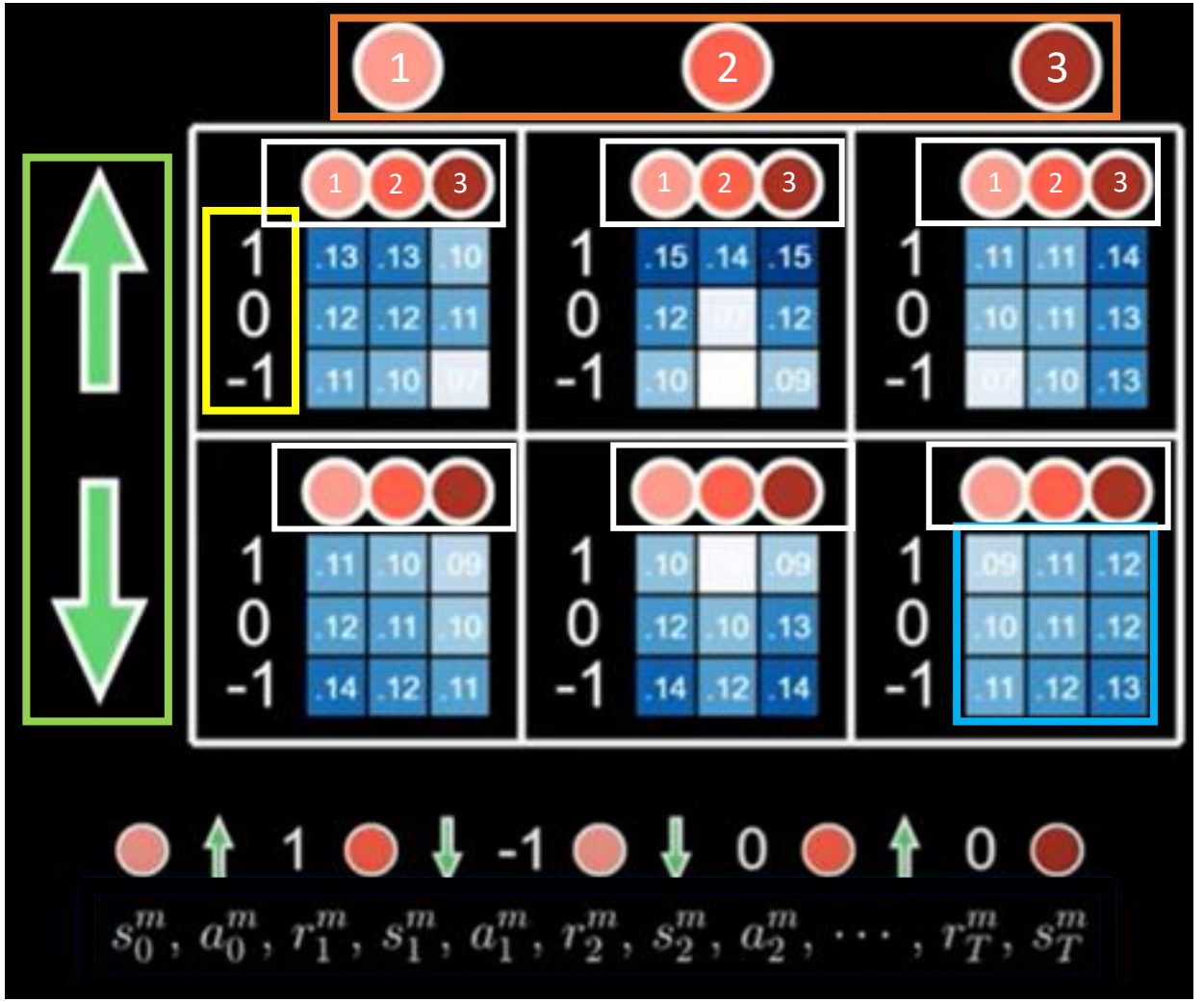
Rewards r

Actions a

Current State s

Next State s'

Transition Probabilities



Transition Probabilities

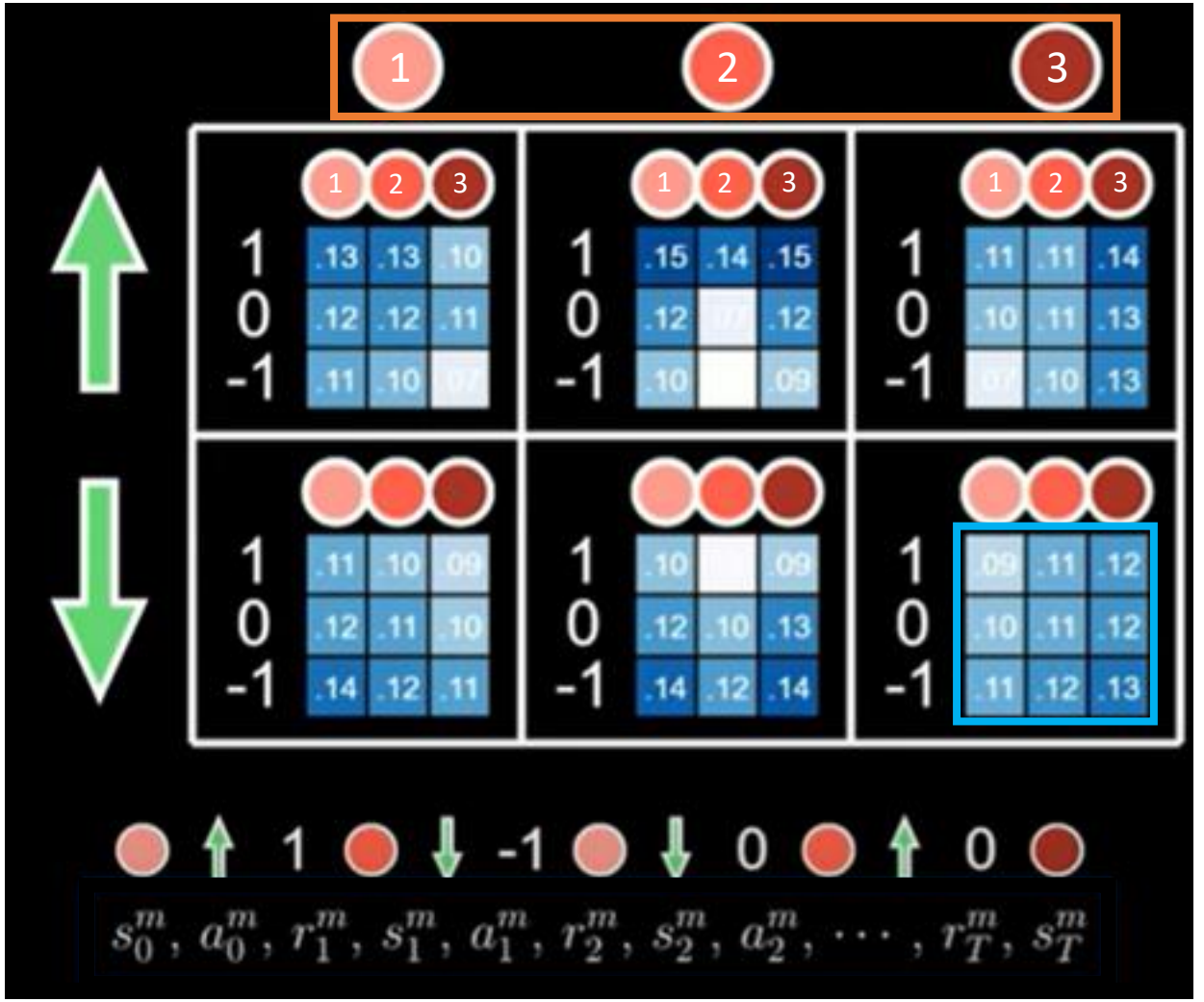
$P(r, s'|s, a)$

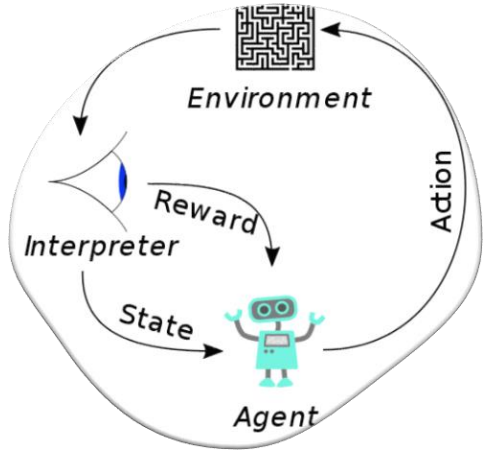
$P(\bullet, 1|1, \Uparrow)$

$P(3, 1|2, \Downarrow)$

https://youtu.be/NFo9v_yKQXA

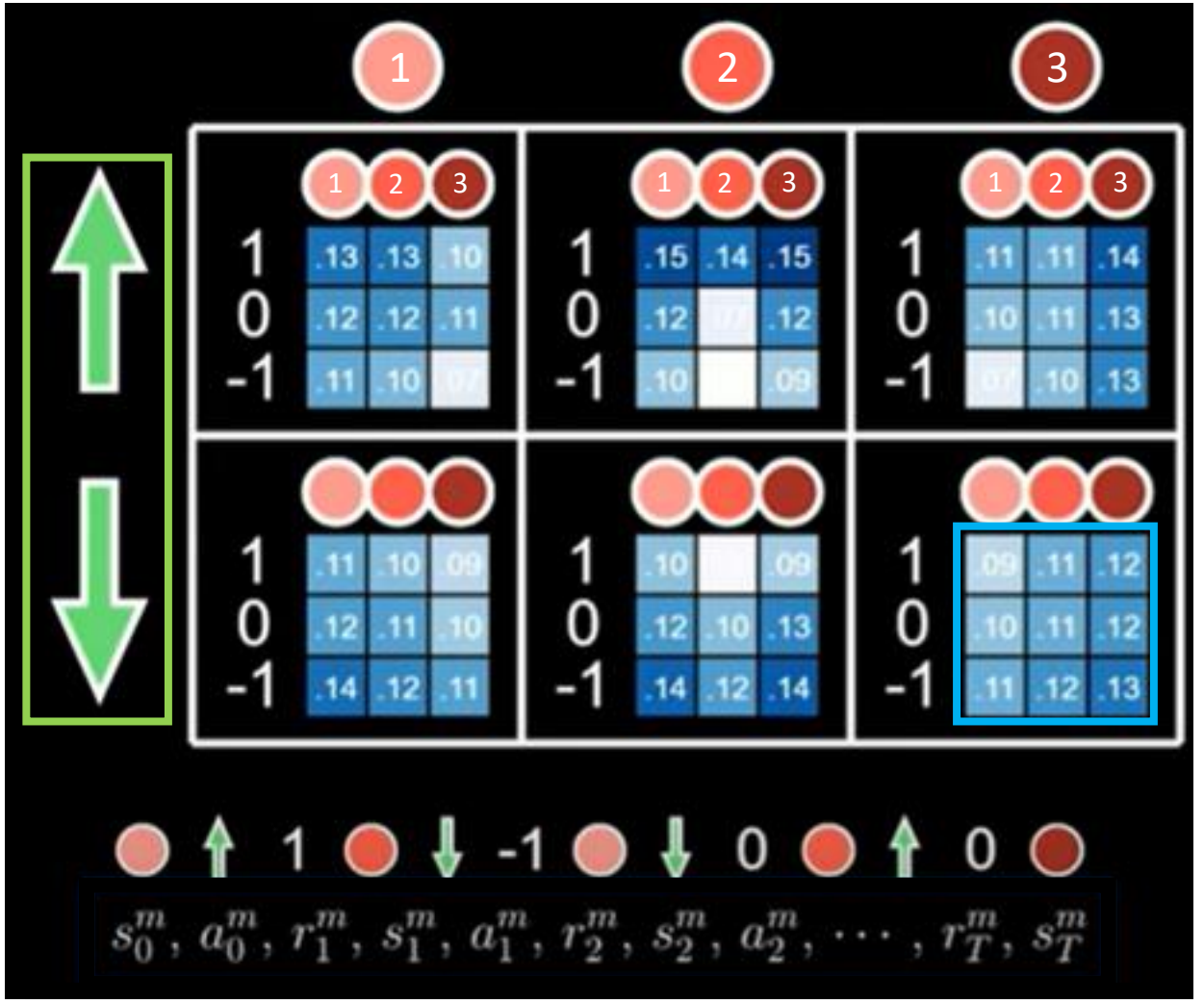
Current State s

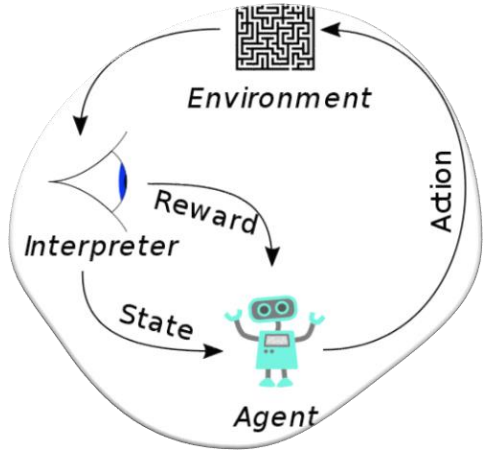




https://youtu.be/NFo9v_yKQXA

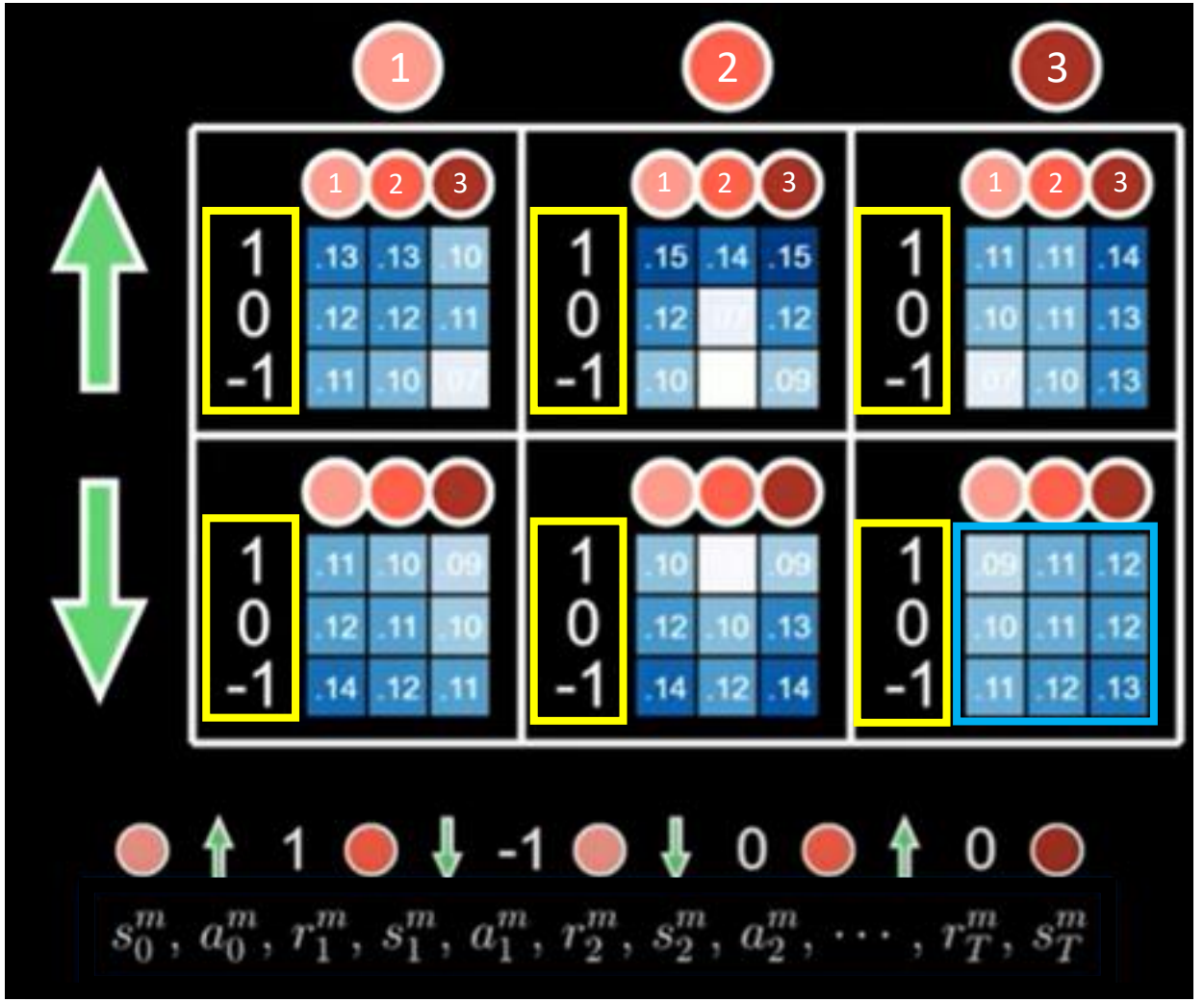
Actions a

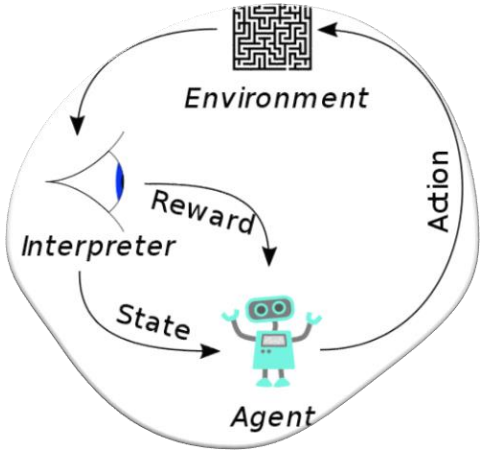




https://youtu.be/NFo9v_yKQXA

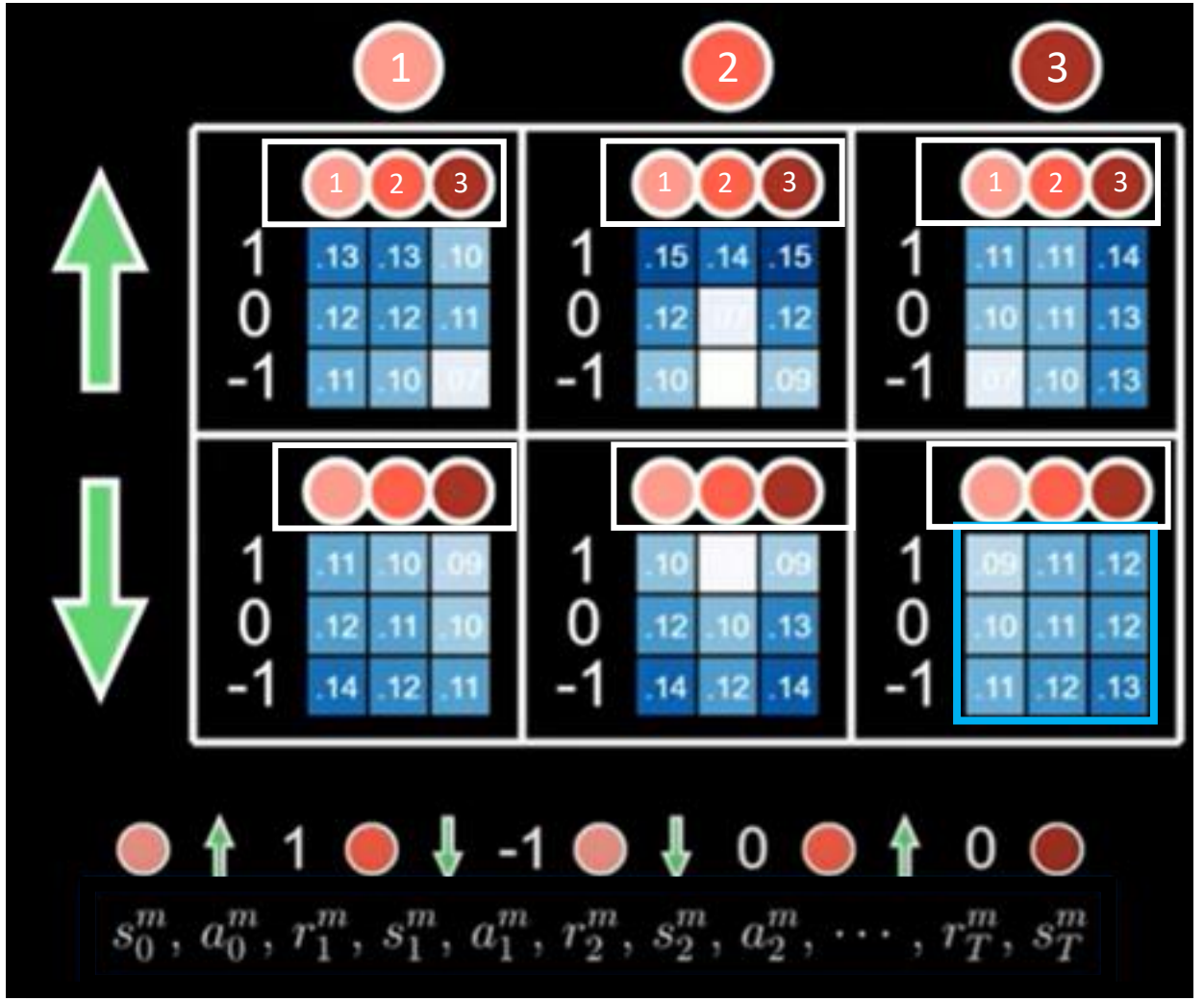
Rewards r





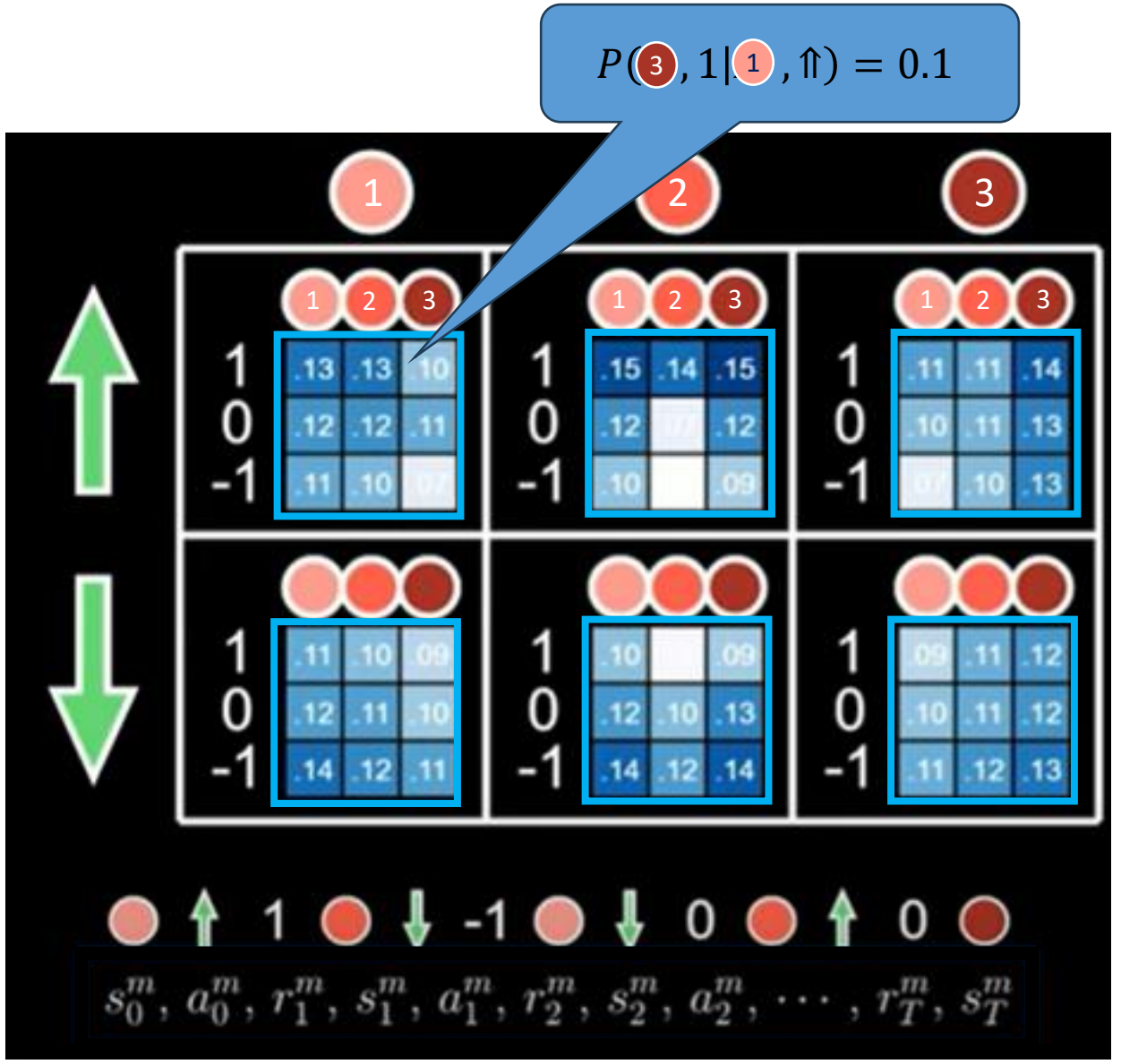
https://youtu.be/NFo9v_yKQXA

Next State s'



https://youtu.be/NFo9v_yKQXA

Transition Probabilities



Transition Probabilities

$P(r, s'|s, a)$

$P(3, 1|1, \Uparrow)$

$P(3, 1|2, \Downarrow)$

https://youtu.be/NFo9v_yKQXA

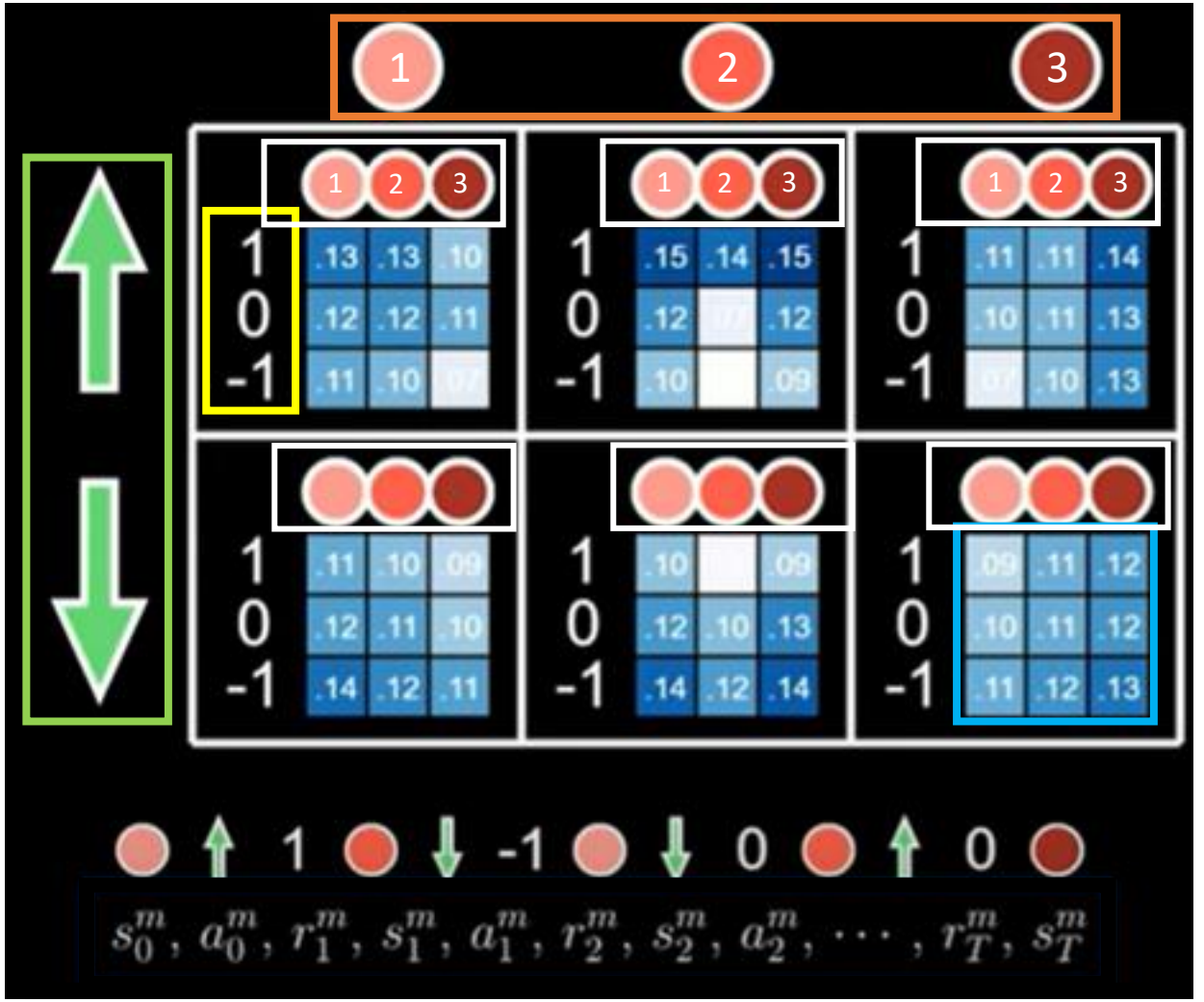
Rewards r

Actions a

Current State s

Next State s'

Transition Probabilities



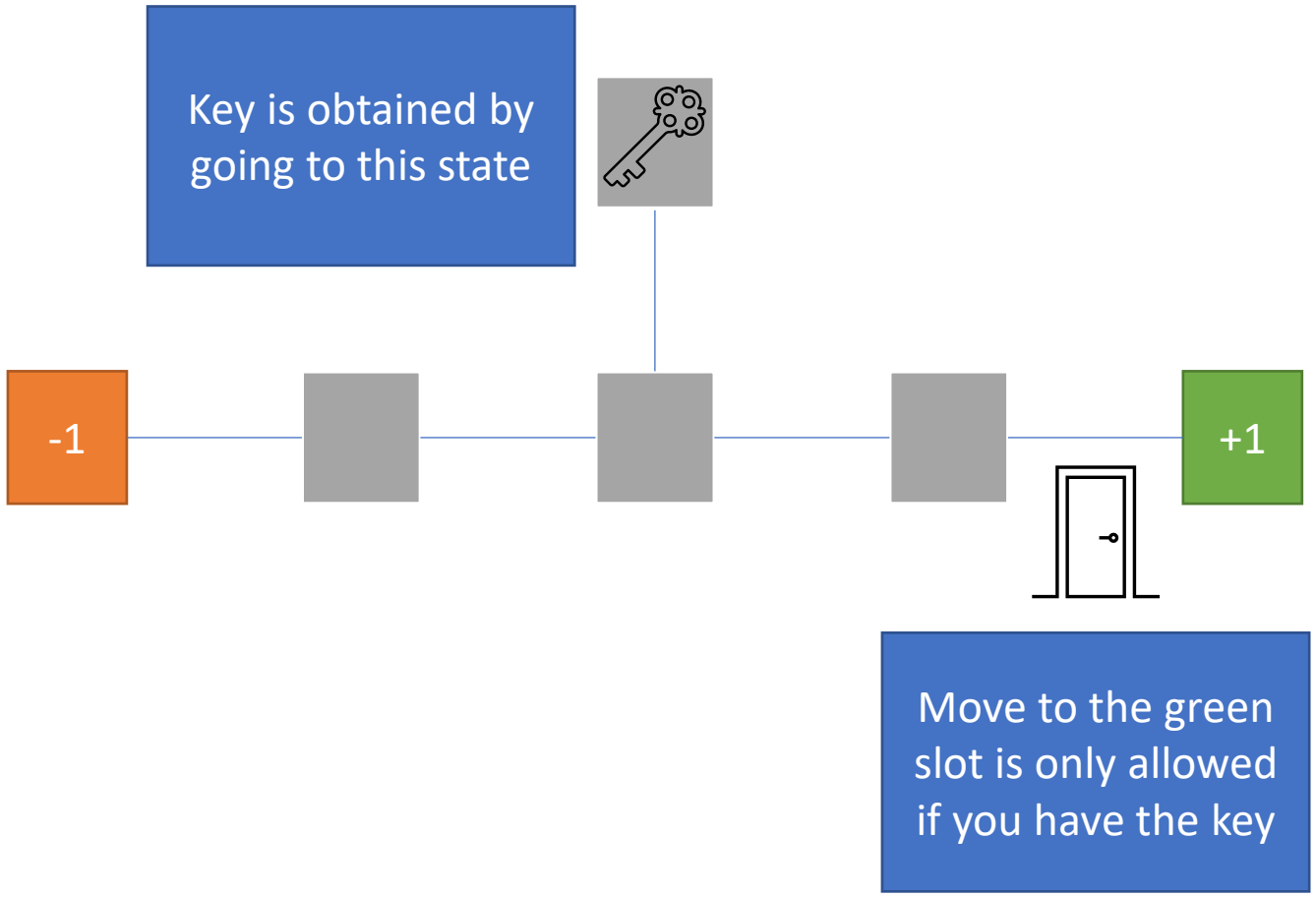
Transition Probabilities

$P(r, s'|s, a)$

$P(\bullet, 1|1, \Uparrow)$

$P(3, 1|2, \Downarrow)$

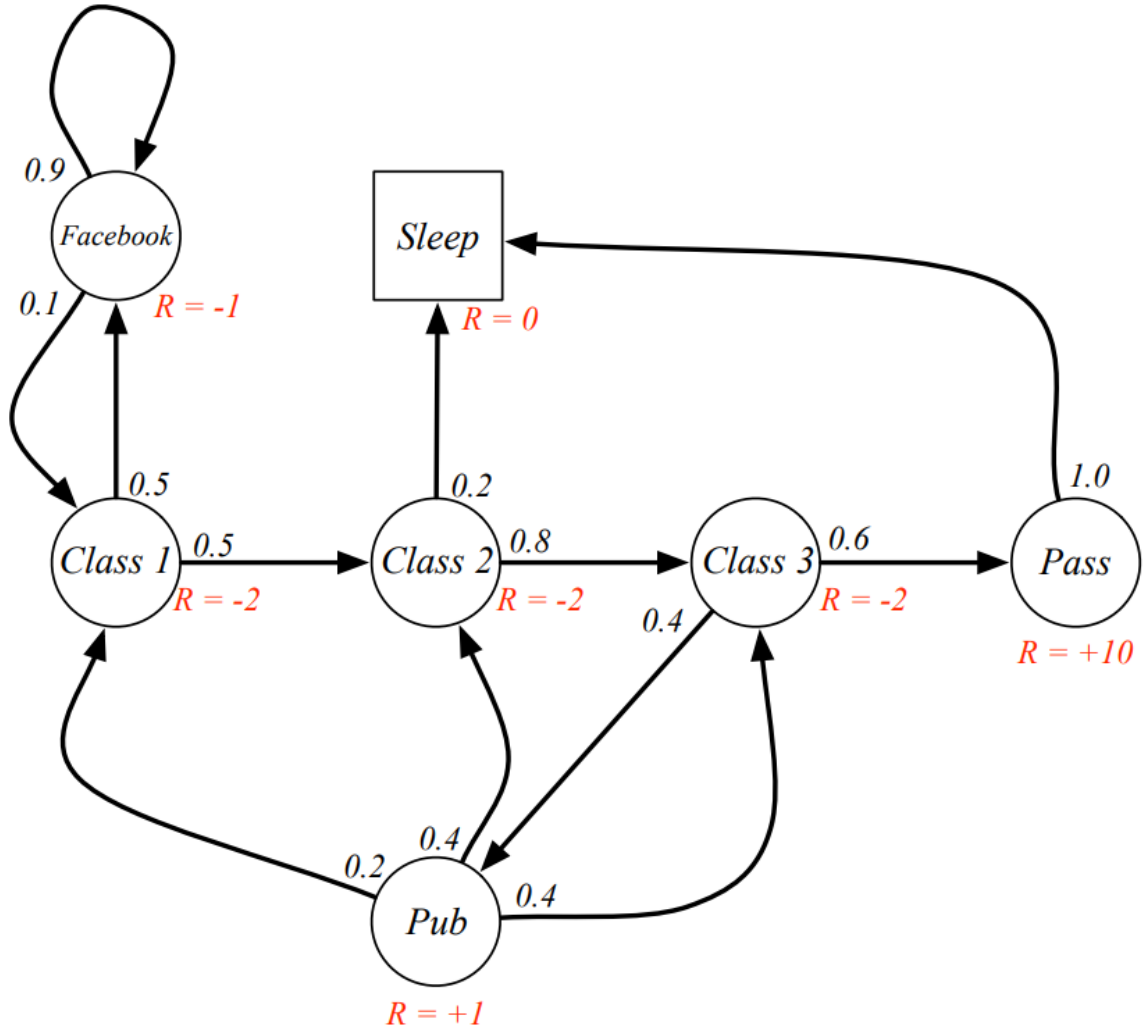
https://youtu.be/NFo9v_yKQXA

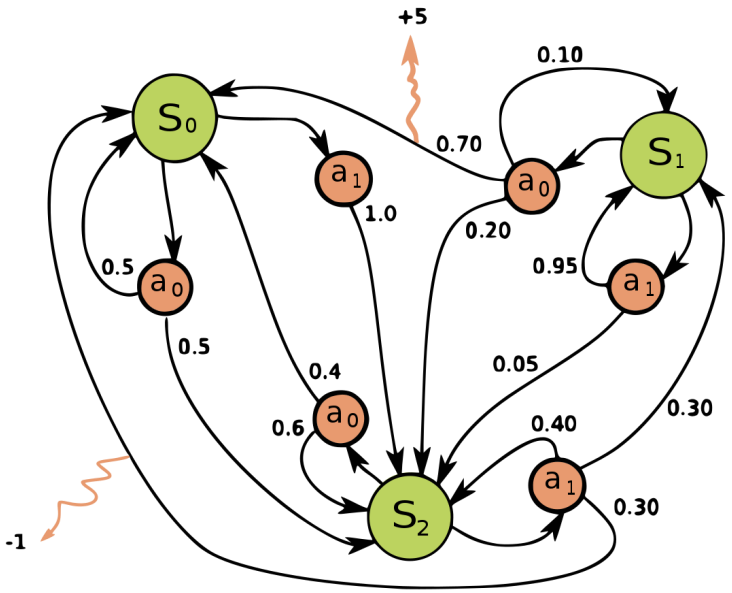


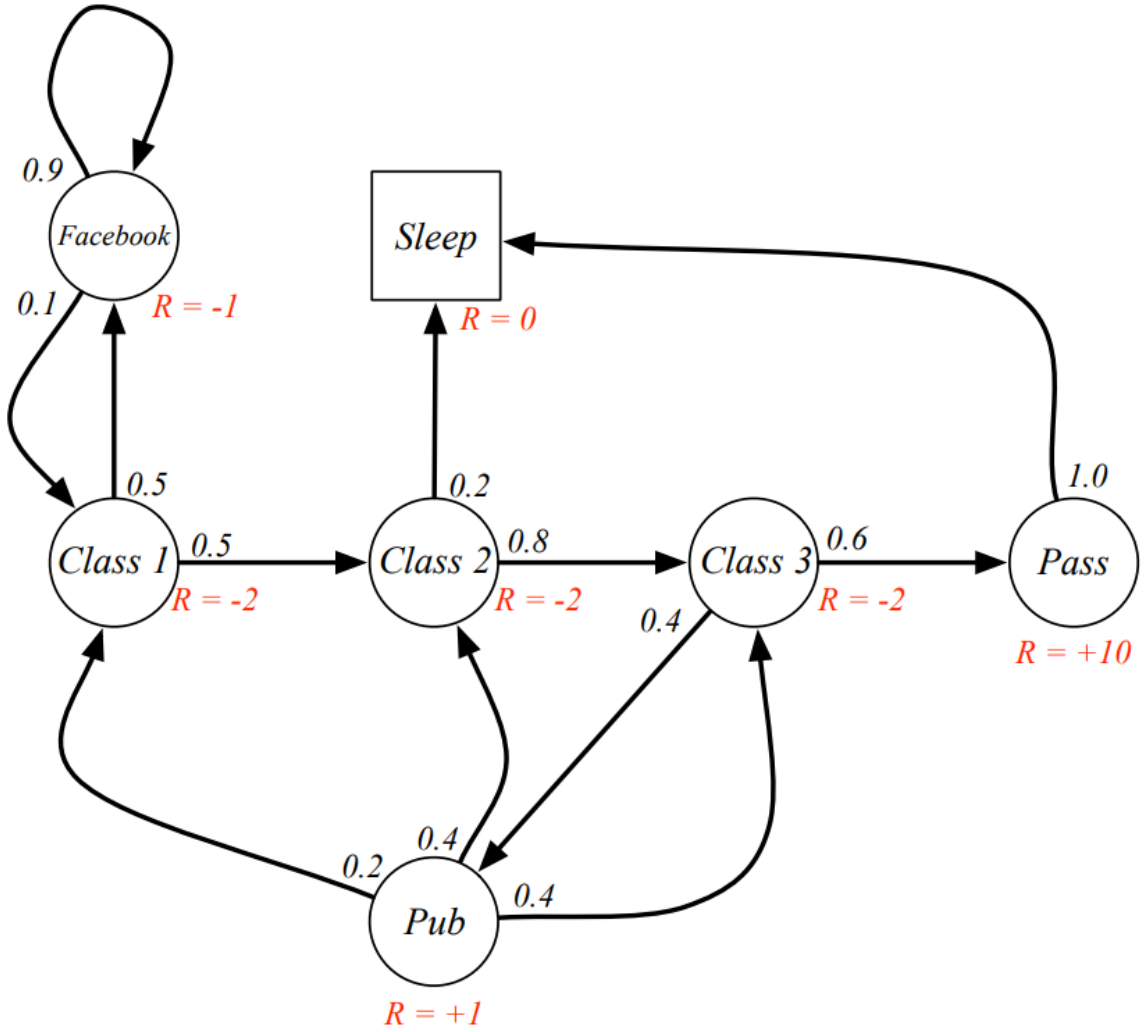
What are the steps to implement this?

Student Life
Not an MDP, why?

[Lecture 2: Markov Decision Processes (stanford.edu)]



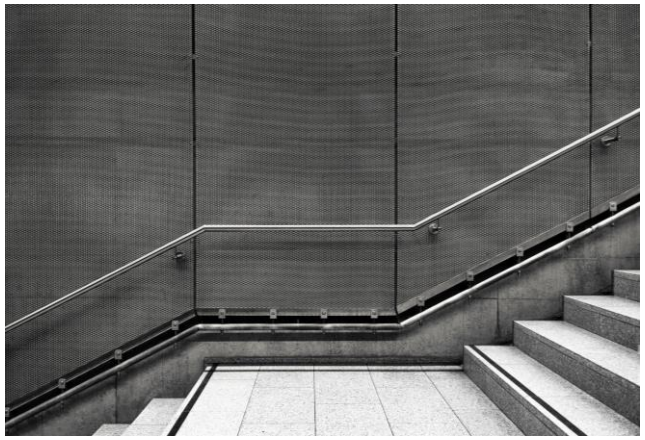




No actions, this is Markov reward process











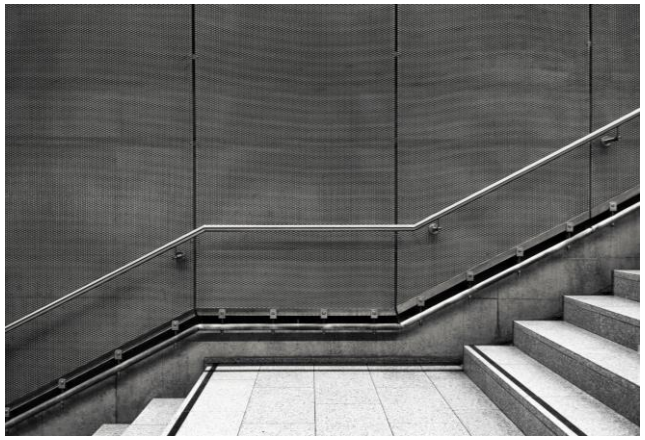
Design your MDP

As a student, you are juggling **time**, **energy**, and **transport options** to reach class **punctually** amidst potential **delays** and varying **energy** levels due to late-night activities. Your daily commute involves navigating between different transport modes (bus or bike, stairs or elevator) while managing energy reserves and potential time constraints.

What are states, actions, uncertainties (delays, oversleeping, …), and day-to-day variations?









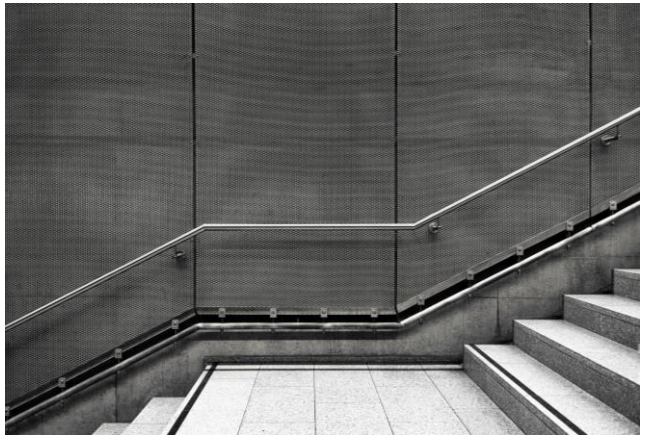


Starting Conditions

- ✓ Random energy level between 1 to 5, depending on your previous night's activities (party/no party).
- ✓ Initial time allowance of up to 2 units before class.
- ✓ Possibility of oversleeping for 0 or 15 minutes based on the previous night's activities (party/no party).









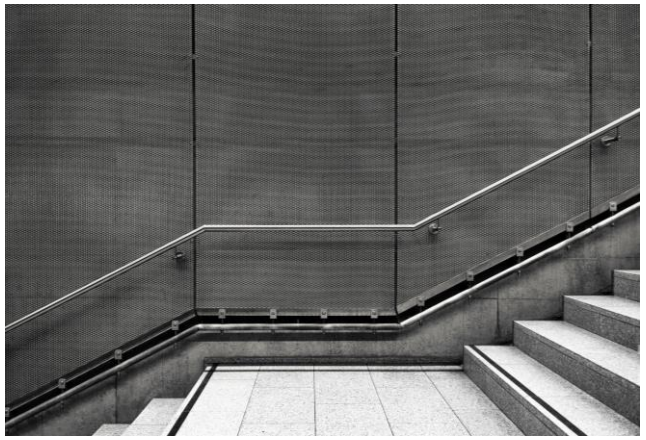


Transport Options

- ✓ **Bike:** 1-hour duration, potential 10 minutes to 1 hour delays. Costs 0 energy.
- ✓ **Bus:** Variable time with potential discrete delays, no direct energy loss unless delayed.
- ✓ **Elevator:** 2-minute duration, potential discrete delays of up to 7 minutes. No energy loss.
- ✓ **Stairs:** 3-minute duration, no delays. Costs -1 energy per floor.





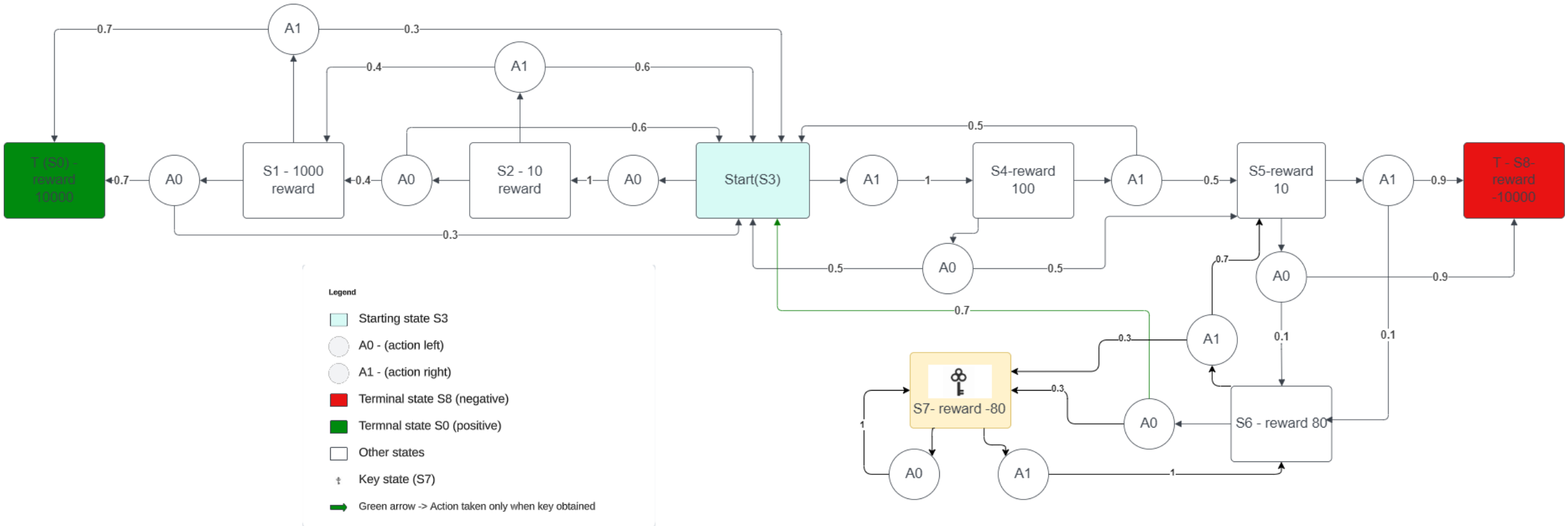






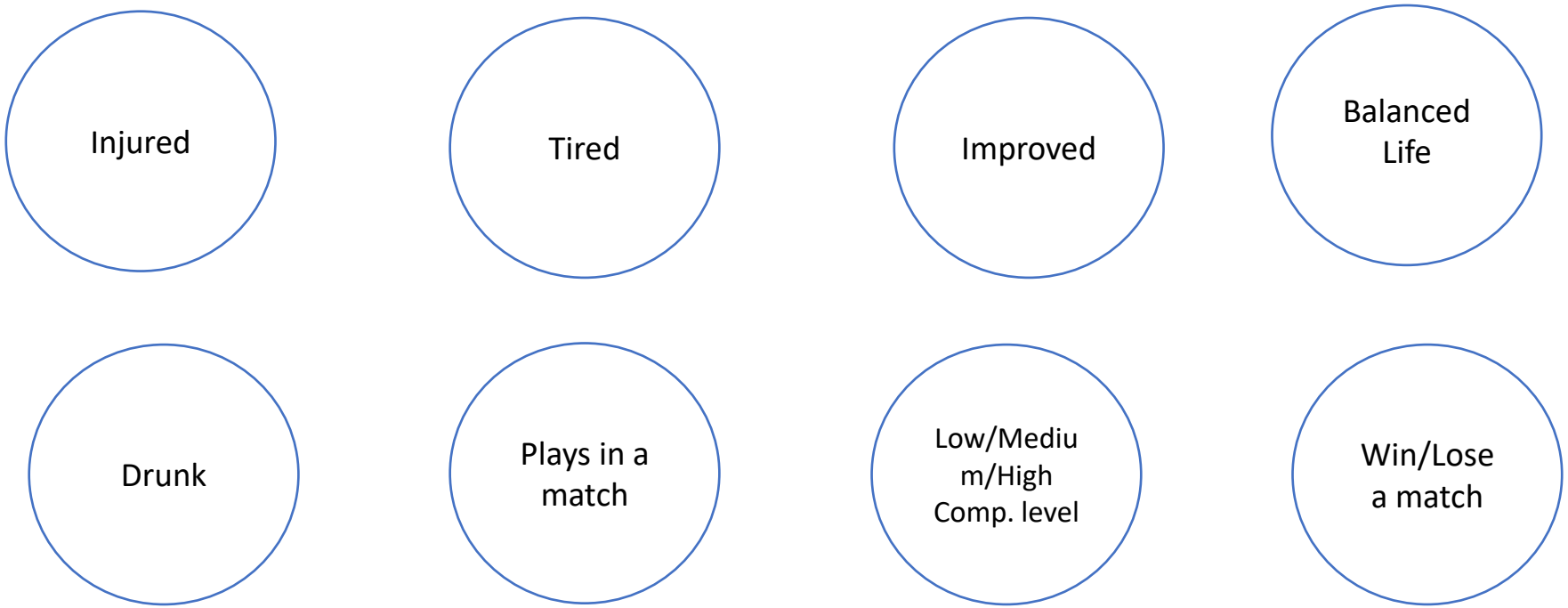
Reward structure

- ✓ **Being Late:** Results in a penalty of -1 point per minute after the initial 2 minutes of delay.
- ✓ **Being Early:** Yields a reward of 2 points.
- ✓ **Energy Levels (1 to 5):** Corresponding rewards are -5, -2, 0, 2, and 5 points, respectively.

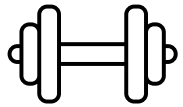


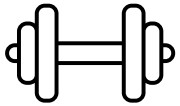
Designed by Mihail Vasilev, Ivana Nedelkova to model addiction

States


Injured
Tired
Improved
Balanced Life
Drunk
Plays in a match
Low/Mediu m/High Comp. level
Win/Lose a match

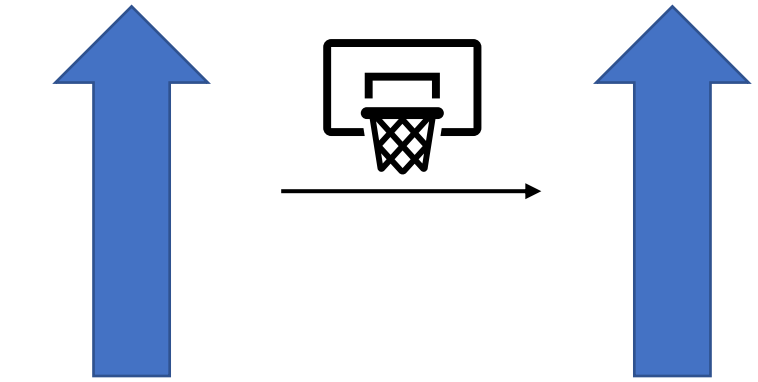
Actions

1 Train 

2 Rest, then train  

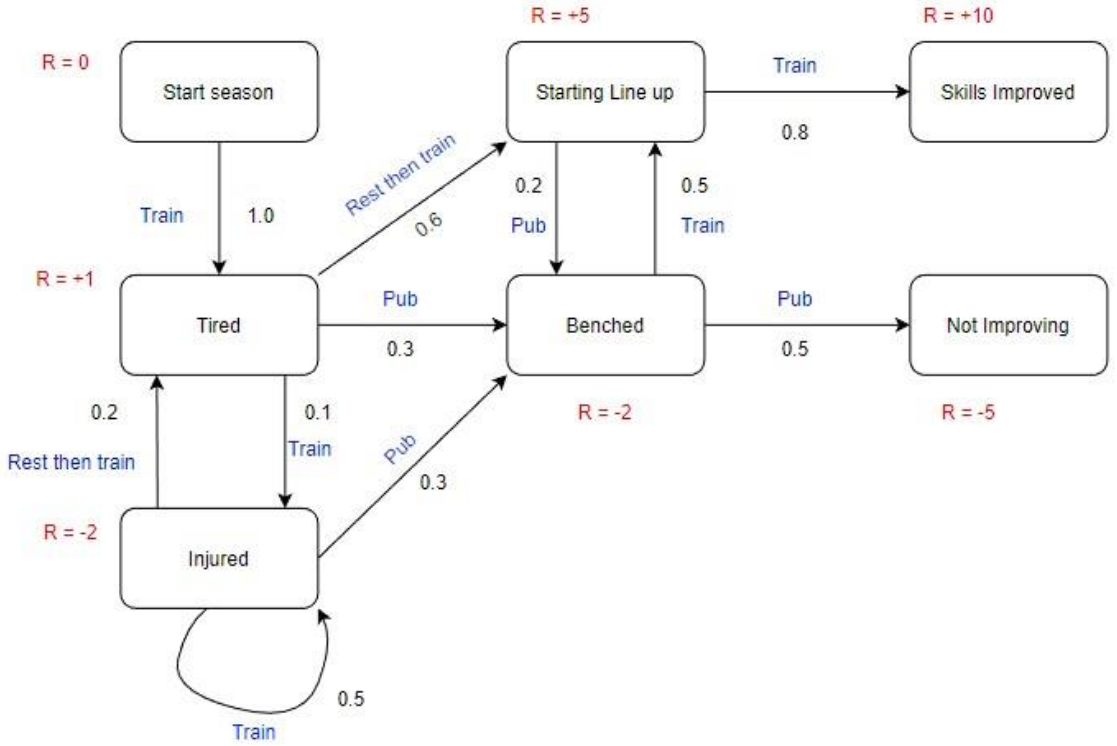
3 Go to the pub 

How likely is it for an athlete to win a game?



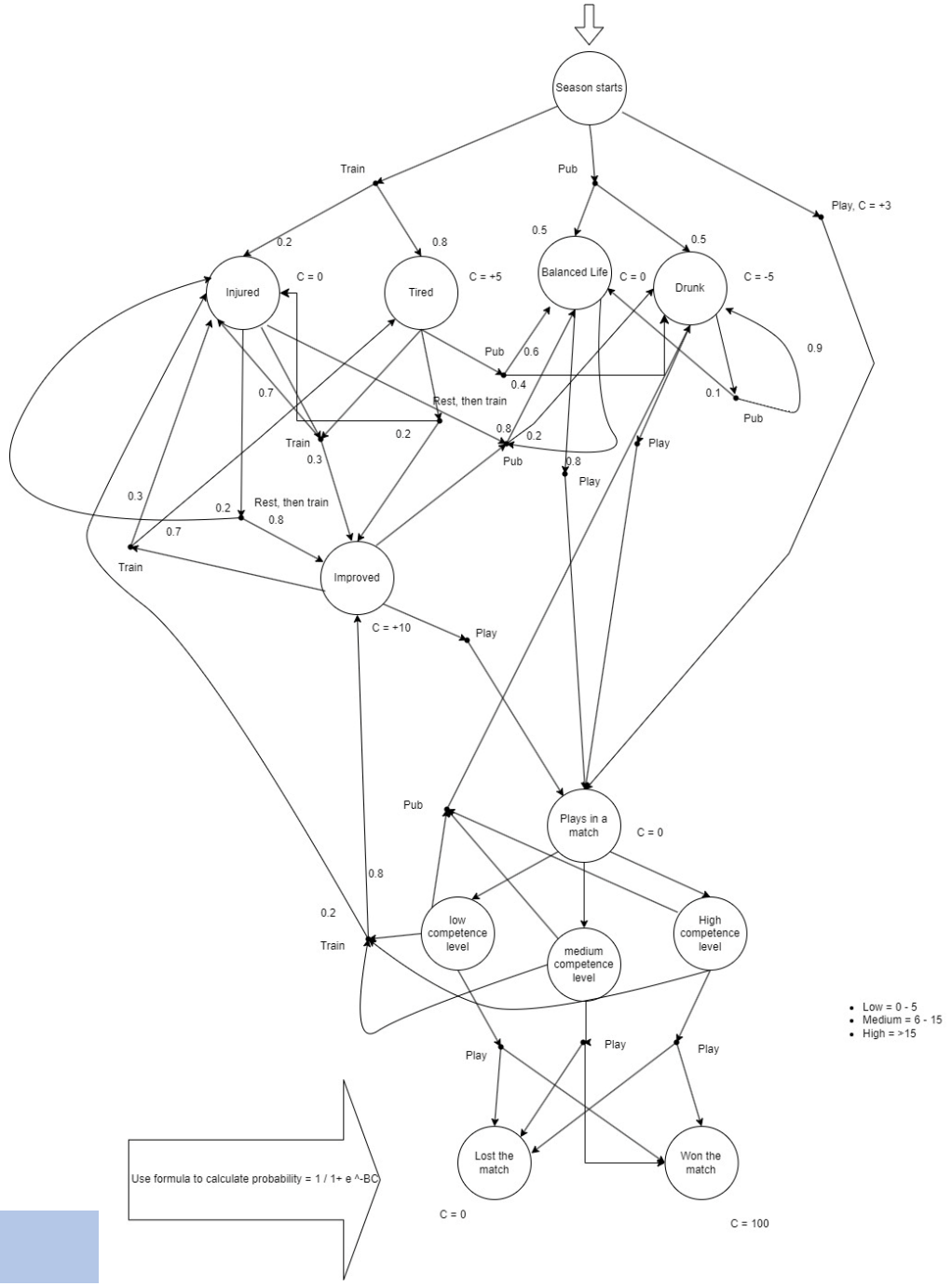
Joseph Abimael & Adel Alaudeen

Initial MDP



Joseph Abimael & Adel Alaudeen

Updated mdp



Joseph Abimael & Adel Alaudeen

CustomEnv Overview

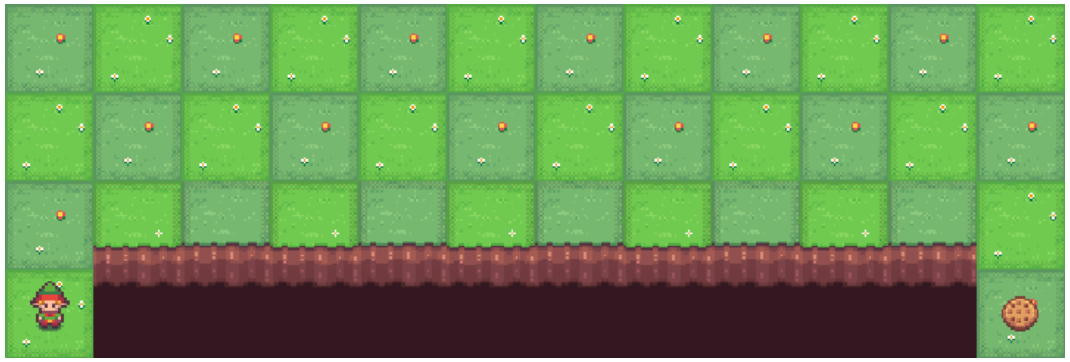
Similarities

- Discount Factor (gamma)
- Learning Rate (alpha)
- Epsilon (decay)
- Reward Structure/Penalties

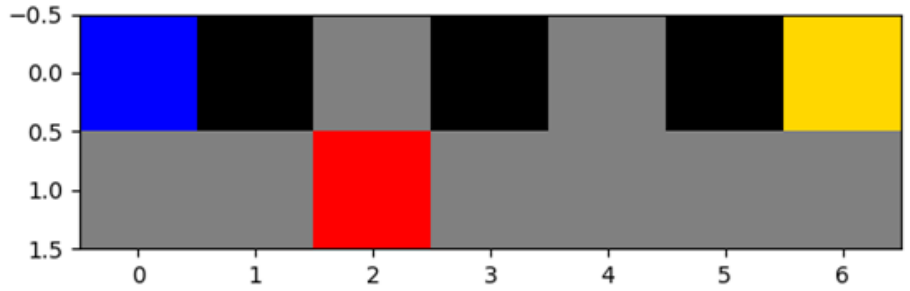
Extended Features

- Extra (moving) Terminal State
- Spacing of Crevices
- Randomness

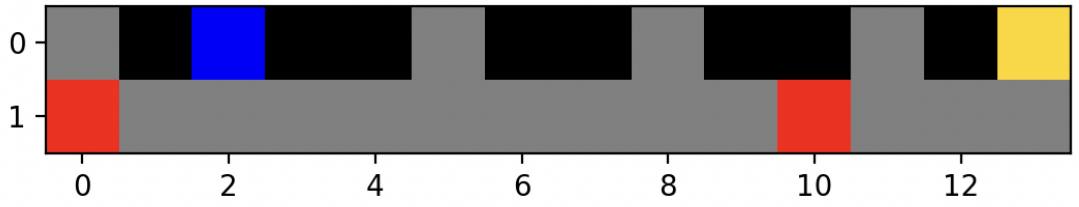
Our Inspiration :



OG Version:



Features Version:

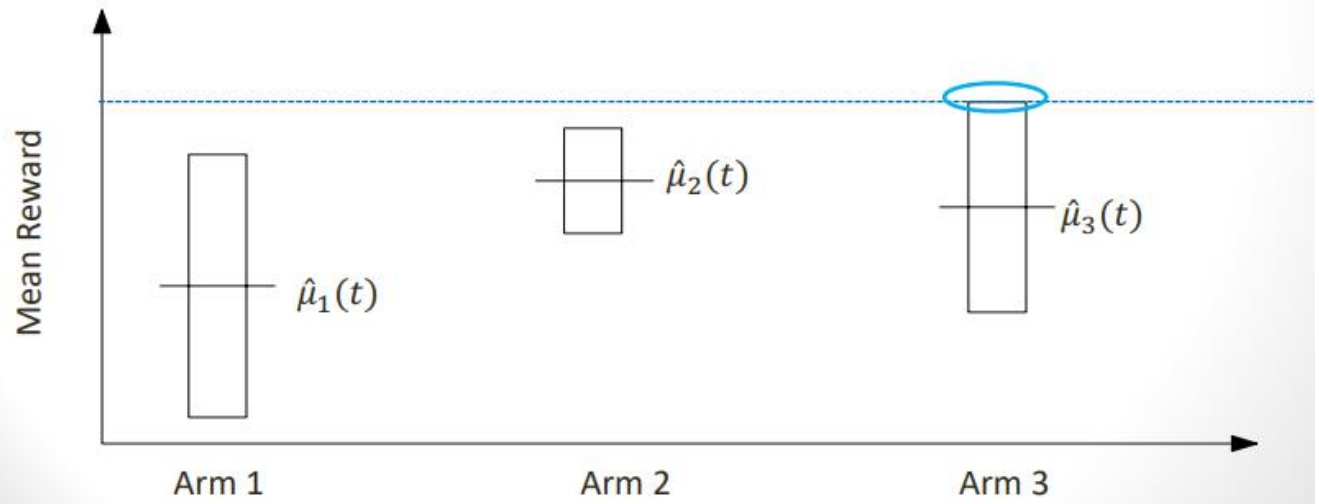


Luka Brandsema, Abe van der Werf

UCB (Upper Confidence Bound) Algorithm

$$k_{t+1} = \arg\max \hat{\mu}_k(t) + \sqrt{\frac{a \log t}{n_k(t)}}$$

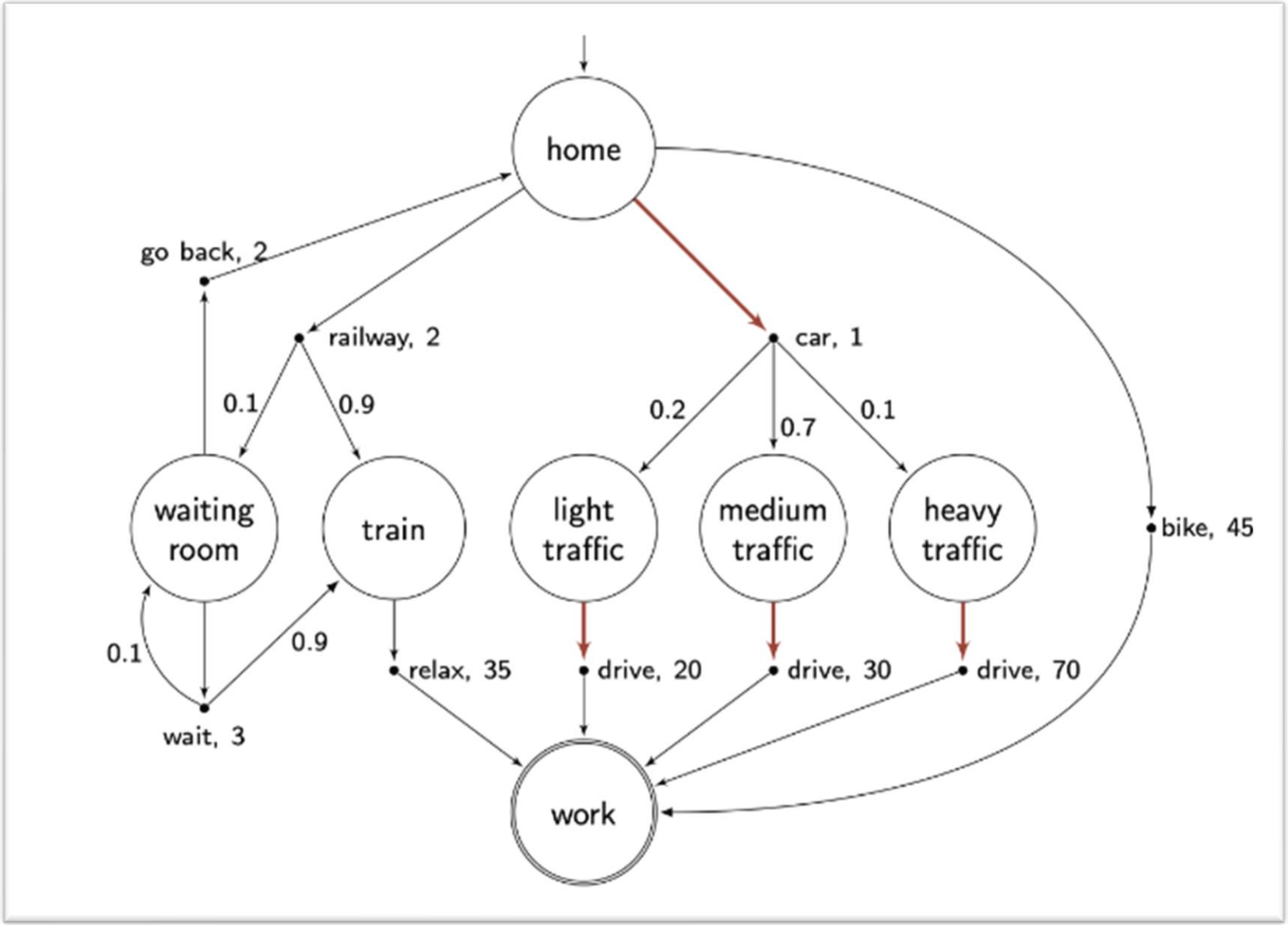
- Selects arms with large $\hat{\mu}_k(t)$ or small $n_k(t)$
- Confidence set shrinks as $n_k(t)$ increases



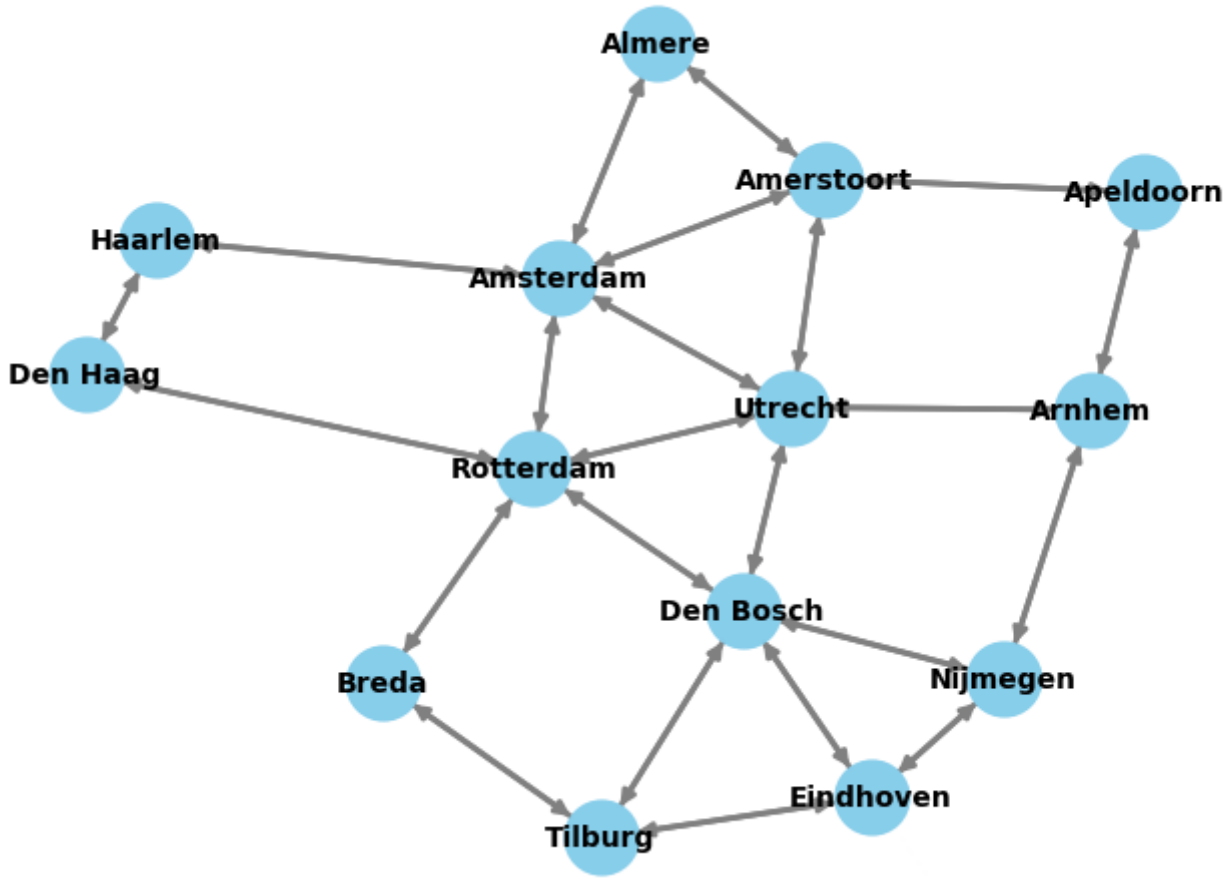
By Lucas Jacobs and Martin Stefanov

MDP

- Home to work
- Railway
- Car
- Bike

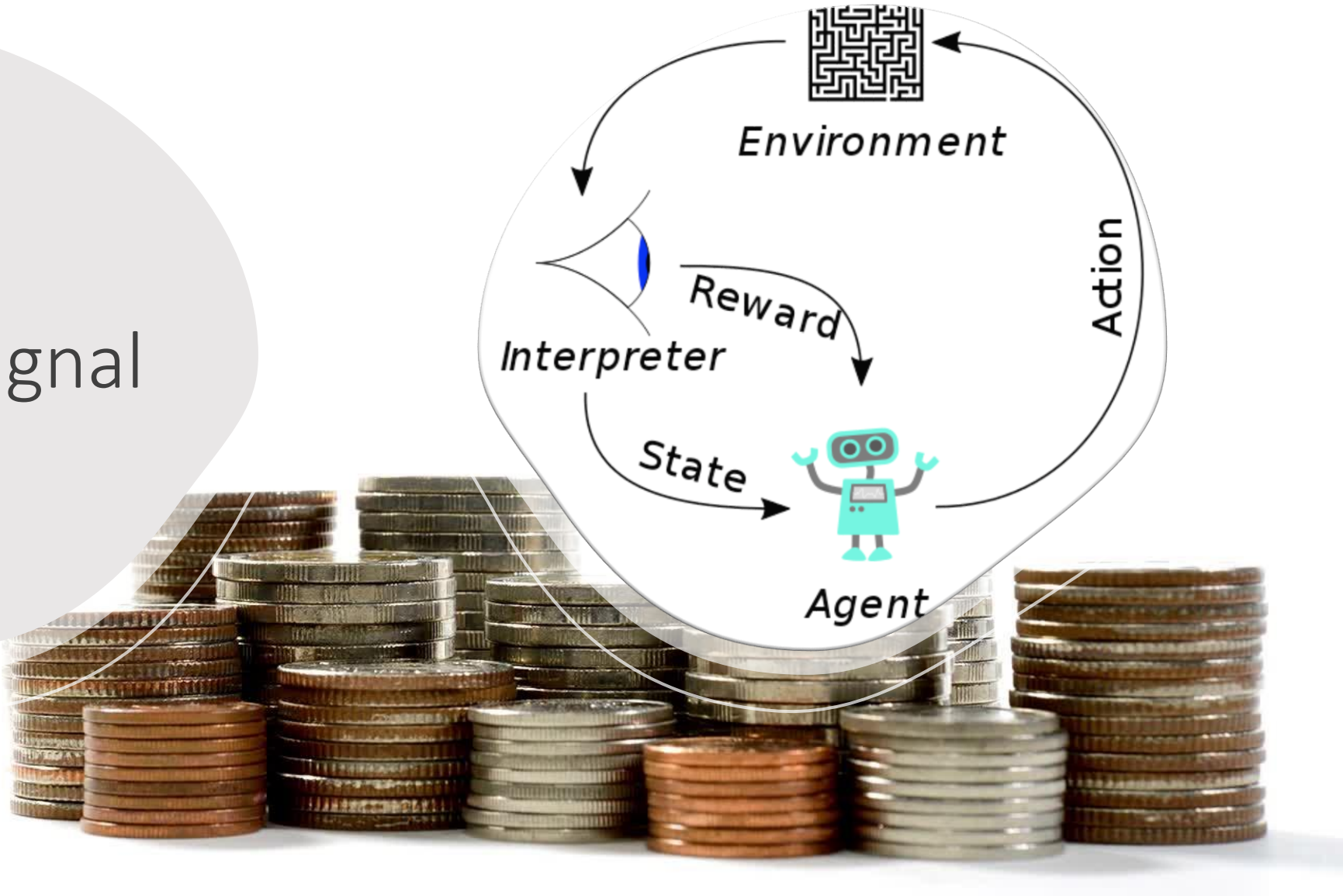


By Jakub Pawlak and Paul Jutte




```
import networkx as nx
```

Reward Signal



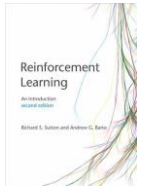
The Return

At any moment in time, we define the **return** as sum of all the rewards in the future to be as large as possible.

$$\boldsymbol{G_t = R_{t+1} + R_{t+2} + R_{t+3} + \cdots + R_T}$$

Our objective is maximizing the **expected** return $\mathbb{E}[G_t]$.

See appendix of expectations.



Sutton and Barto Book

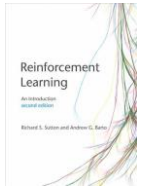
Example (Return)

Suppose the sequence below is generated by an agent interacting with an MDP.

$$\boldsymbol{S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, S_3, A_3, R_4, S_4, A_4, R_5, S_5, A_5}$$

At time $t = 2$, the return for the agent is: $G_2 = R_3 + R_4 + R_5$

And at time $t = 4$, the return for the agent is: $G_4 = R_5$



Sutton and Barto Book

The discount Factor

Often we have a different valuation for short-term and long-term gains.

We can do this by introducing a ***discount factor***

$$0 < \gamma < 1$$

for rewards that come later by.

$$\boldsymbol{G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T}$$

Smaller gamma $\gamma \to 0$: focus on short-term

Larger gamma $\gamma \to 1$: focus on long-term



Return example 2 (Slide 1/2)

Suppose the sequence below is generated by an agent interacting with an MDP:

$$S_0, A_0, R_1, S_1, A_1, \cdots, R_5, S_5, A_5$$

Based on the definition of the return we have:

$$G_2 = R_3 + \gamma R_4 + \gamma^2 R_5$$
$$G_3 = R_4 + \gamma R_5$$

Return example 2 (Slide 2/2)

With these relations:

$$G_2 = R_3 + \gamma R_4 + \gamma^2 R_5$$
$$G_3 = R_4 + \gamma R_5$$

we can conclude the following recursion:

$$G_2 = R_3 + \gamma G_3$$

This property is the basis of most of the algorithms we study in this course.

Goal

Given the return:

$$\boldsymbol{G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T}$$

Our goal is to maximize the **expected return** given our state

$$V(s) = \mathbb{E}[G_t | S_t = s\,]$$

$V(s)$ represents the return the agent **expects** to receive in the future while at state s.

Reminder

- At this point you need to know how **conditional expectations** such as

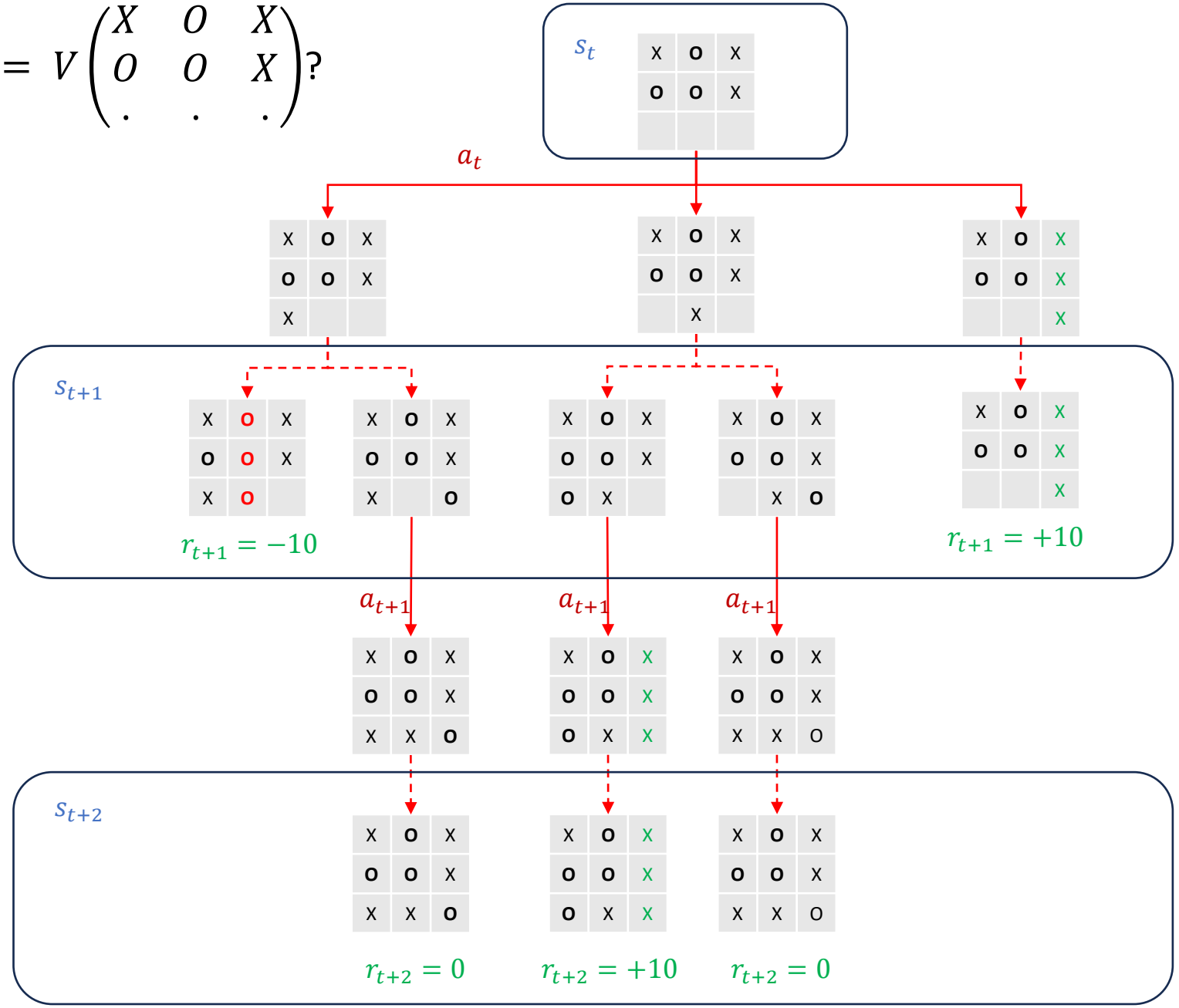
$$\mathbb{E}[G_t|S_t = s\ ]$$

Work.

If you need to remind your stats, please see the appendix on expectations.

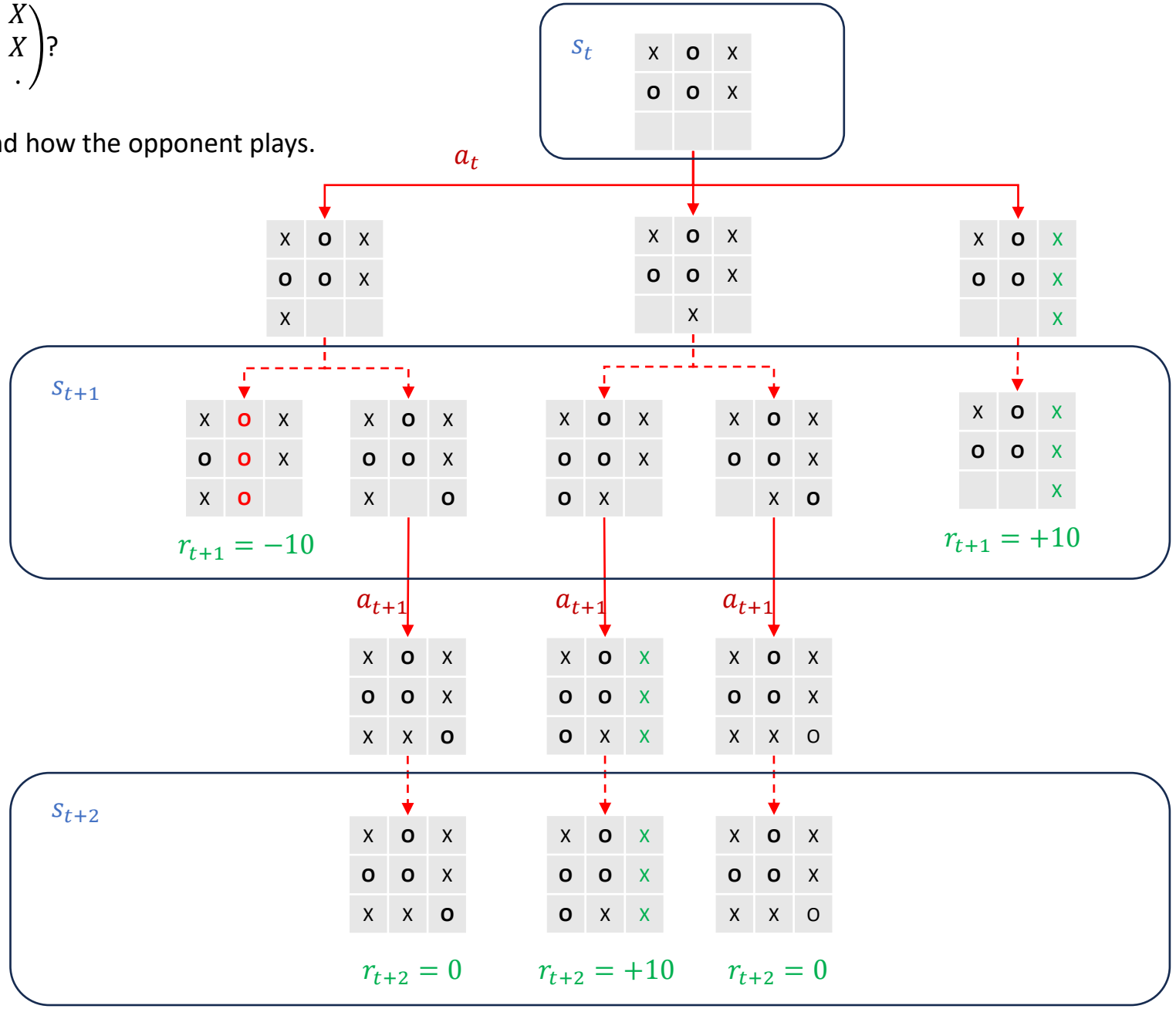
If you want to understand conditioning you please study the appendix on probabilities.

What is the value function $V(s_t) = V\begin{pmatrix} X & O & X \\ O & O & X \\ . & . & . \end{pmatrix}$?



What is the value function $\mathrm{V}(\mathrm{s_t}) = V\begin{pmatrix} X & O & X \\ O & O & X \\ . & . & . \end{pmatrix}$?

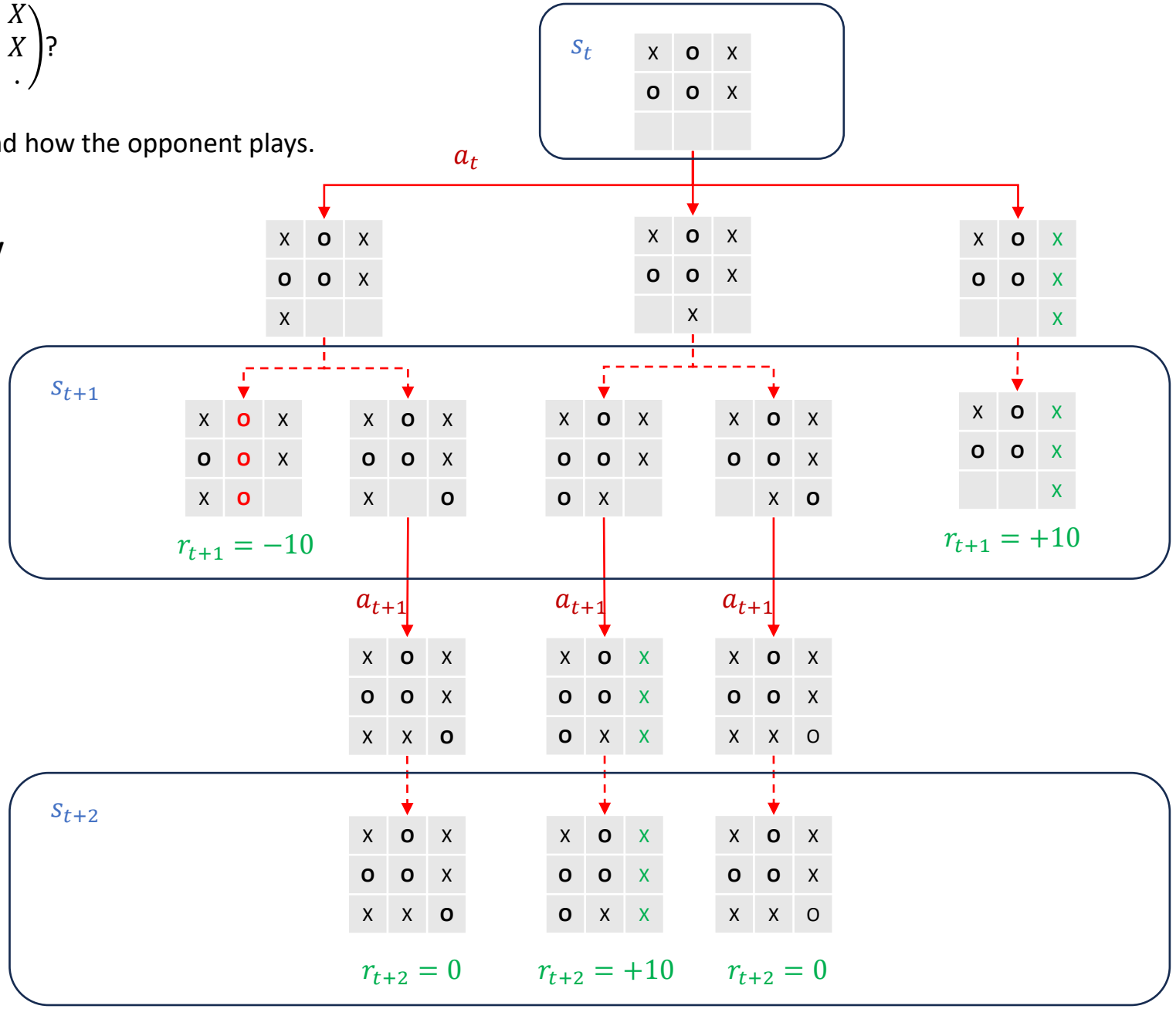
The answer depends on how the agent plays and how the opponent plays.



What is the value function $V(s_t) = V\begin{pmatrix} X & O & X \\ O & O & X \\ . & . & . \end{pmatrix}$?

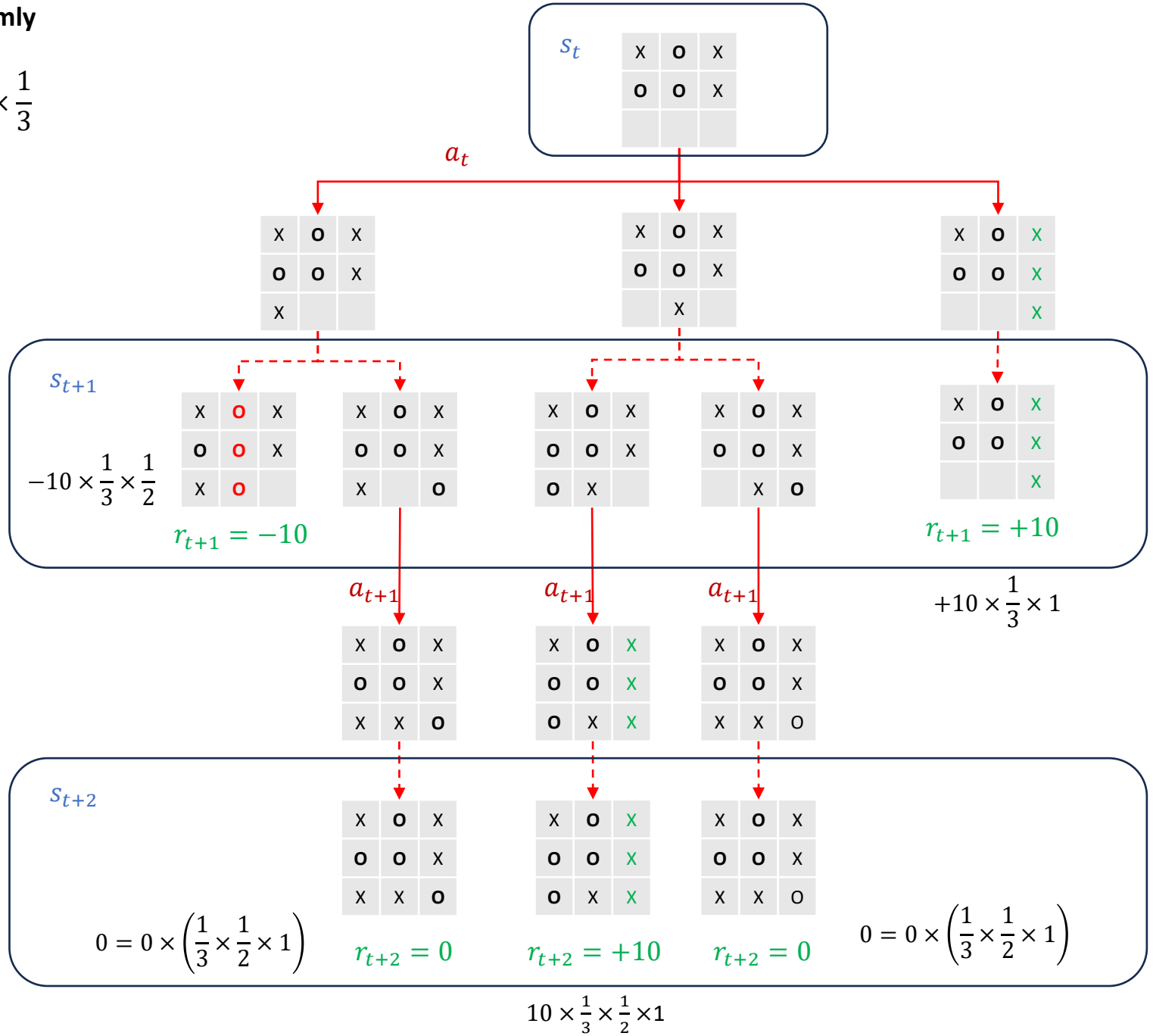
The answer depends on how the agent plays and how the opponent plays.

How the agent plays: **Policy**
How the opponent plays: **Transition Probability**

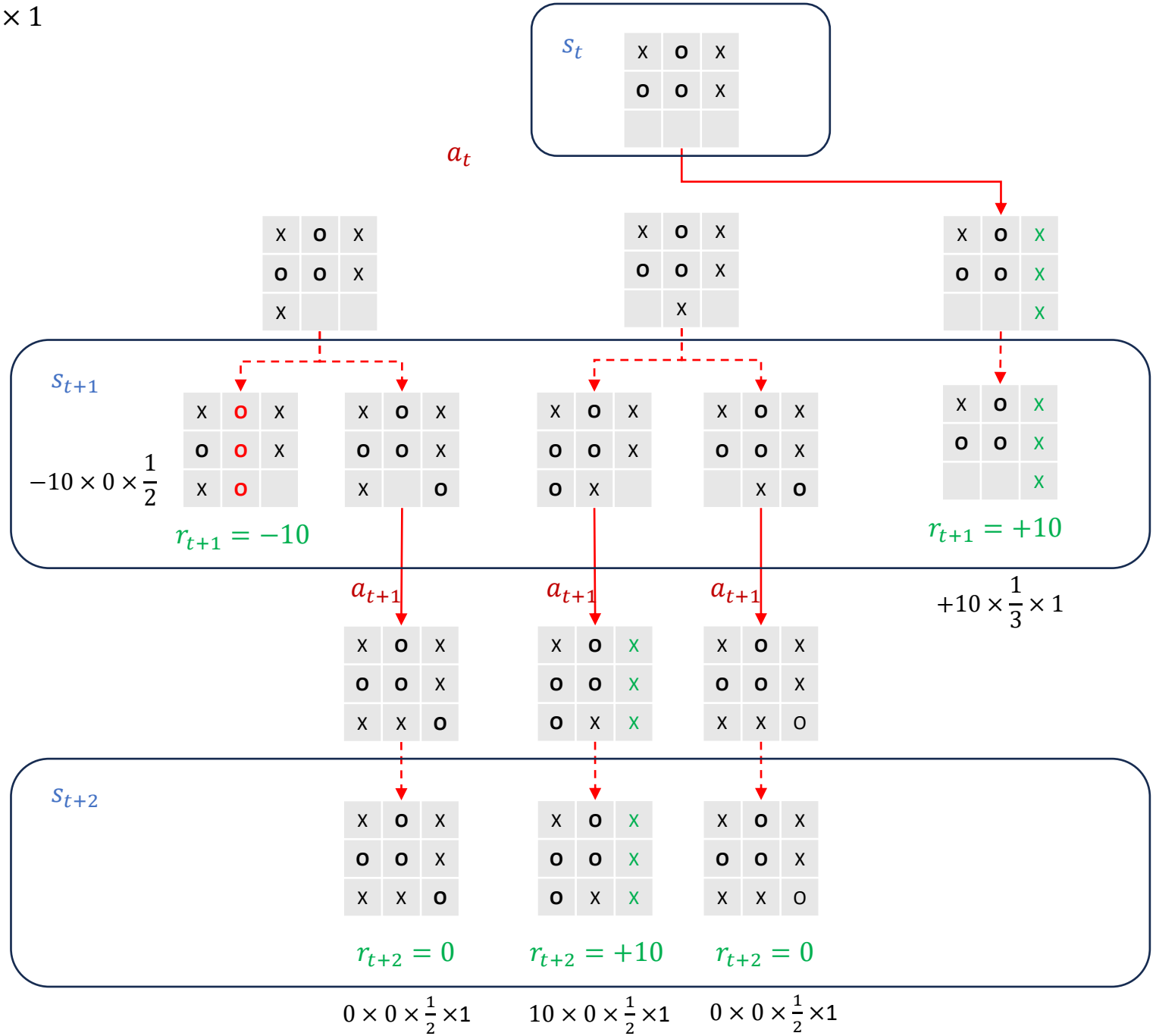


If both the agent and the opponent play **randomly**

$$V(s_t) = -10 \times \frac{1}{6} + 0 + 10 \times \frac{1}{6} + 0 + 10 \times \frac{1}{3}$$

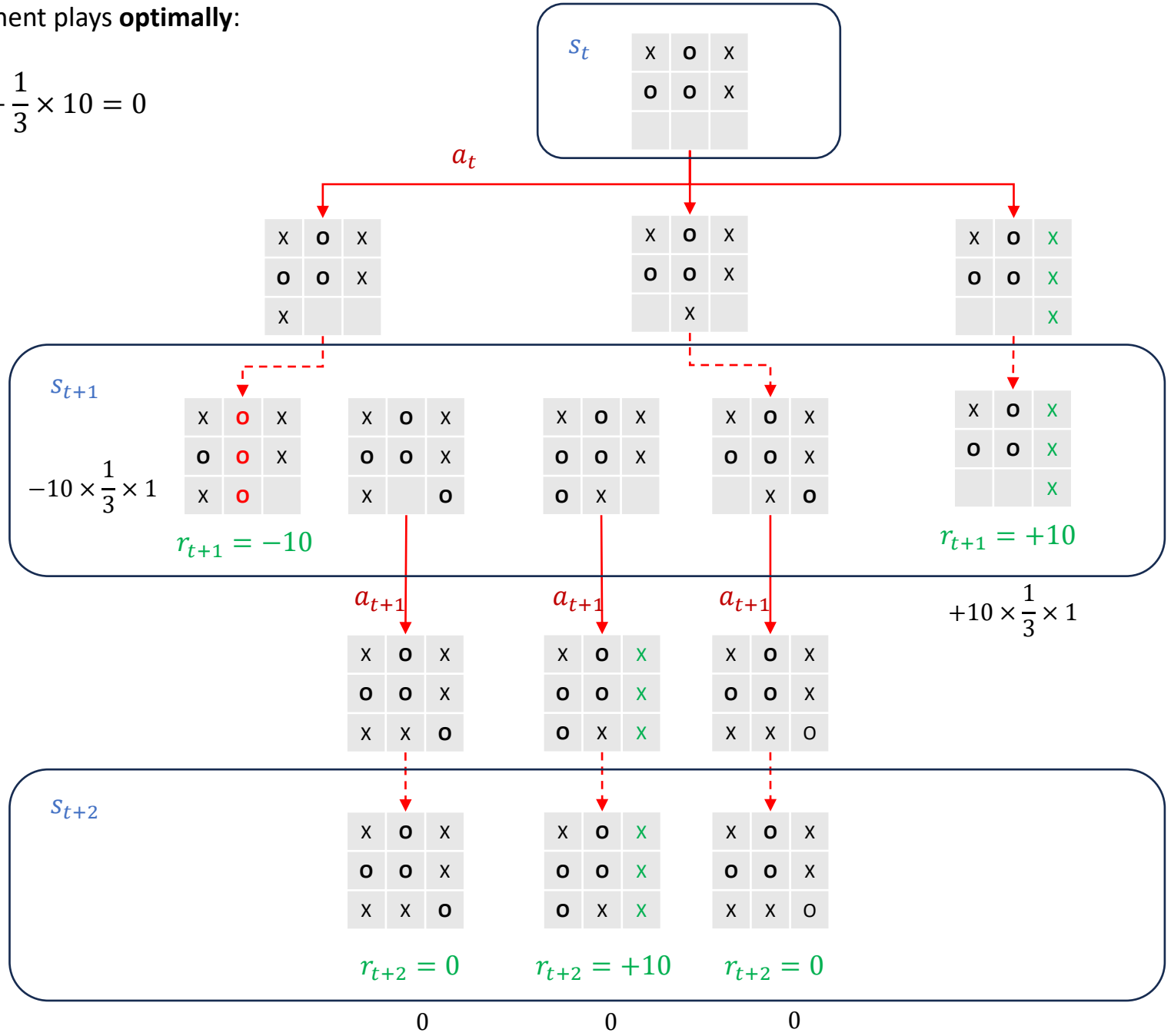


If the agent plays **optimally** $V(s_t) = +10 \times 1 \times 1$
Regardless of how the opponent plays.



If both the agent plays **randomly** and the opponent plays **optimally**:

$$V(s_t) = \frac{1}{3} \times 1 \times (-10) + 0 + 0 + 0 + \frac{1}{3} \times 10 = 0$$



Action-value function and Q-learning

Given the return:

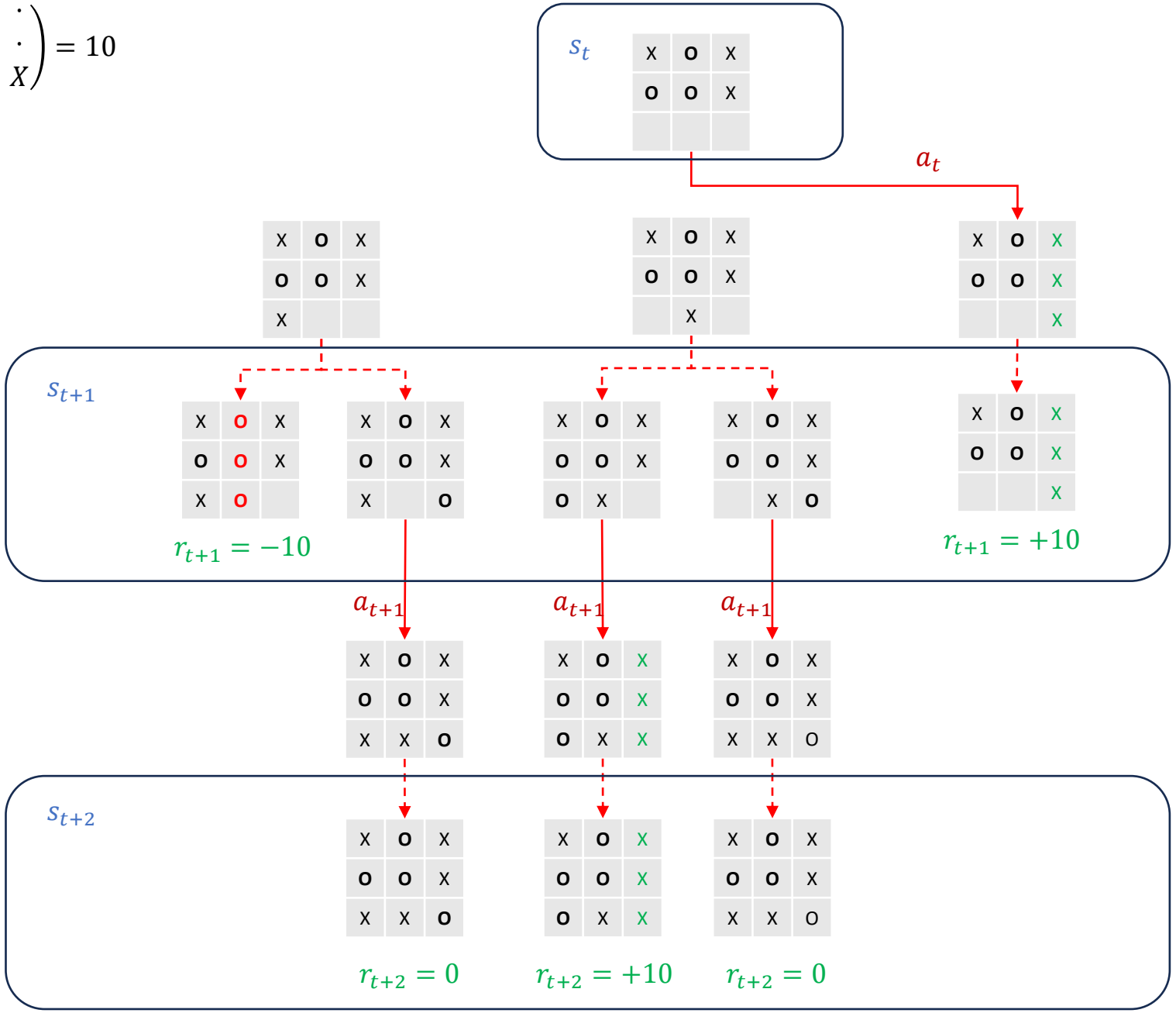
$$\boldsymbol{G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T}$$

To judge the value of an action given the circumstances, We define the state-action quality as the **expected** return given the state and action pair

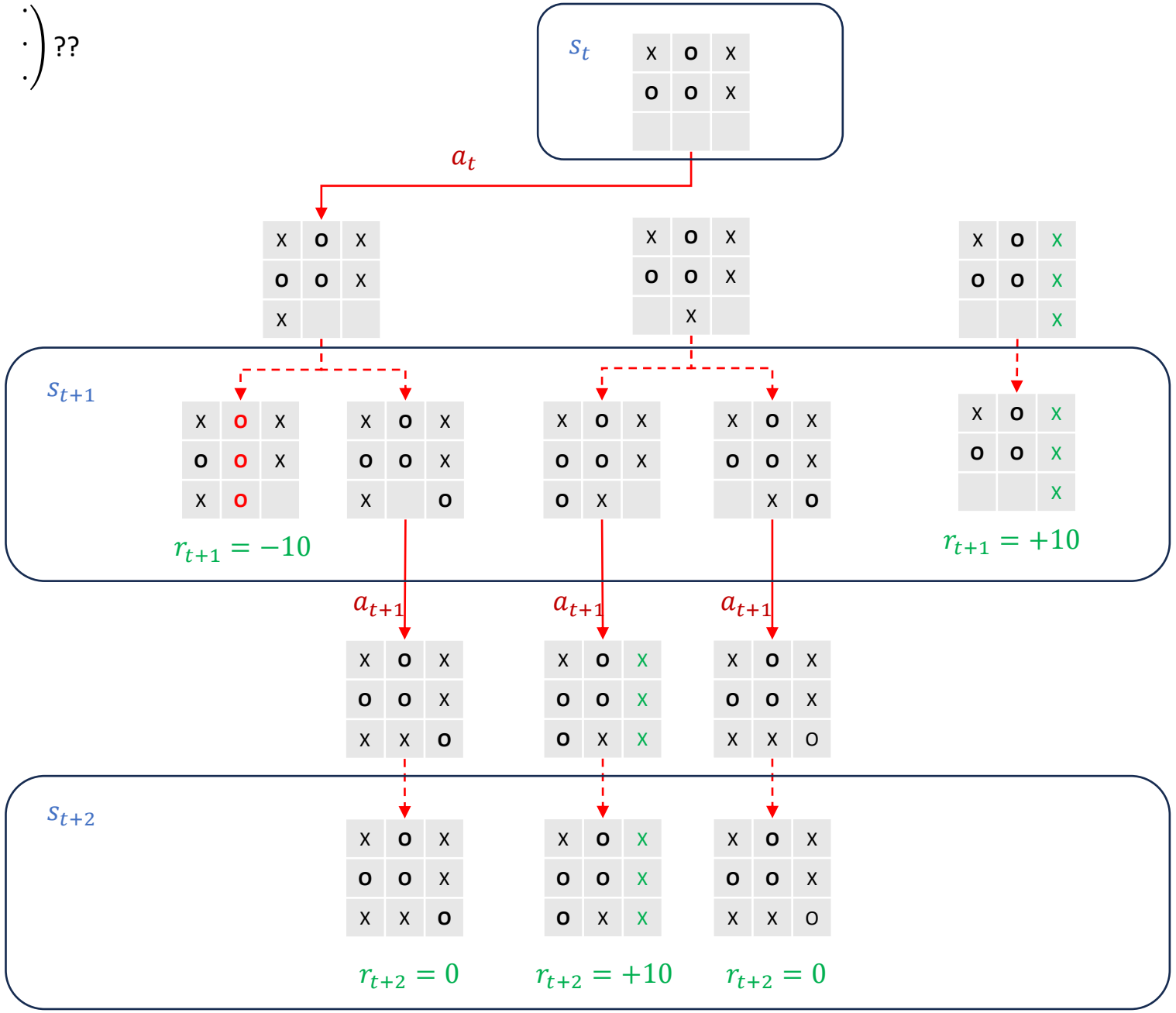
$$Q(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a\ ]$$

Q represents what the agent expects to receive at time if it decides to take action a while at state s

The Q function $Q\left(s_t = \begin{matrix} X & O & X \\ O & O & X \\ \cdot & \cdot & \cdot \end{matrix}, a_t = \begin{matrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & X \end{matrix}\right) = 10$

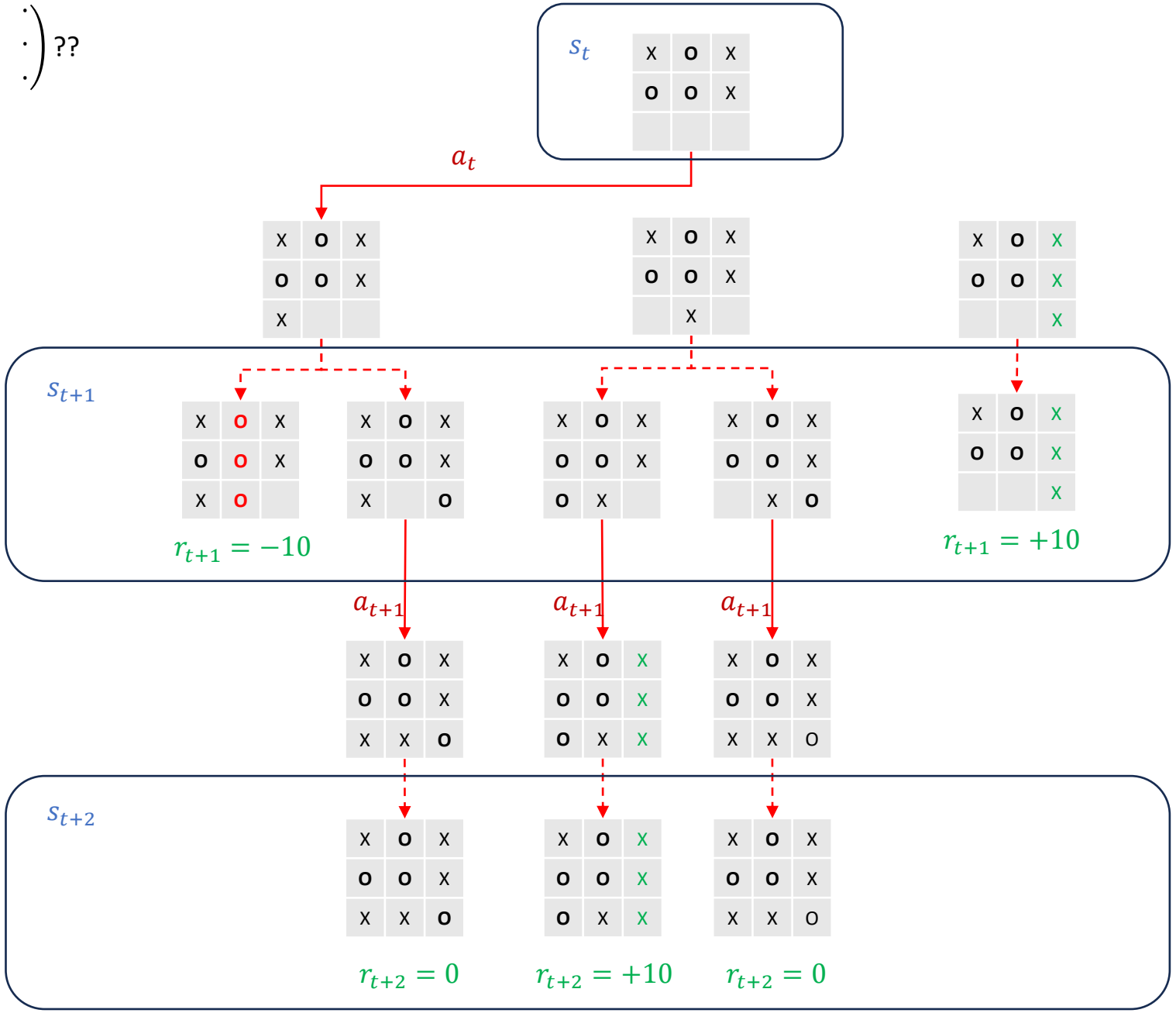


The Q function $Q\left(s_t = \begin{matrix} X & O & X \\ O & O & X \\ \cdot & \cdot & \cdot \end{matrix}, a_t = \begin{matrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ X & \cdot & \cdot \end{matrix}\right)$??



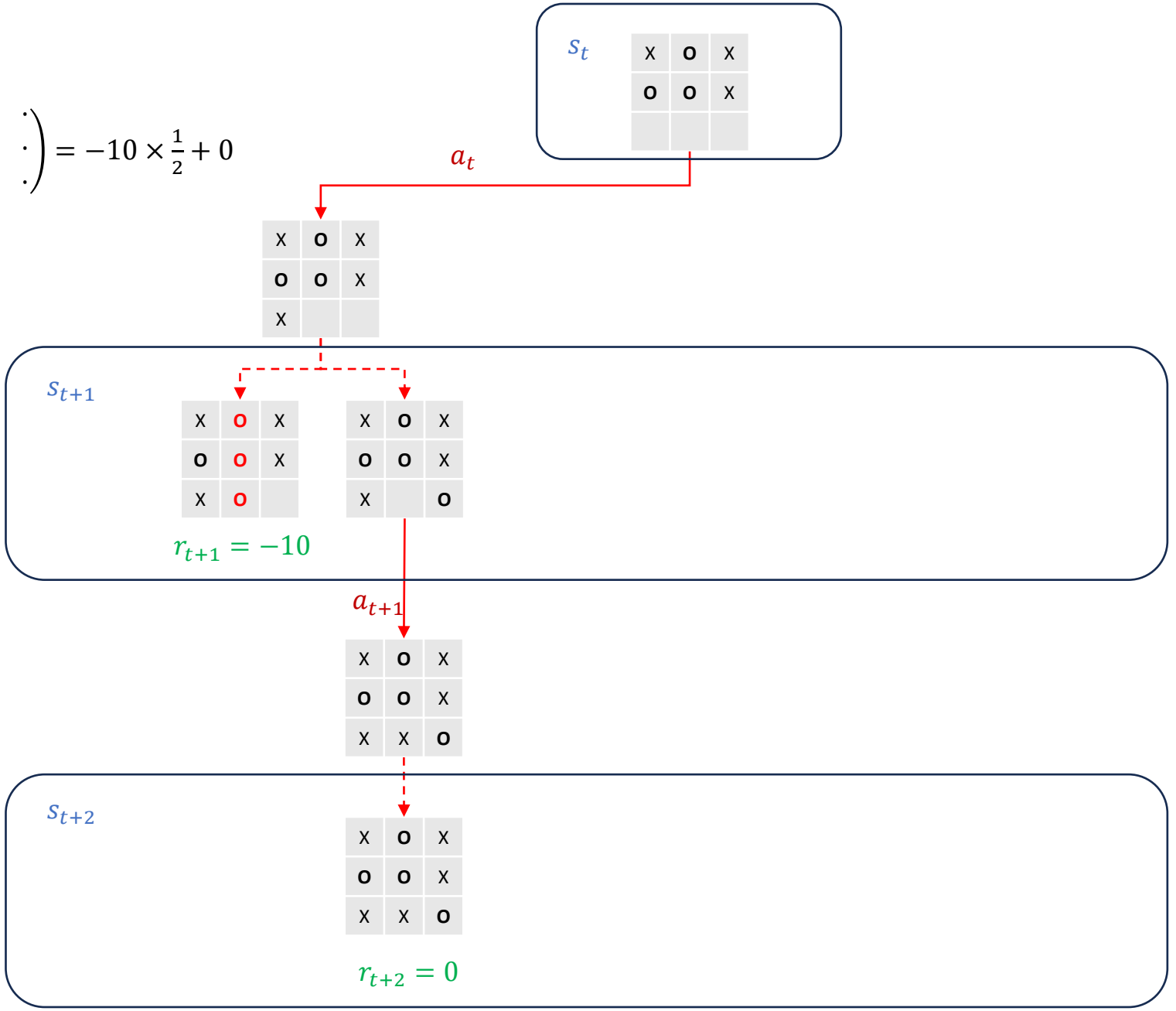
The Q function $Q\left(s_t = \begin{matrix} X & O & X \\ O & O & X \\ \cdot & \cdot & \cdot \end{matrix}, a_t = \begin{matrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ X & \cdot & \cdot \end{matrix}\right)$??

It depends on the opponent!



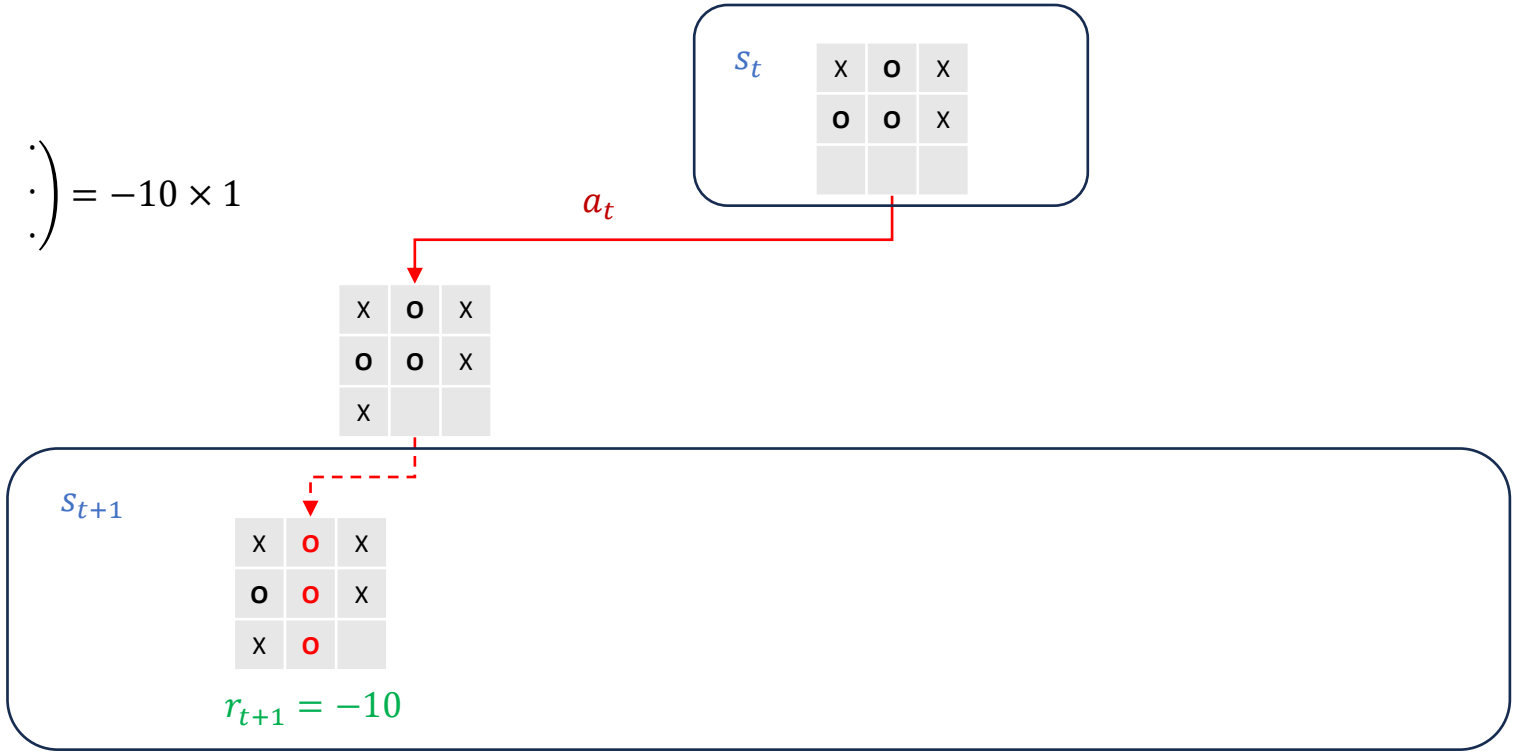
For a **random** opponent

The Q function $Q\left(s_t = \begin{matrix} X & O & X \\ O & O & X \\ \cdot & \cdot & \cdot \end{matrix}, a_t = \begin{matrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ X & \cdot & \cdot \end{matrix}\right) = -10 \times \frac{1}{2} + 0$

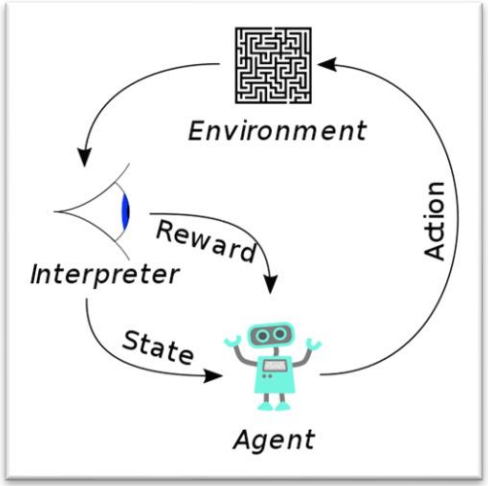


For an optimal opponent

The Q function $Q\left(s_t = \begin{matrix} X & O & X \\ O & O & X \\ . & . & . \end{matrix}, a_t = \begin{matrix} . & . & . \\ . & . & . \\ X & . & . \end{matrix}\right) = -10 \times 1$



Expected Future Reward or Quality Q is state and action dependent



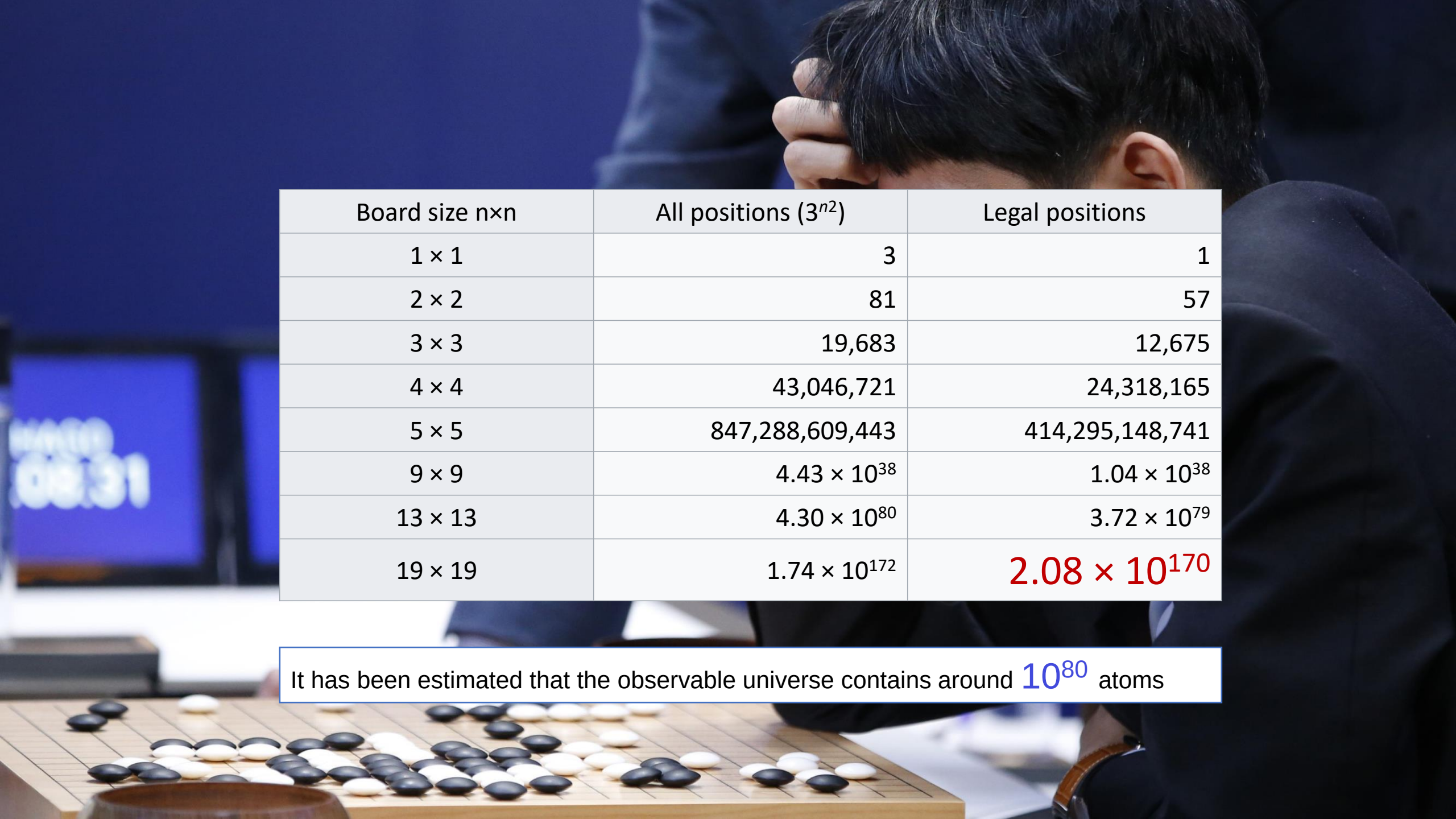
	action 1	action 2	action 3
state 1	0,11	0,51	0,94
state 2	0,09	0,09	0,11
state 3	0,96	0,10	0,30
state 4	0,14	0,21	0,34

In typical scenarios we must learn the unknown Q function

If only your agent had a table with all the average reward for each state-action pair (or the quality)



	action 1	action 2	action 3
state 1	?	?	?
state 2	?	?	?
state 3	?	?	?
state 4	?	?	?



Board size n×n	All positions (3^{n^2})	Legal positions
1 × 1	3	1
2 × 2	81	57
3 × 3	19,683	12,675
4 × 4	43,046,721	24,318,165
5 × 5	847,288,609,443	414,295,148,741
9 × 9	4.43×10^{38}	1.04×10^{38}
13 × 13	4.30×10^{80}	3.72×10^{79}
19 × 19	1.74×10^{172}	2.08×10^{170}

It has been estimated that the observable universe contains around 10^{80} atoms

Recursive Relation

For our MDP sequence:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, \ldots, R_T, S_T$$

Prove that the discounted cumulative reward have the structure

$$G_t = R_{t+1} + \gamma G_{t+1}$$

Recursion

Suppose the agent took action a_t at state s_t, it was then transitioned to state s_{t+1} and it received the reward r_{t+1}

Prove that for quality and value functions this relation holds:

$$Q_t(s_t, a_t) = \mathbb{E}[R_{t+1} | S_t = s_t, A_t = a_t\ ] + \gamma V_{t+1}(s_{t+1})$$

Why the last term is V and not Q? Explain the conditioning.

Optimal Agent

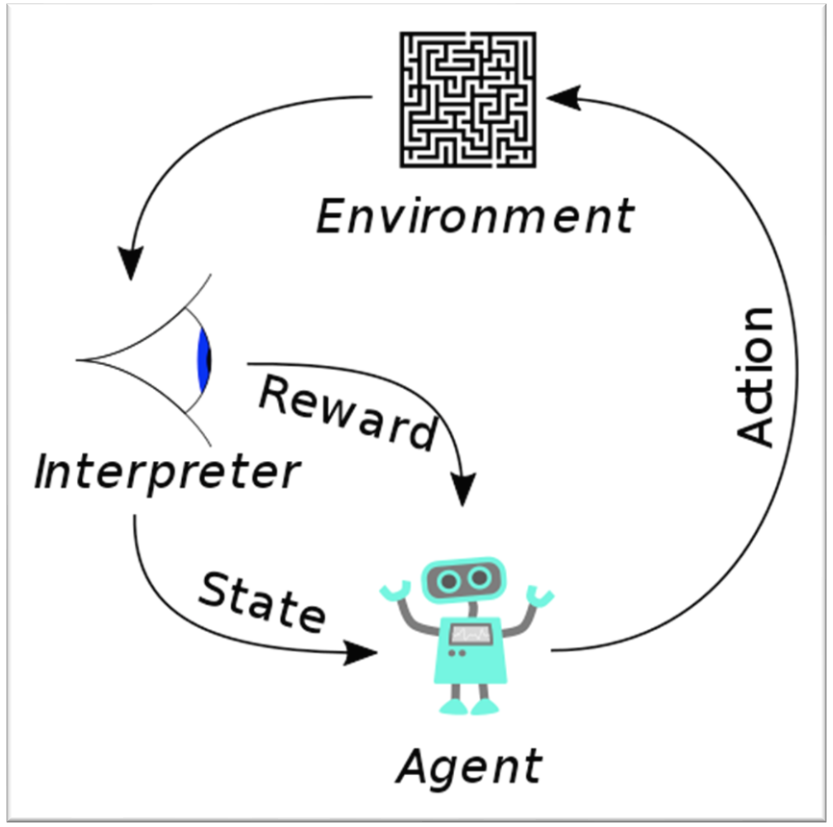
Prove that for an optimal agent

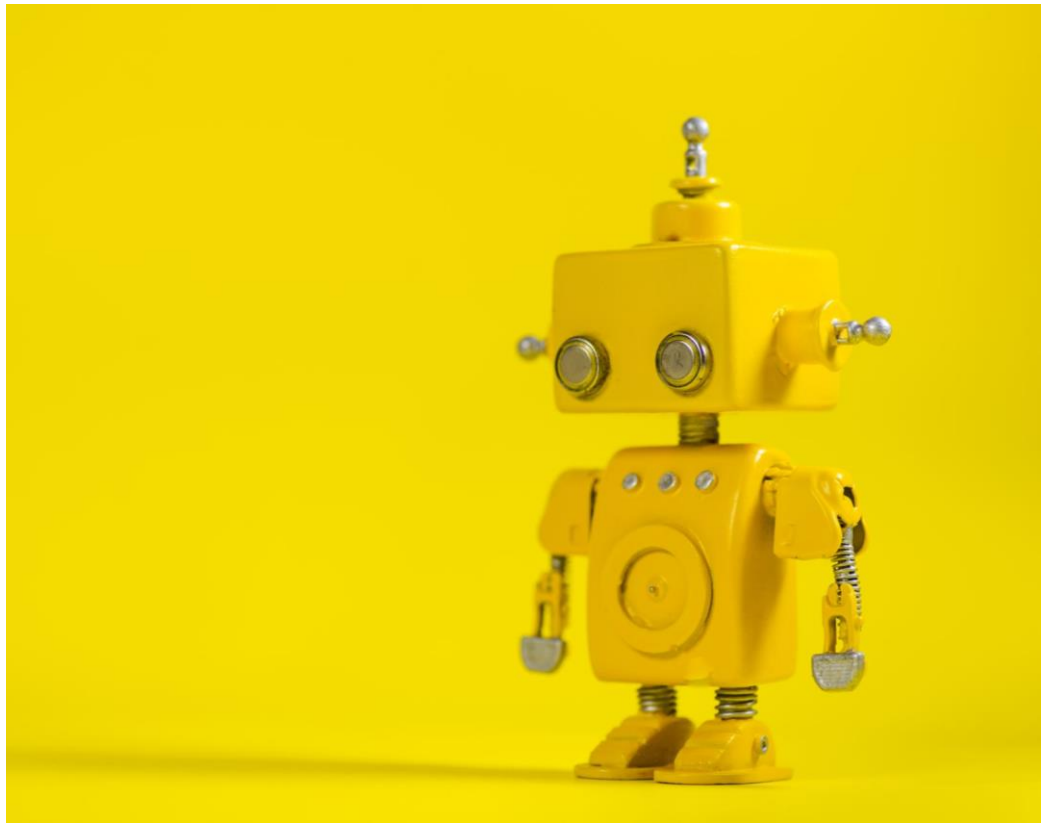
$$V(s_t) = \max_{a_t} Q_t(s_t, a_t)$$

and optimal decision is:

$$a^* = \arg\max_{a_t} Q_t(s_t, a_t)$$

The optimal agent knows the Q values, it does not need to learn it, thus there is no exploration.





Agent (Decision Maker)

In typical scenarios we have to learn the Q function



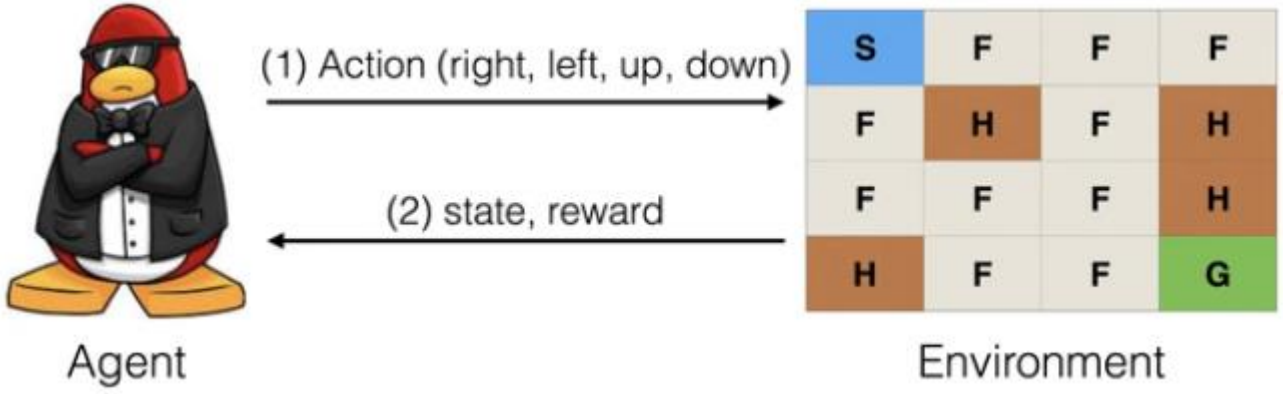
	action 1	action 2	action 3
state 1	?	?	?
state 2	?	?	?
state 3	?	?	?
state 4	?	?	?



The agent does not initially have an overview, it must experience and find out

Example: Slippery Frozen Lake

Frozen Lake - Gymnasium Documentation (farama.org)



The agent starts at **S** and receives a reward when reaching **G**. If it falls into hole **H** the episode ends (and it may start over)

The agent does NOT have this overview initially, it must figure out the map by experimenting.

Explore-Exploit Trade-offs



GATHER NEW INFORMATION



LEVERAGING EXISTING KNOWLEDGE.

Use-cases



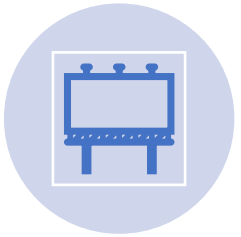
RESTAURANTS: CHOOSING BETWEEN A FAVORITE PLACE AND A NEWLY OPENED RESTAURANT.



MUSIC: OPTING FOR A FAVORITE ALBUM OR EXPLORING NEW MUSIC.



SOCIAL LIVES: BALANCING TIME WITH CLOSE CIRCLE FRIENDS AND FOSTERING NEW ACQUAINTANCESHIPS.

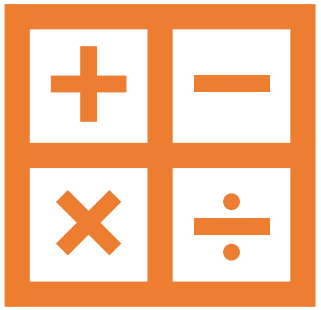


BUSINESS: DECIDING BETWEEN RUNNING A PROVEN AD WITH HIGH CLICK RATES OR TRYING NEW, UNTESTED ADS.

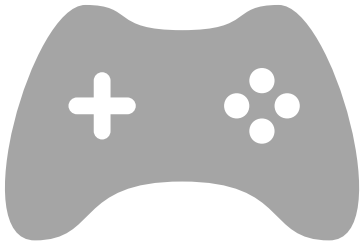


MEDICINE: WEIGHING THE BEST-KNOWN TREATMENT AGAINST EXPERIMENTAL OPTIONS IN CLINICAL TRIALS.

A good policy is a balance



Use your knowledge to get rewards



Play and find out

The formula to update the estimate

- We start from some random initial estimate of the Q function.
- We play the game, we observe the rewards and states

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, \dots, R_T, S_T$$

- We use these observations to update our estimate of Q based on some scary update formula.
- Our estimate keeps improving as we play many, many times.
- As our estimate of the Q improves, we start to make better decisions. Why?

Recall

Suppose the agent took action a_t at state s_t, it was then transitioned to state s_{t+1} and it received the reward r_{t+1}

$$Q_t(s_t, a_t) = \mathbb{E}[r_{t+1}] + \gamma V_{t+1}(s_{t+1})$$

For an optimal agent that knows the Q function, the value function is

$$V(s_t) = \arg\max_{a_t} Q_t(s_t, a_t)$$

Estimation of Q

Recall we do not have access to Q

The idea is to **estimate** Q by some variable $\hat{Q}$, start from "some" initial estimate and iteratively improve it.

The idea of Q-learning is that we want our estimate to meet the criteria

$$\hat{Q}(S_t, A_t) = R_{t+1} + \gamma \hat{Q}(S_{t+1}, \tilde{a}_t)$$

But in reality there is always a difference, and that difference is what we use to update $\hat{Q}$.

Remember: $\hat{Q}$ is an imperfect estimate of Q

Temporal Difference Error

Given $\hat{Q}$ our current approximation to Q, define the action:

$$\tilde{a}_t = \underset{\text{a}}{\text{argmax}}\, \hat{Q}(S_{t+1}, a)$$

and the target

$$Y_t = R_{t+1} + \gamma \hat{Q}(S_{t+1}, \tilde{a}_t)$$

The TD error is defined as:

$$\delta_j = Y_t - \hat{Q}(S_t, A_t)$$

Update $\hat{Q}(S_1, A_1)$ based on $Y_1 = R_2 + \gamma \hat{Q}\left(S_2, \underset{a}{\text{argmax}}\, \hat{Q}(S_2, a)\right)$

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, \ldots, R_T, S_T$$

Update $\hat{Q}(S_0, A_0)$ based on $Y_0 = R_1 + \gamma \hat{Q}\left(S_1, \underset{a}{\text{argmax}}\, \hat{Q}(S_1, a)\right)$

Q-learning is adjusting an estimate based on another estimate.

Update $\hat{Q}(\boldsymbol{S_1}, \boldsymbol{A_1})$ based on $\mathrm{Y}_1 = \boldsymbol{R_2} + \gamma \hat{Q}\left(\boldsymbol{S_2}, \underset{\mathrm{a}}{\operatorname{argmax}}\, Q(\boldsymbol{S_2}, a)\right)$

$$\boldsymbol{S_0}, \boldsymbol{A_0}, \boldsymbol{R_1}, \boldsymbol{S_1}, \boldsymbol{A_1}, \boldsymbol{R_2}, \boldsymbol{S_2}, \boldsymbol{A_2}, \ldots, \boldsymbol{R_T}, \boldsymbol{S_T}$$

Update$\hat{Q}(\boldsymbol{S_0}, \boldsymbol{A_0})$ based on $\mathrm{Y}_0 = \boldsymbol{R_1} + \gamma \hat{Q}\left(\boldsymbol{S_1}, \underset{\mathrm{a}}{\operatorname{argmax}}\, Q(\boldsymbol{S_1}, a)\right)$



Bootstrap

Image credit:Pinterest

Q-Learning is adjusting the current estimate based on the most optimistic future estimate

Update $\hat{Q}(\boldsymbol{S_1}, \boldsymbol{A_1})$ based on $\mathrm{Y}_1 = \boldsymbol{R_2} + \gamma \hat{Q}\left(\boldsymbol{S_2}, \underset{\mathrm{a}}{\operatorname{argmax}}\, Q(\boldsymbol{S_2}, a)\right)$

$$\boldsymbol{S_0}, \boldsymbol{A_0}, \boldsymbol{R_1}, \boldsymbol{S_1}, \boldsymbol{A_1}, \boldsymbol{R_2}, \boldsymbol{S_2}, \boldsymbol{A_2}, \ldots, \boldsymbol{R_T}, \boldsymbol{S_T}$$

Update$\hat{Q}(\boldsymbol{S_0}, \boldsymbol{A_0})$ based on $\mathrm{Y}_0 = \boldsymbol{R_1} + \gamma \hat{Q}\left(\boldsymbol{S_1}, \underset{\mathrm{a}}{\operatorname{argmax}}\, Q(\boldsymbol{S_1}, a)\right)$



Bootstrap

Image credit:Pinterest

Q-Learning is adjusting the current estimate based on the *most optimistic* future estimate

Update $\hat{Q}(\boldsymbol{S_1}, \boldsymbol{A_1})$ based on $\mathrm{Y}_0 = \boldsymbol{R_2} + \gamma \hat{Q}\left(\boldsymbol{S_2}, \underset{\mathrm{a}}{\operatorname{argmax}}\, Q(\boldsymbol{S_2}, a)\right)$

$$\boldsymbol{S_0}, \boldsymbol{A_0}, \boldsymbol{R_1}, \boldsymbol{S_1}, \boldsymbol{A_1}, \boldsymbol{R_2}, \boldsymbol{S_2}, \boldsymbol{A_2}, \ldots, \boldsymbol{R_T}, \boldsymbol{S_T}$$

Update$\hat{Q}(\boldsymbol{S_0}, \boldsymbol{A_0})$ based on $\mathrm{Y}_0 = \boldsymbol{R_1} + \gamma \hat{Q}\left(\boldsymbol{S_1}, \underset{\mathrm{a}}{\operatorname{argmax}}\, Q(\boldsymbol{S_1}, a)\right)$



Bootstrap

Image credit:Pinterest

Q-Learning is Off-line

Update $\hat{Q}(\boldsymbol{S_1}, \boldsymbol{A_1})$ based on $\mathrm{Y}_1 = \boldsymbol{R_2} + \gamma \hat{Q}\left(\boldsymbol{S_2}, \underset{\mathrm{a}}{\operatorname{argmax}}\, Q(\boldsymbol{S_2}, a)\right)$

$$\boldsymbol{S_0}, \boldsymbol{A_0}, \boldsymbol{R_1}, \boldsymbol{S_1}, \boldsymbol{A_1}, \boldsymbol{R_2}, \boldsymbol{S_2}, \boldsymbol{A_2}, \ldots, \boldsymbol{R_T}, \boldsymbol{S_T}$$

Update$\hat{Q}(\boldsymbol{S_0}, \boldsymbol{A_0})$ based on $\mathrm{Y}_0 = \boldsymbol{R_1} + \gamma \hat{Q}\left(\boldsymbol{S_1}, \underset{\mathrm{a}}{\operatorname{argmax}}\, Q(\boldsymbol{S_1}, a)\right)$



Bootstrap

Image credit:Pinterest

Greedy Policy

$$A_{t+1} = \underset{\mathrm{a}}{\operatorname{argmax}}\, \hat{Q}(S_t, a)$$

This policy ignores that $\hat{Q}$ is an imperfect estimate of Q



Don’t be Greedy

Or you will get stuck

exploration - exploitation



During the learning process, the agent needs to explore different actions and their outcomes to discover the optimal policy.



If the agent only chooses actions randomly, it may take a long time to converge to the optimal policy.



If the agent only chooses actions based on the current Q-values, it may get stuck in a suboptimal policy.



The epsilon-greedy strategy is used to balance between exploration and exploitation by **choosing a random action with a small probability** (epsilon) and choosing the action with the highest Q-value in the current state with probability (1-epsilon).

Epsilon Greedy method

With probability ϵ make a random move:

$$A_{t+1} = env.action.sample(\cdot)$$

Otherwise (With probability $1 - \epsilon$):

$$A_{t+1} = \underset{\mathrm{a}}{\operatorname{argmax}}\, \hat{Q}(S_t, a)$$

Hyper parameters



ε (Epsilon, greediness)



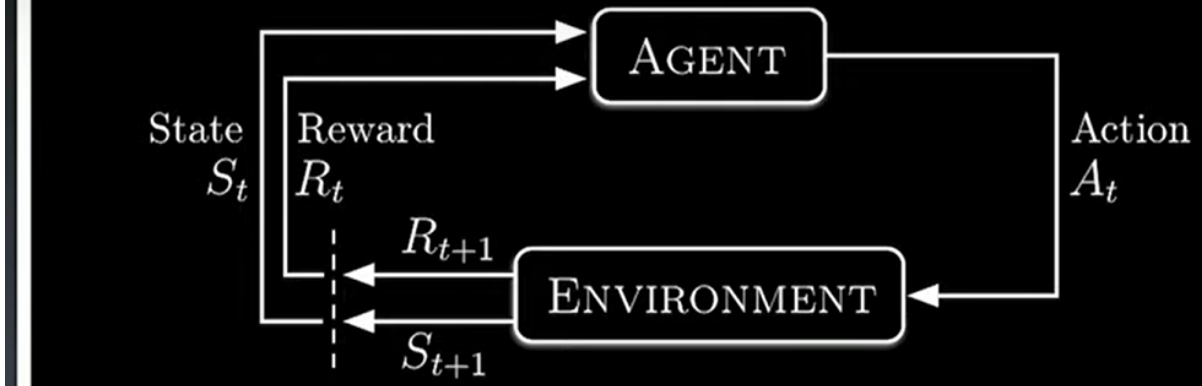
η/**α** (Eta/alpha, learning rate)



γ (Gamma, discount)

Recap Decision Theory

Finite Markov Decision Process



$s \in \mathcal{S}$ $a \in \mathcal{A}(s)$ $r \in \mathcal{R} \in \mathbb{R}$ $p(s', r|s, a)$

$\pi(a|s)$

$$G_t = \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$$

$$\max_{\pi} \mathbb{E}_{\pi}[G_t]$$

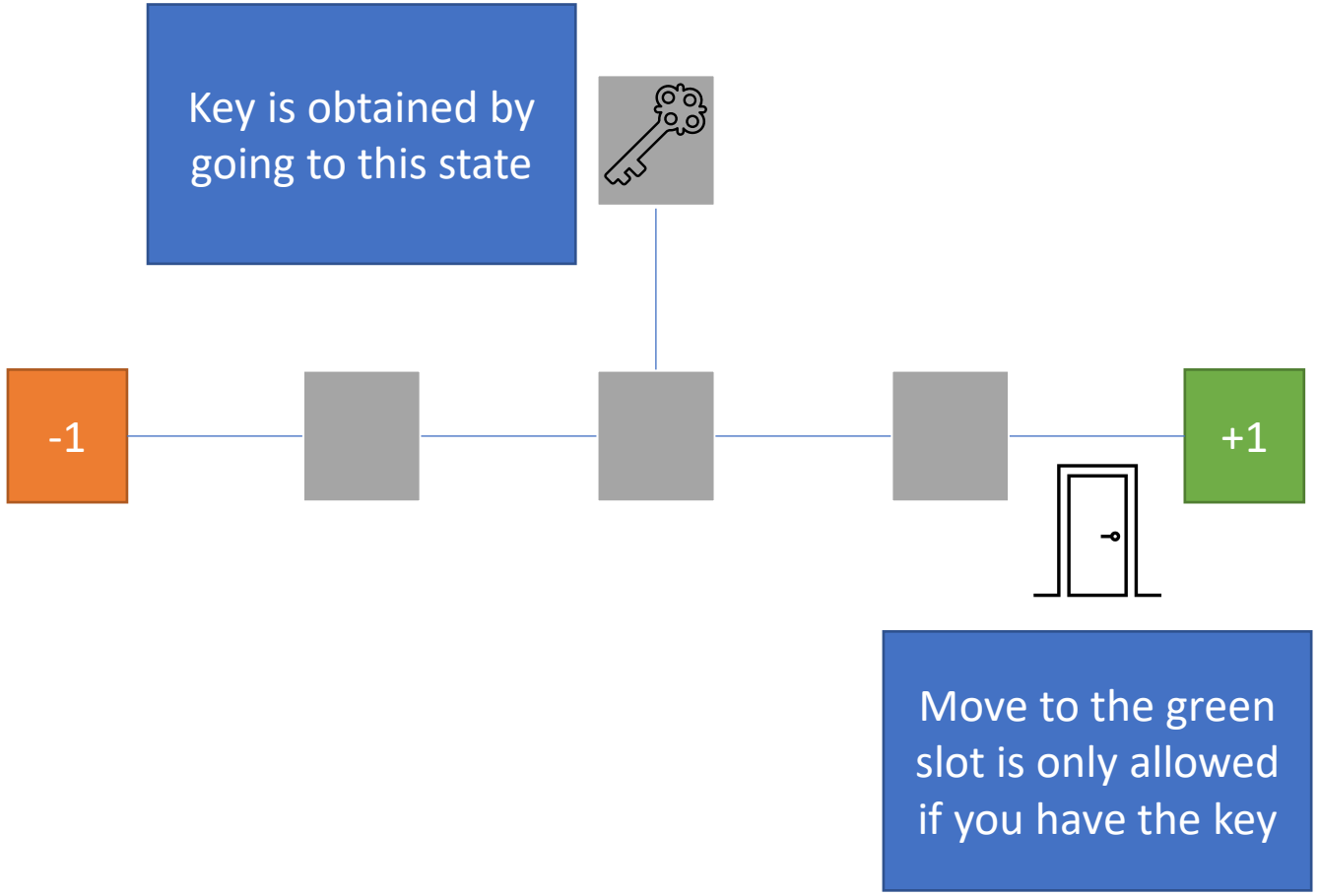
$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t|S_t = s]$$

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t|S_t = s, A_t = a]$$

$$v_*(s) = \max_{\pi} v_{\pi}(s)$$

$$q_*(s, a) = \max_{\pi} q_{\pi}(s, a)$$

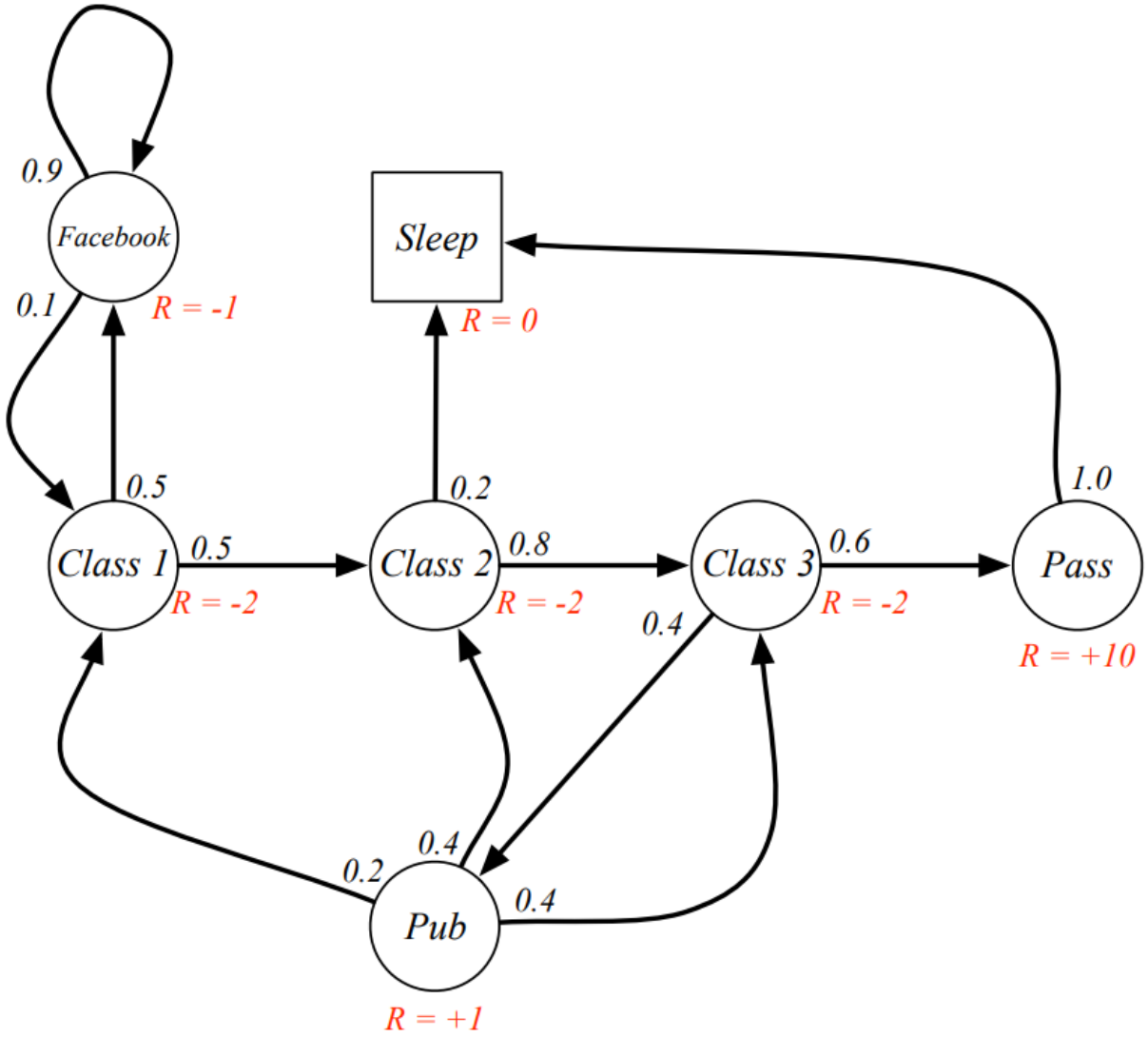
Write down the MDP formally.



What are the steps to implement this?

Assignment

- Write a code that models an MDP that corresponds to this (or an MDP of similar complexity representing your life!)
- Generate a few random episodes starting at C1.
- Interpret the probabilities as the initial policy $\pi(a|s)$.
- Find the optimal policy.
- Can you do this for your daily life?



[Lecture 2: Markov Decision Processes (stanford.edu)]

Dynamical Programming

Summary (Decision Theory)

- You have learned about the return G, the quality function Q and the state value function V.
- Our goal to maximize the **expected** return G
  - $Q(s, a)$ is the **expected** return **given** that we take action a in state s
  - $V(s)$ is the **expected** return **given** that we are in state s
- Both Q and V functions depend on the policy $P(a|s)$
- Our goal is to pick a policy that maximizes $V(s)$ for all states.
- The best action when in state s is the one that maximizes Q: $a^* = \underset{\mathrm{a}}{\operatorname{argmax}}\, Q(S_t, a)$

Summary (Statistics)

- In the previous slide we see two statistical concepts:
  - Conditional Probabilities (see slides here)
  - Expectations (see slides here)

How does Q-learning or Monte-Carlo treat Expectations?

How does Q-learning or Monte-Carlo treat Expectations?

- They estimate the averages based on iterative methods and data.

- They do not know about the MDP internal working, i.e. the transition probabilities.

Summary (Statistics)

Generally, you deal with expectations in one of two ways:

- Estimate the expectations based on the data (compute averages)
  - Q-learning, Monte-Carlo, …. treat the expectations this way
- Compute the expectations given the probabilities.
  - Dynamical Programming treats the expectations this way

Objective today

Goal: Given the known transition probabilities of an MDP, compute the optimal policy.

[Bellman Equations, Dynamic Programming, Generalized Policy Iteration | Reinforcement Learning Part 2 - YouTube]

Objective today

Goal: Given the known transition probabilities of an MDP, compute the optimal policy.

In other words, we now assume that the transition probabilities are known and we try to find the policy $P(a|s)$ that maximizes $V(s)$ for all states.

Objective today

Goal: Given the known transition probabilities of an MDP, compute the optimal policy.

In other words, we now assume that the transition probabilities are known and we try to find the policy $P(a|s)$ that maximizes $V(s)$ for all states.

What is the difference with Q-learning or Monte-Carlo?

Write the Q-function based on the transition probabilities.

$$Q(s,a) = \mathbb{E}[\boldsymbol{G_t}|S_t = s, A_t = a\ ]$$

Write the Q-function based on the transition probabilities.

$$Q(s,a) = \sum_{\acute{r},\acute{s}} P(\acute{r},\acute{s}|s,a)\mathbb{E}[G_t]$$

Note that both $\mathbb{E}[G_t]$ depends on the policy (see slides on the tic-tac-toe)

Note that we write $\mathbb{E}[G_t]$ and not G_t because future events are still uncertain.

The structure of the Q-Function

Remember:

$$\boldsymbol{G_t = R_{t+1} + \gamma G_{t+1}}$$

which means:

$$Q(s,a) = \mathbb{E}[\boldsymbol{R_{t+1} + \gamma G_{t+1}} | S_t = s, A_t = a\ ]$$

The structure of the Q-Function

$$Q(s,a) = \mathbb{E}[\boldsymbol{R_{t+1}} + \boldsymbol{\gamma G_{t+1}} | S_t = s, A_t = a\ ]$$

Thus

$$Q(s,a) = \sum_{\acute{r},\acute{s}} P(\acute{r},\acute{s}|s,a)\mathbb{E}[G_t]$$

Gives us

$$Q(s,a) = \sum_{\acute{r},\acute{s}} P(\acute{r},\acute{s}|s,a)(\boldsymbol{R_{t+1}} + \boldsymbol{\gamma}\mathbb{E}[\boldsymbol{G_{t+1}}])$$

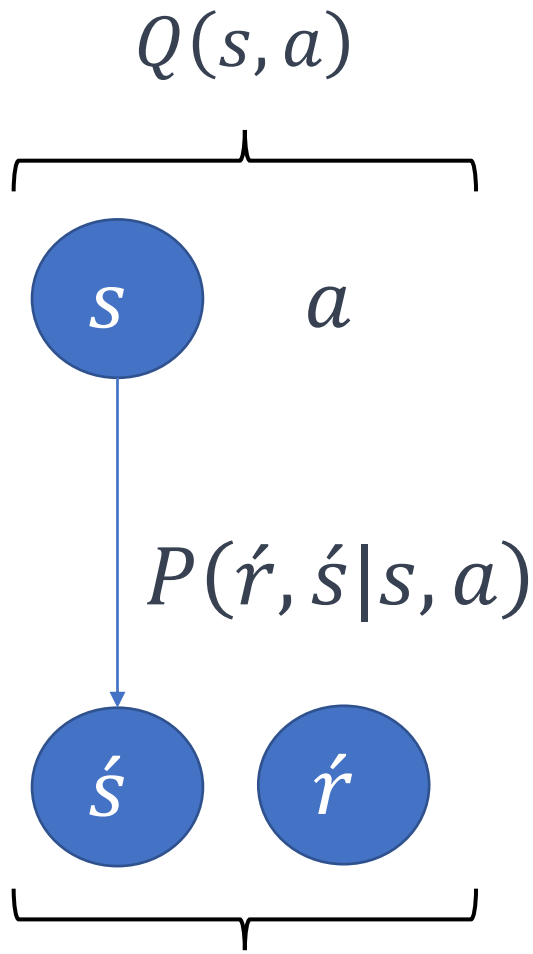
Transition Probabilities

Remember:

The transition probabilities are:

$$P(s', r|s, a)$$

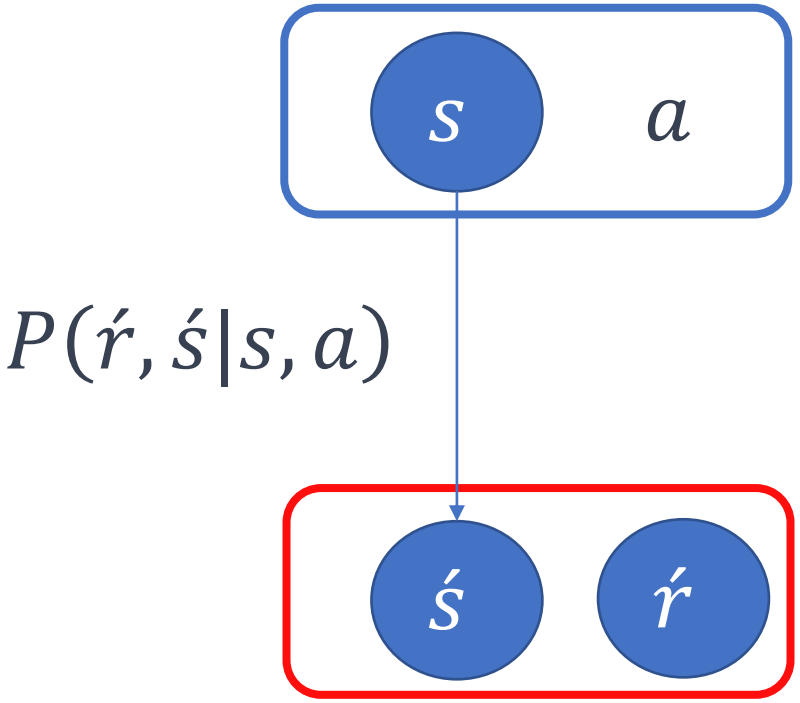
which is the probability of getting reward r and ending up in state s' after performing action a is state s.



$$\mathbb{E}[\acute{r}' + \gamma G_{t+1}|S_{t+1} = \acute{s}]$$

Putting it all together

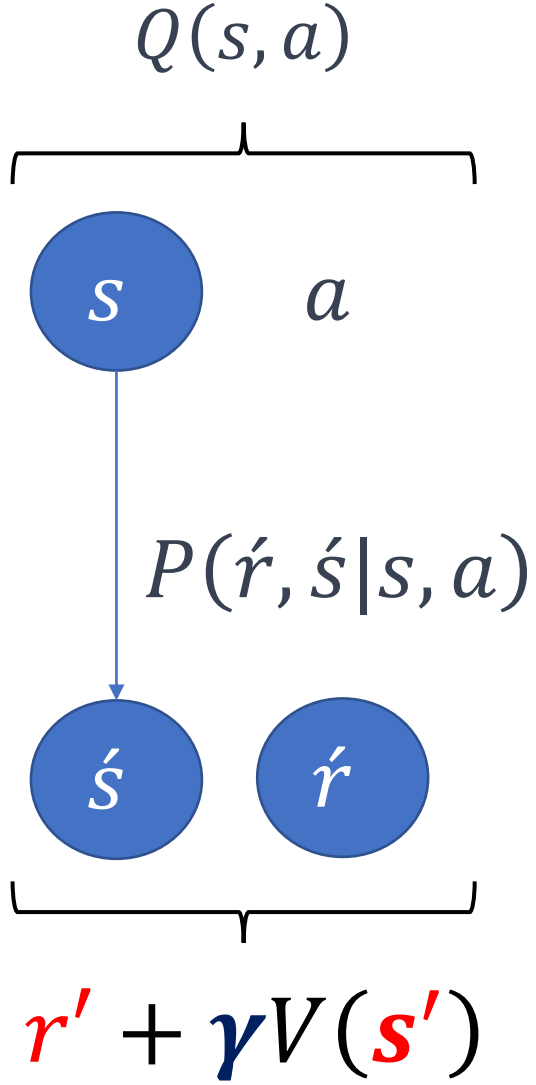
$$Q(s,a) = \sum_{\acute{r},\acute{s}} \left(P(\acute{r},\acute{s}|s,a) \left(\acute{r}' + \gamma \mathbb{E}[G_{t+1}|S_{t+1} = \acute{s}] \right) \right)$$



From Q-Function to value function

Based on the definition of the value function

$$Q(\boldsymbol{s}, a) = \sum_{\acute{r},\acute{s}} \left(P(\acute{r}, \acute{s}|s, a) \left(\acute{r} + \gamma V(\boldsymbol{\acute{s}}) \right) \right)$$



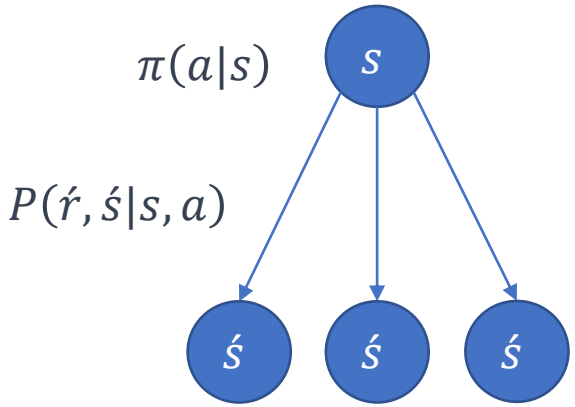
The structure of the Value-Function

It is easy to show that:

$$\boldsymbol{G_t = R_{t+1} + \gamma G_{t+1}}$$

which means:

$$V_\pi(s) = \mathbb{E}[\boldsymbol{R_{t+1} + \gamma G_{t+1}} | S_t = s\ ]$$



There are two sources of randomness we need to consider in the expectation, the agent policy and the **MDP transition**

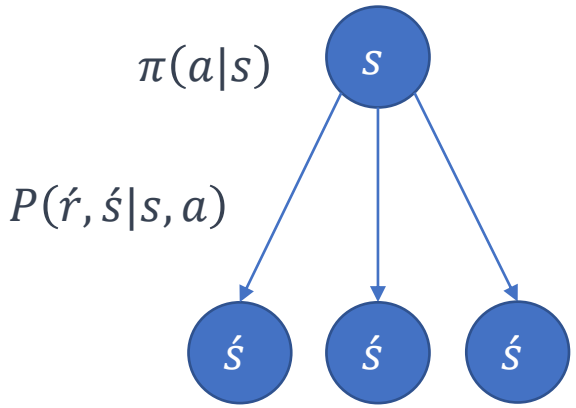
The structure of the Value-Function

There are two sources of randomness we need to consider in the expectation, the agent policy and the **MDP transition**

$$V_\pi(s) = \mathbb{E}[\boldsymbol{R_{t+1}} + \boldsymbol{\gamma G_{t+1}} | S_t = s\ ]$$

For the agent randomness in this formula, we can sum over all actions. In other words,

$$V_\pi(s) = \sum_a \pi(a|s)\, \mathbb{E}[\boldsymbol{R_{t+1}} + \boldsymbol{\gamma G_{t+1}} | S_t = s, A_t = a]$$



But the inner expectation is known to us, we identify as the quality function

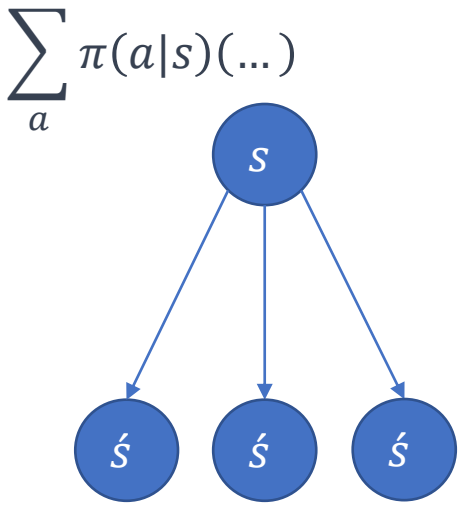
$$V_\pi(s) = \sum_a \pi(a|s)\, Q(s, a)$$

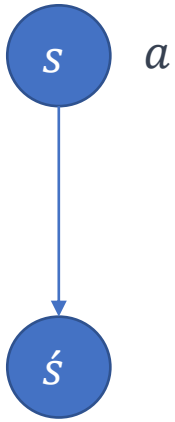
The Bellman Equations

For *any* policy π, all $s \in \mathcal{S}$ and all $a \in \mathcal{A}(s)$:

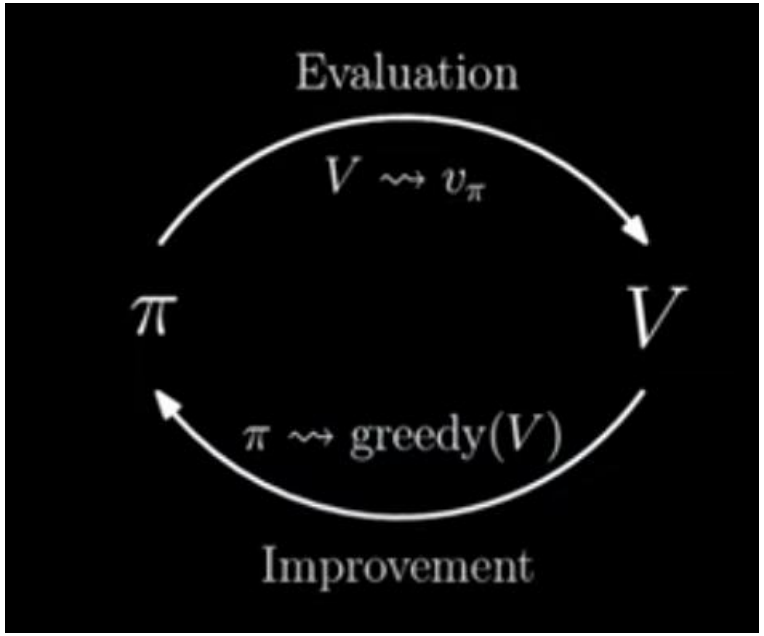
$$v_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a|s) q_\pi(s,a) \qquad (1)$$

$$q_\pi(s,a) = \sum_{\substack{s' \in \mathcal{S} \\ r \in \mathcal{R}}} p(s', r|s, a)[r + \gamma v_\pi(s')] \qquad (2)$$

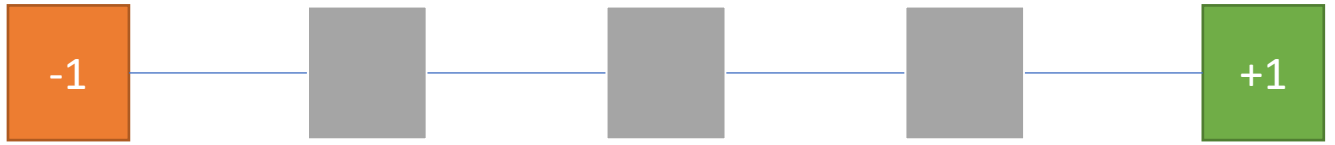




Generalized Policy Iteration



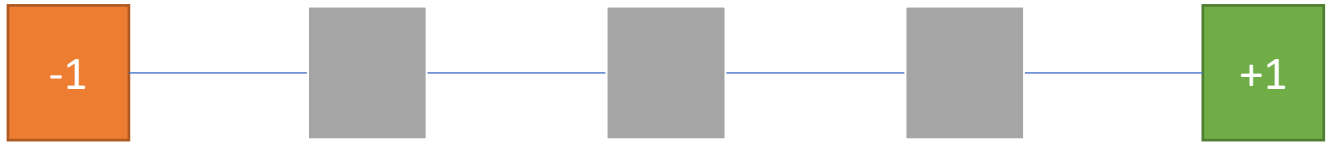
Perform GPI (set $\gamma = 0.1$, deterministic env.)



Terminal State

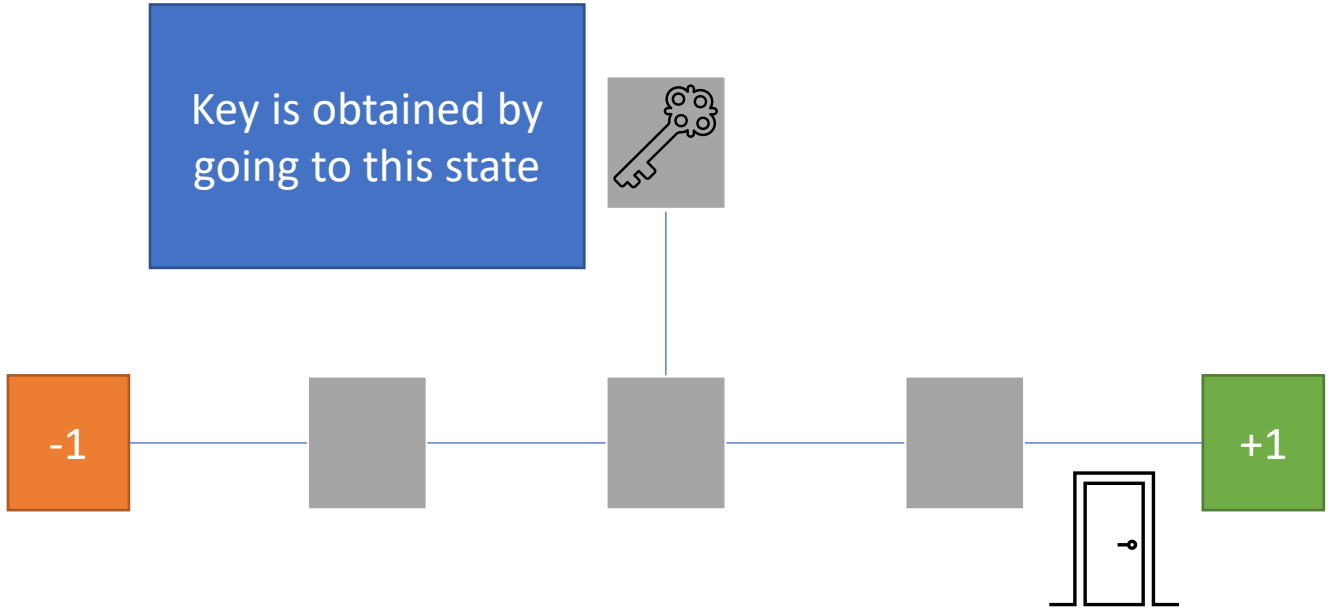
Terminal State

Perform GPI (set $\gamma = 0.1$, deterministic env.)



Now cost of living is -2

Perform GPI (set $\gamma = 0.1$, deterministic env.)



What are the steps to implement this?

Major Assumptions

We assume:

- The agent observes S_t.
- S_t has the MARKOV PROPERTY.

We *sometimes* assume:

- TABULAR: All states and actions can be listed.
- COMPLETE KNOWLEDGE: We have complete access to the environment's dynamics.

Do you recognize this slightly different formulation?

- The policy is given by:

$$\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r|s, a)\left[r + \gamma V(s')\right]$$

- What is the quantity in the green box?
- Why is this policy greedy?
- Why is s and s' in the formula? Why V(s') and not V(s)?

Policy Iteration

- Update Policy for a given value

$$\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r|s, a)\big[r + \gamma V(s')\big].$$

- Update the values

$$V(s) \leftarrow \sum_{s',r} p\big(s', r|s, \pi(s)\big)\big[r + \gamma V(s')\big],$$

- Requires knowledge of transition probabilities $P(s', r|s\ , a)$.

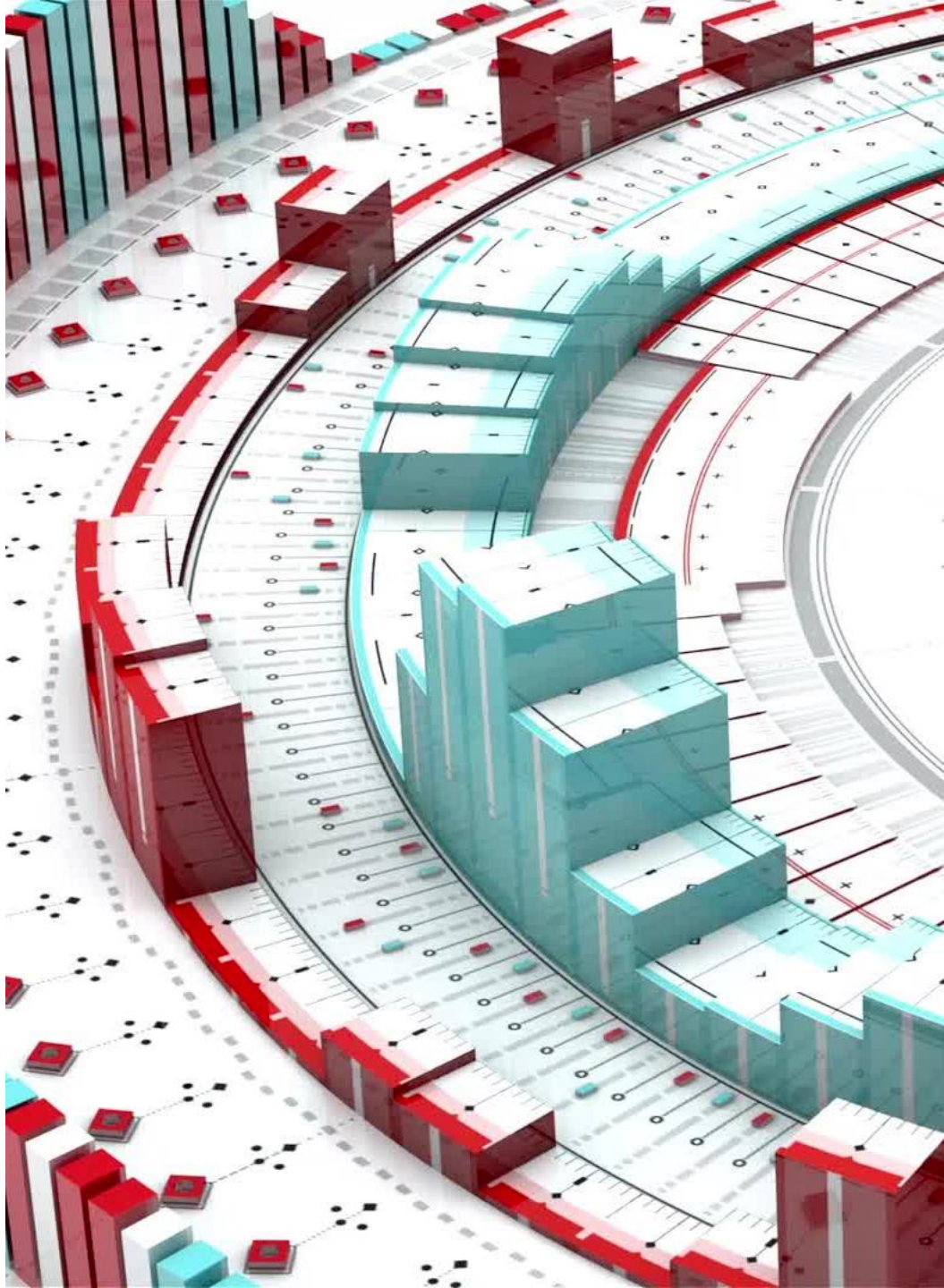
This is not exactly true! $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(\acute{s}, r|s, a)[r + \gamma V(\acute{s})]$



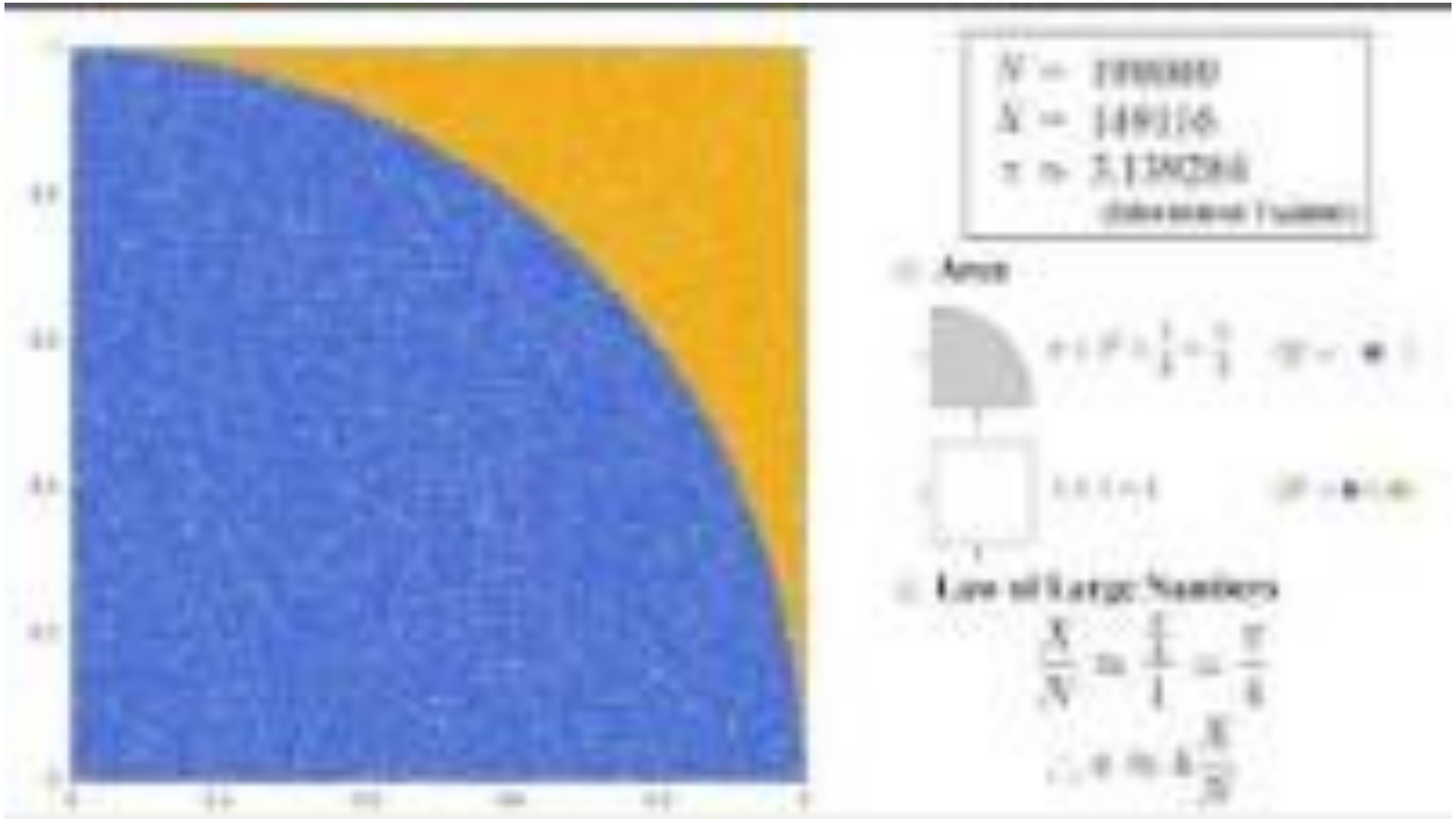
Monte Carlo

Monte Carlo Idea

- Model and analyze complex systems through **random sampling** and probability calculations.
- It is named after the Monte Carlo Casino in Monaco, known for its games of chance.
- Developed in the 1940s while working on the **Manhattan Project**.
- **Widely used in various fields**, allowing researchers to simulate complex systems by generating random samples and making probabilistic calculations based on those samples.



Use a rope, a chalk and pebbles to estimate Pi

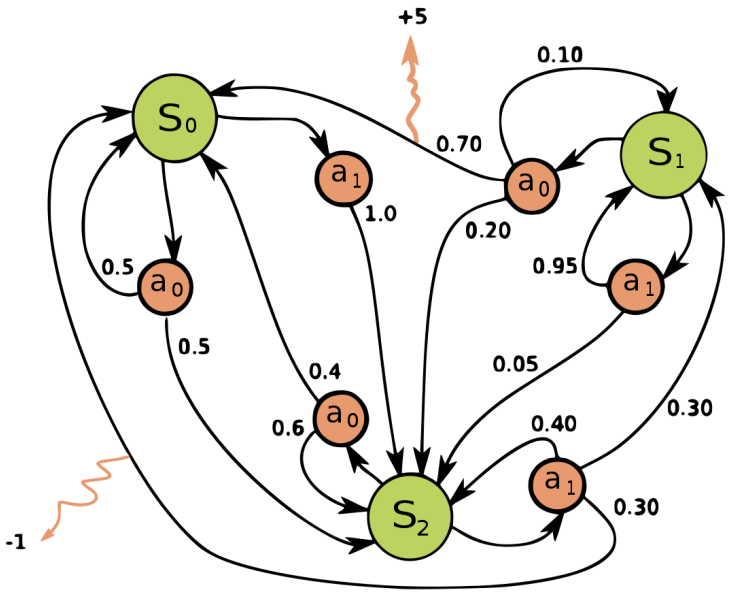


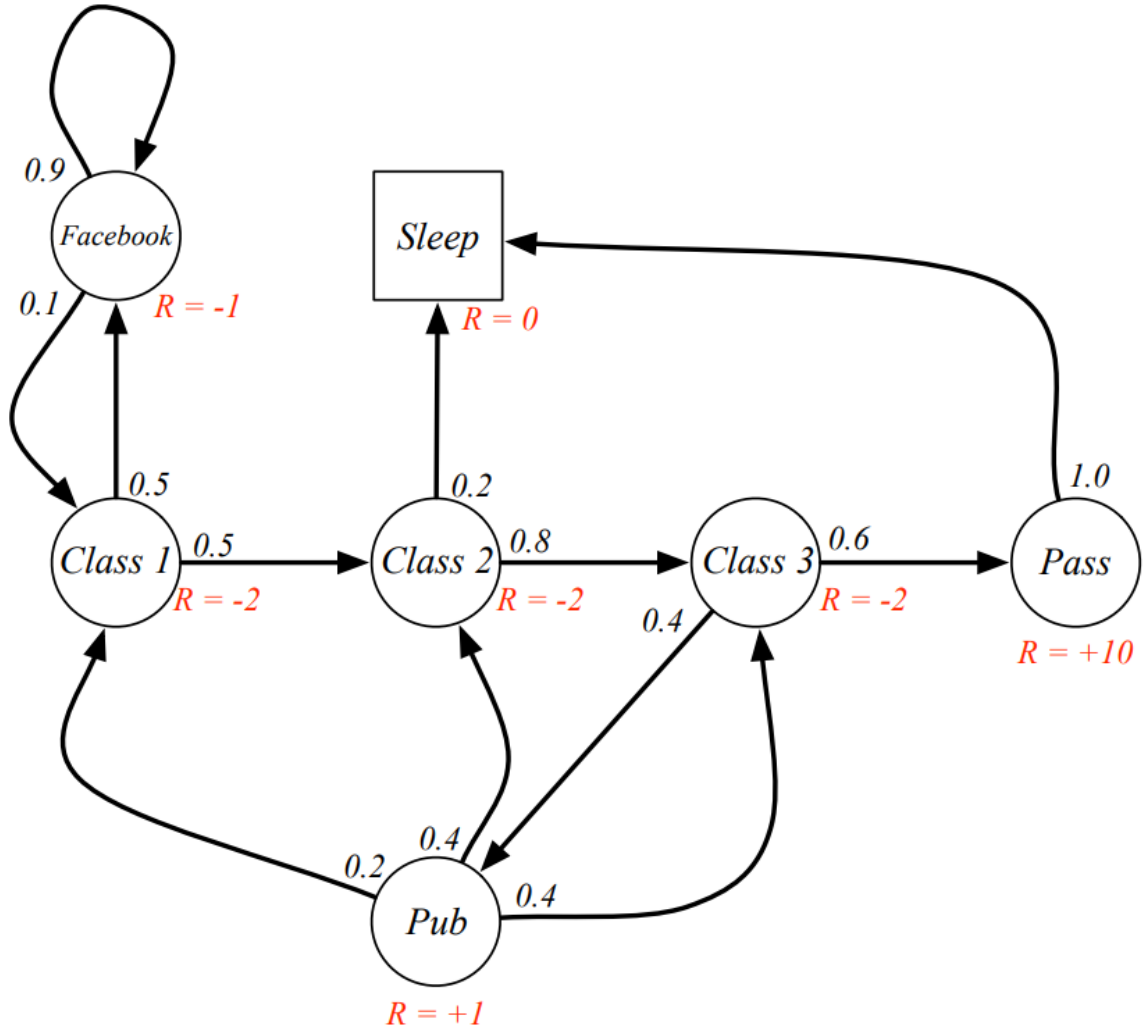
https://youtu.be/HUBNQicYDkU

A Random Walk & Monte Carlo Simulation

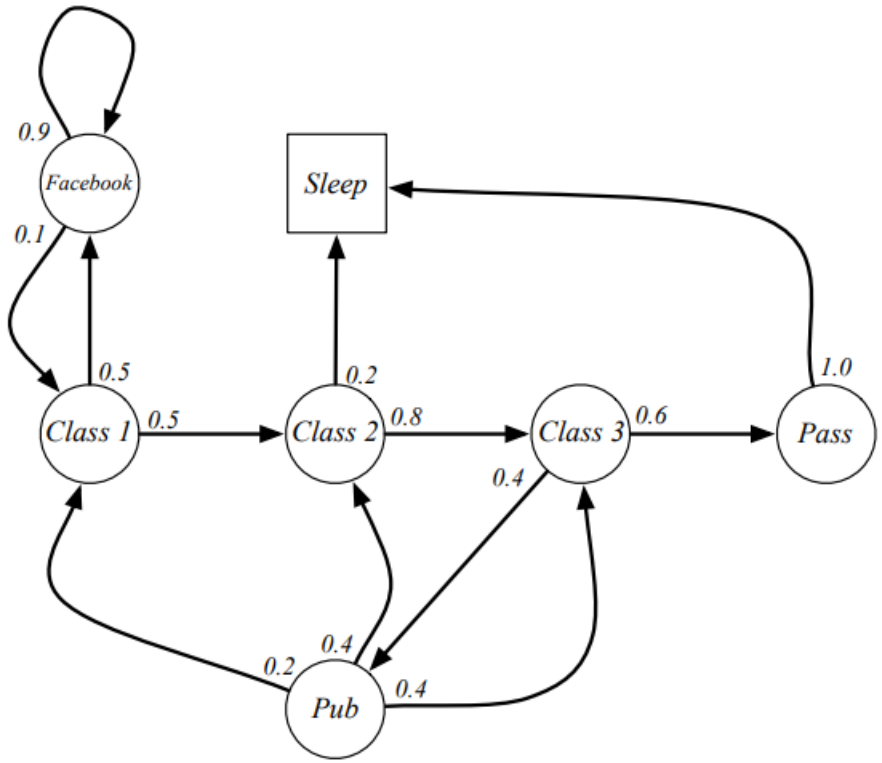


[A Random Walk & Monte Carlo Simulation || Python Tutorial || Learn Python Programming - YouTube]





Student Life MRP
Markov Reward Process



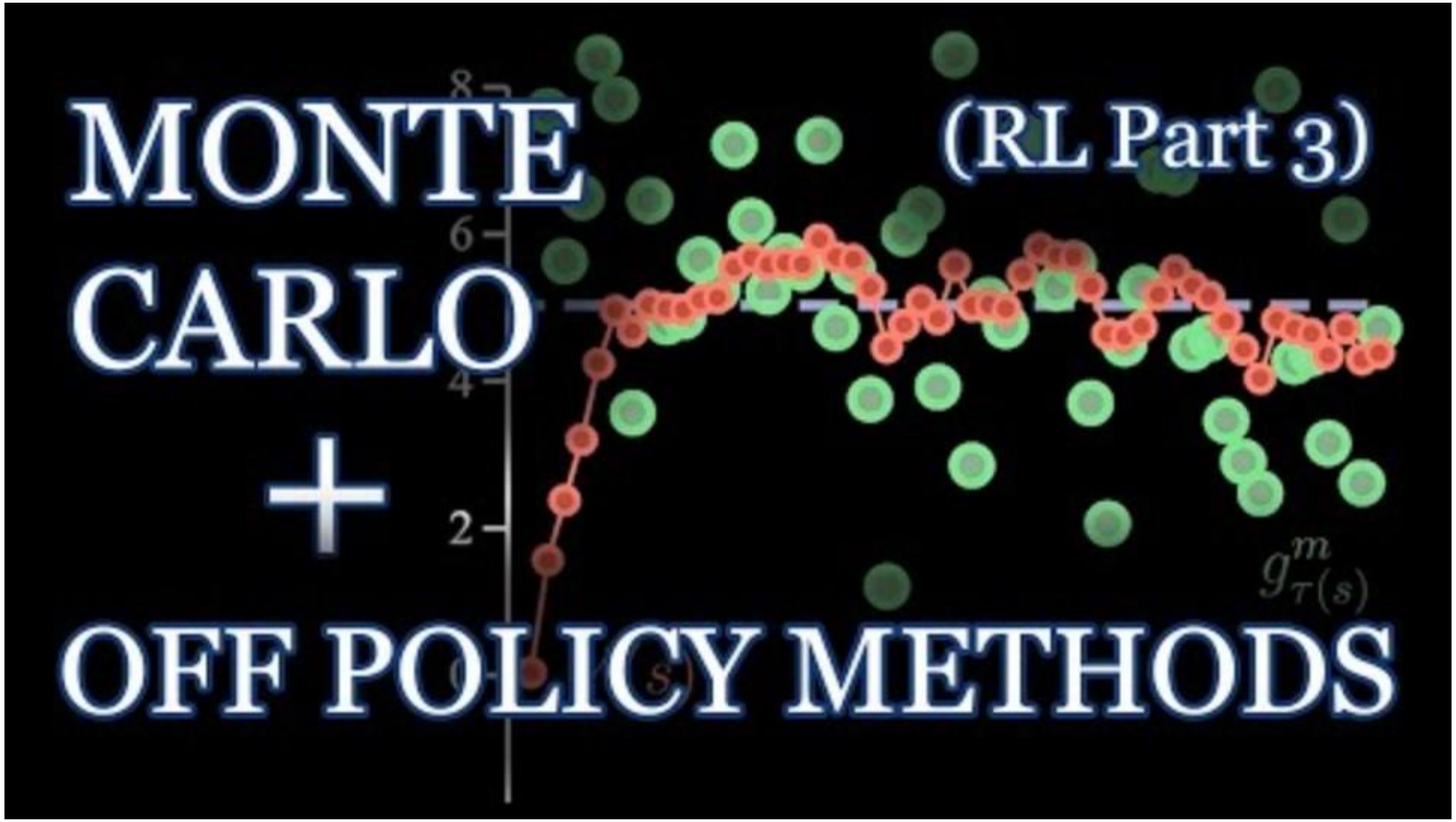
Sample episodes for Student Markov Chain starting from $S_1 = \text{C1}$

$$S_1, S_2, ..., S_T$$

- C1 C2 C3 Pass Sleep
- C1 FB FB C1 C2 Sleep
- C1 C2 C3 Pub C2 C3 Pass Sleep
- C1 FB FB C1 C2 C3 Pub C1 FB FB FB C1 C2 C3 Pub C2 Sleep

Lecture 2: Markov Decision Processes (stanford.edu)

Monte Carlo Model Free RL
(we do not cover off-policy)



https://youtu.be/bpUszPiWM7o?si=yJ1MVOh9dZSx223r

Monte-Carlo Estimation of states in an MRP

- MRP (Markov reward process) states encompass (state,action) pairs.

$$\mathbb{E}[G_t|S_t = s] \approx \frac{1}{C(s)} \sum_{m=1}^{M} \sum_{\tau}^{T_m-1} \mathbb{I}[s_\tau^m = s] g_\tau^m$$

We have M episodes, for each state we process all the episodes, $\mathbb{I}[s_\tau^m = s]$ is one when $s_\tau^m = s$, and 0 otherwise. $C(s)$ is the number of times s is observed in these episodes, and g_τ^m is the discounted sum of all future rewards in episode m at time τ.

Monte-Carlo Estimation of states in an MRP

- MRP (Markov reward process) states encompass (state,action) pairs.

$$\mathbb{E}[G_t|S_t = s] \approx \frac{1}{C(s)} \sum_{m=1}^{M} \sum_{\tau}^{T_m - 1} \mathbb{I}[s_\tau^m = s] g_\tau^m$$

How it works: Basically, we scan through all the episodes, and each time we encounter state s we add the g_τ^m we add it to a sum. We count how many times we encounter s i.e. $C(s)$ and divide the sum by $C(s)$ to estimate $\mathbb{E}[G_t|S_t = s]$ by an empirical average.

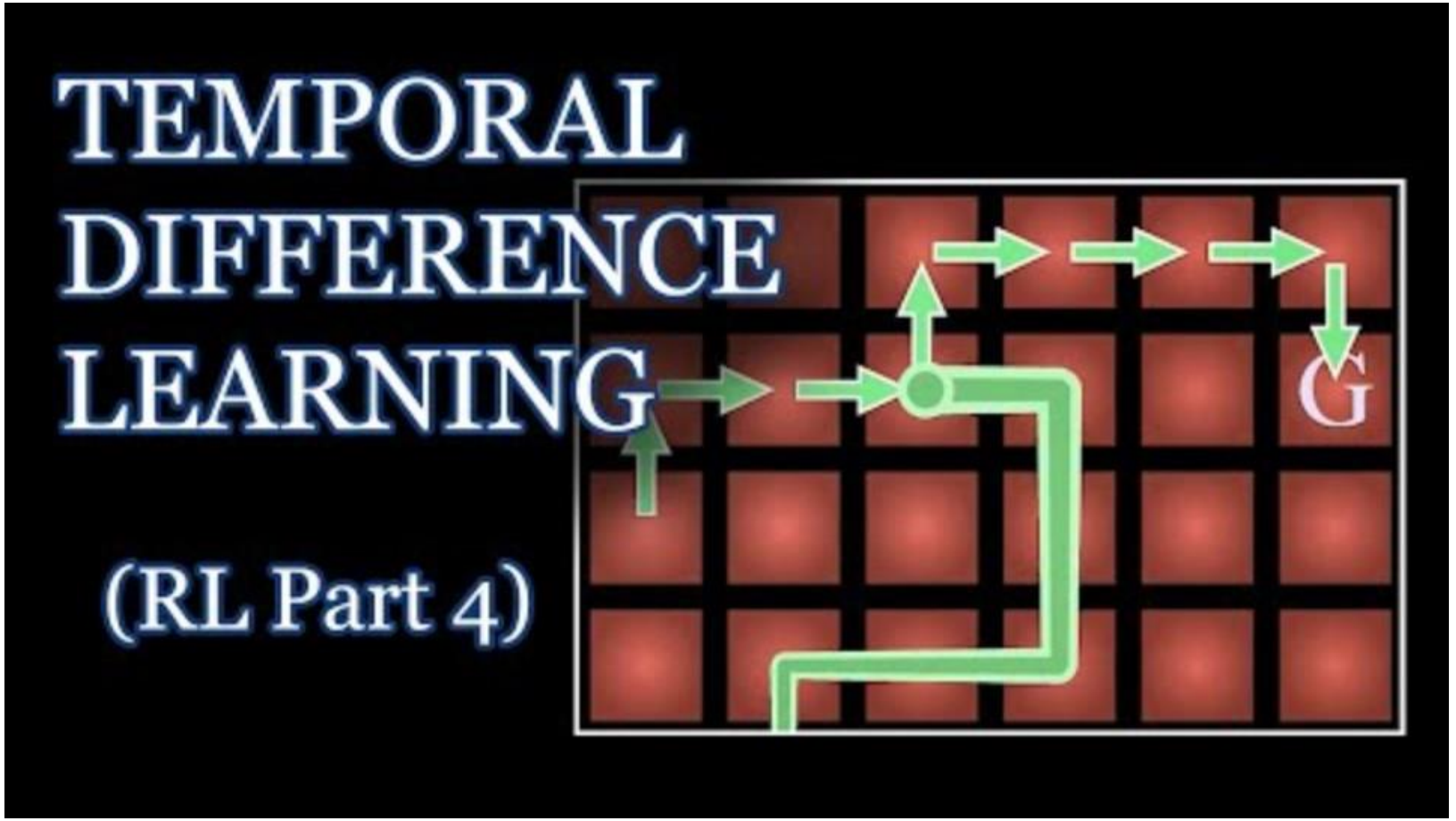
Constant Alpha MC

- We can achieve the same effect with a much simpler update formula that does not need to keep the sum in memory for all the episodes.

$$V(s_\tau^m) \leftarrow V(s_\tau^m) + \alpha\left(g_\tau^m - V(s_\tau^m)\right)$$

Here we update $V(s_\tau^m)$, the value function of the state we encountered in episode m at time τ . If the value we associated with this state $V(s_\tau^m)$

Temporal Difference Learning



https://youtu.be/AJiG3ykOxmY?si=Be-1by46zwfKnRs8

Boltzmann Exploration

- Epsilon-greedy methods pick a random action with a fixed probability.
- For a non-random action max value of the Q-function is used.

- In Boltzmann policy we use the soft-max function, all decisions are random, and the higher the Q-values the higher the selection chance.

$$\pi(a_i|s) \propto \exp(-\beta Q(s, a_i)$$

note that $\sum_i \pi(a_i|s) = 1$ and that let us normalize the probabilities.

(Prioritized) Experience Replay

- In all algorithms so far, you have used each experience once to update the agent Q-values.
- Not all experiences have the same value. Maybe you want to repeat some experience.
- The idea of the experience replay is to put the experience in a buffer, and randomly sample the buffer. As a bonus, this also decorates the time dependency among updates.
- Prioritized experience replay, uses the temporal difference error to guide the selection, the larger the TD-error, the larger the chance of getting selected.

Appendix

MDP Research

Machine Learning (2021) 110:2419–2468
https://doi.org/10.1007/s10994-021-05961-4



Challenges of real-world reinforcement learning: definitions, benchmarks and analysis

Gabriel Dulac-Arnold[1] · Nir Levine[2] · Daniel J. Mankowitz[2] · Jerry Li[2] · Cosmin Paduraru[2] · Sven Gowal[2] · Todd Hester[2]

Received: 16 March 2020 / Revised: 27 September 2020 / Accepted: 16 February 2021 /
Published online: 22 April 2021
© The Author(s), under exclusive licence to Springer Science+Business Media LLC, part of Springer Nature 2021

Abstract

Reinforcement learning (RL) has proven its worth in a series of artificial domains, and is beginning to show some successes in real-world scenarios. However, much of the research advances in RL are hard to leverage in real-world systems due to a series of assumptions that are rarely satisfied in practice. In this work, we identify and formalize a series of independent challenges that embody the difficulties that must be addressed for RL to be commonly deployed in real-world systems. For each challenge, we define it formally in the context of a Markov Decision Process, analyze the effects of the challenge on state-of-the-art learning algorithms, and present some existing attempts at tackling it. We believe that an approach that addresses our set of proposed challenges would be readily deployable in a large number of real world problems. Our proposed challenges are implemented in a suite of continuous control environments called `realworldrl-suite` which we propose an as an open-source benchmark.

Keywords Reinforcement learning · Real-world · Applied reinforcement learning

Reinforcement Learning for Automatic Test Case Prioritization and Selection in Continuous Integration

Helge Spieker
Simula Research Laboratory
Lysaker, Norway
helge@simula.no

Arnaud Gotlieb
Simula Research Laboratory
Lysaker, Norway
arnaud@simula.no

Dusica Marijan
Simula Research Laboratory
Lysaker, Norway
dusica@simula.no

Morten Mossige
University of Stavanger
Stavanger, Norway
ABB Robotics
Bryne, Norway
morten.mossige@uis.no

arXiv:1811.04122v1 [cs.SE] 9 Nov 2018

ABSTRACT

Testing in Continuous Integration (CI) involves test case prioritization, selection, and execution at each cycle. Selecting the most promising test cases to detect bugs is hard if there are uncertainties on the impact of committed code changes or, if traceability links between code and tests are not available. This paper introduces Retecs, a new method for automatically learning test case selection and prioritization in CI with the goal to minimize the round-trip time between code commits and developer feedback on failed test cases. The Retecs method uses reinforcement learning to select and prioritize test cases according to their duration, previous last execution and failure history. In a constantly changing environment, where new test cases are created and obsolete test cases are deleted, the Retecs method learns to prioritize error-prone test cases higher under guidance of a reward function and by observing previous CI cycles. By applying Retecs on data extracted from three industrial case studies, we show for the first time that reinforcement learning enables fruitful automatic adaptive test case selection and prioritization in CI and regression testing.

CCS CONCEPTS

•Software and its engineering → Software verification and validation; Software testing and debugging;

KEYWORDS

Regression testing, Test case prioritization, Test case selection, Reinforcement Learning, Machine Learning, Continuous Integration

Software Testing and Analysis , Santa Barbara, CA, USA, July 2017 (ISSTA'17), 12 pages.
DOI: 10.1145/3092703.3092709

1 INTRODUCTION

Context. Continuous Integration (CI) is a cost-effective software development practice commonly used in industry [10, 13] where developers frequently integrate their work. It involves several tasks, including version control, software configuration management, automatic build and regression testing of new software release candidates. Automatic regression testing is a crucial step which aims at detecting defects as early as possible in the process by selecting and executing available and relevant test cases. CI is seen as an essential method for improving software quality while keeping verification costs at a low level [24, 34].

Unlike usual testing methods, testing in CI requires tight control over the selection and prioritization of the most promising test cases. By most promising, we mean test cases that are prone to detect failures early in the process. Admittedly, selecting test cases which execute the most recent code changes is a good strategy in CI, such as, for example in coverage-based test case prioritization [9]. However, traceability links between code and test cases are not always available or easily accessible when test cases correspond to system tests. In system testing for example, test cases are designed for testing the overall system instead of simple units of code and instrumenting the system for code coverage monitoring is not easy. In that case, test case selection and prioritization has to be handled differently and using historical data about failures and successes of

Optimal Dispatch in Emergency Service System via Reinforcement Learning

Cheng Hua , Tauhid Zaman
School of Management, Yale University
{cheng.hua, tauhid.zaman}@yale.edu

arXiv:2010.07513v1 [eess.SY] 15 Oct 2020

Abstract

In the United States, medical responses by fire departments over the last four decades increased by 367%. This had made it critical to decision makers in emergency response departments that existing resources are efficiently used. In this paper, we model the ambulance dispatch problem as an average-cost Markov decision process and present a policy iteration approach to find an optimal dispatch policy. We then propose an alternative formulation using post-decision states that is shown to be mathematically equivalent to the original model, but with a much smaller state space. We present a temporal difference learning approach to the dispatch problem based on the post-decision states. In our numerical experiments, we show that our obtained temporal-difference policy outperforms the benchmark myopic policy. Our findings suggest that emergency response departments can improve their performance with minimal to no cost.

1 Introduction

In the United States, medical responses by fire departments over the last four decades has increased by 367% (Figure 1), as reported by [Evarts, 2019]. The reasons for the dramatic increase in medical calls include the aging population and health insurance that covers most of the ambulance costs (Boston Globe, Nov 29, 2015). Cities with tight budgets are short of response units to respond to the growing amount of medical calls in time. NBC10 in Philadelphia, on Feb 28, 2019, reported that two thirds of the emergency medical calls had an ambulance response time of more than nine minutes. Thus, how to efficiently use the existing resources becomes an important topic to decision makers in emergency response departments.

In current practice, cities dispatch ambulance units according to a fixed dispatch rule, which always dispatches the closest available unit. The dispatch policy is deemed as a myopic policy as it only considers the current call and ignores the impact of dispatching a unit to future calls. In this paper, we model the ambulance dispatch problem as an average-cost Markov decision process (MDP). We finds a dispatch policy that minimizes the mean response time using reinforcement learning. We propose an alternative MDP formulation for the problem using post-decision states that we show is mathematically equivalent to the original MDP formulation, but with a much smaller state space.

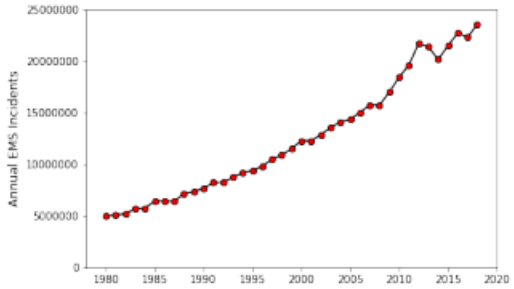


Figure 1: Annual emergency medical servie (EMS) incidents from 1980 to 2018.

Due to the curse of dimensionality, the application of both formulations are only restricted to small problems. To solve larger problems, we use temporal difference learning (TD-learning) with the post-decision states. In our numerical experiments, we show that the TD-learning algorithm converges quickly and the policies obtained from our method outperform the myopic policy that always dispatches the closest available unit.

The remainder of this paper is organized as follows. In Section 2, we provide a review of the relevant literature. In Section 3, we present the Markov decision process formulation. Section 4 presents the formulation using post-decision states which reduces the state space of the original formulation. In Section 5, we present the temporal difference learning algorithm that is based on the post-decision states and its theoretical properties, while in Section 6, we show the performance of the algorithm in numerical experiments. We conclude the paper in Section 7.

2 Literature Review

There are a number of papers that study dispatch policies for emergency medical services (EMS) in different settings. The optimal dispatch rule was first studied in [Carter *et al.*, 1972].

2010.07513.pdf (arxiv.org)

Hint Generation

- This is an old paper from 2008, what can do more with these ideas and conversational agents like ChatGPT: [(PDF) Toward Automatic Hint Generation for Logic Proof Tutoring Using Historical Student Data (researchgate.net)]

Table 1. Sample Proof 1 Solution

	Premise	Line	Reason
1.	a → b		Given
2.	c → d		Given
3.	¬ (a → d)		Given
	¬ a v d	**3**	**rule IM (error)**
4.	a ^ ¬ d	3	rule IM implication
5.	a	4	rule S simplification
	b	**4**	**rule MP (error)**
6.	b	1,5	rule MP modus ponens
7.	¬ d	4	rule S simplification
8.	¬c	2,7	rule MT modus tollens
9.	b ^ ¬c	6,8	rule CJ conjunction

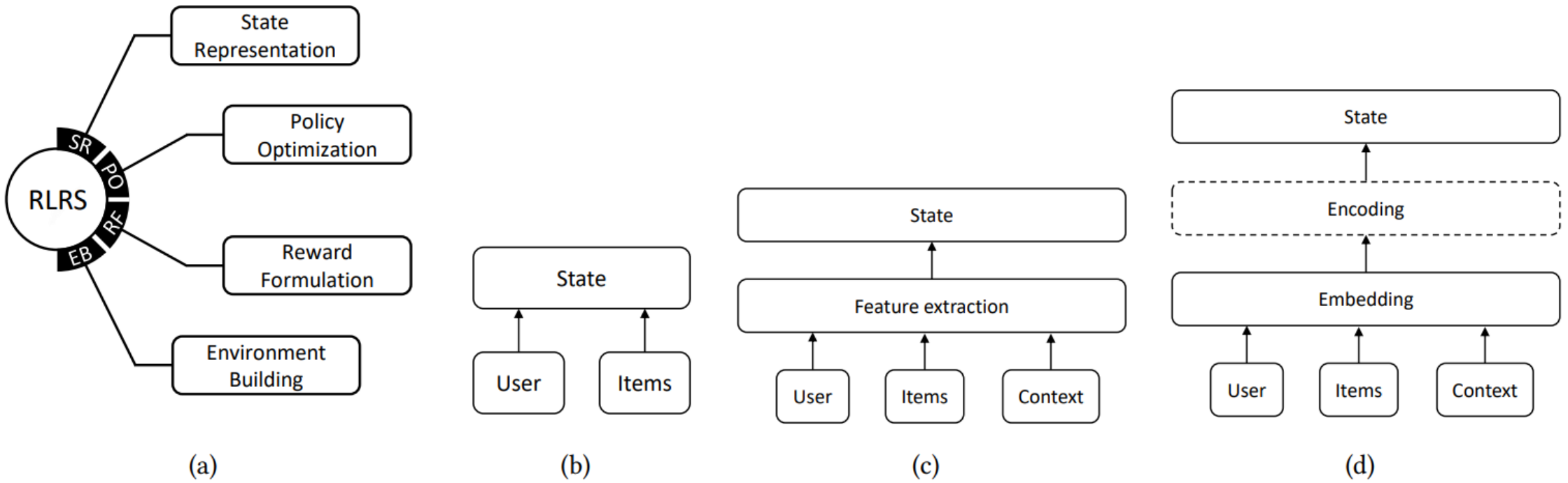
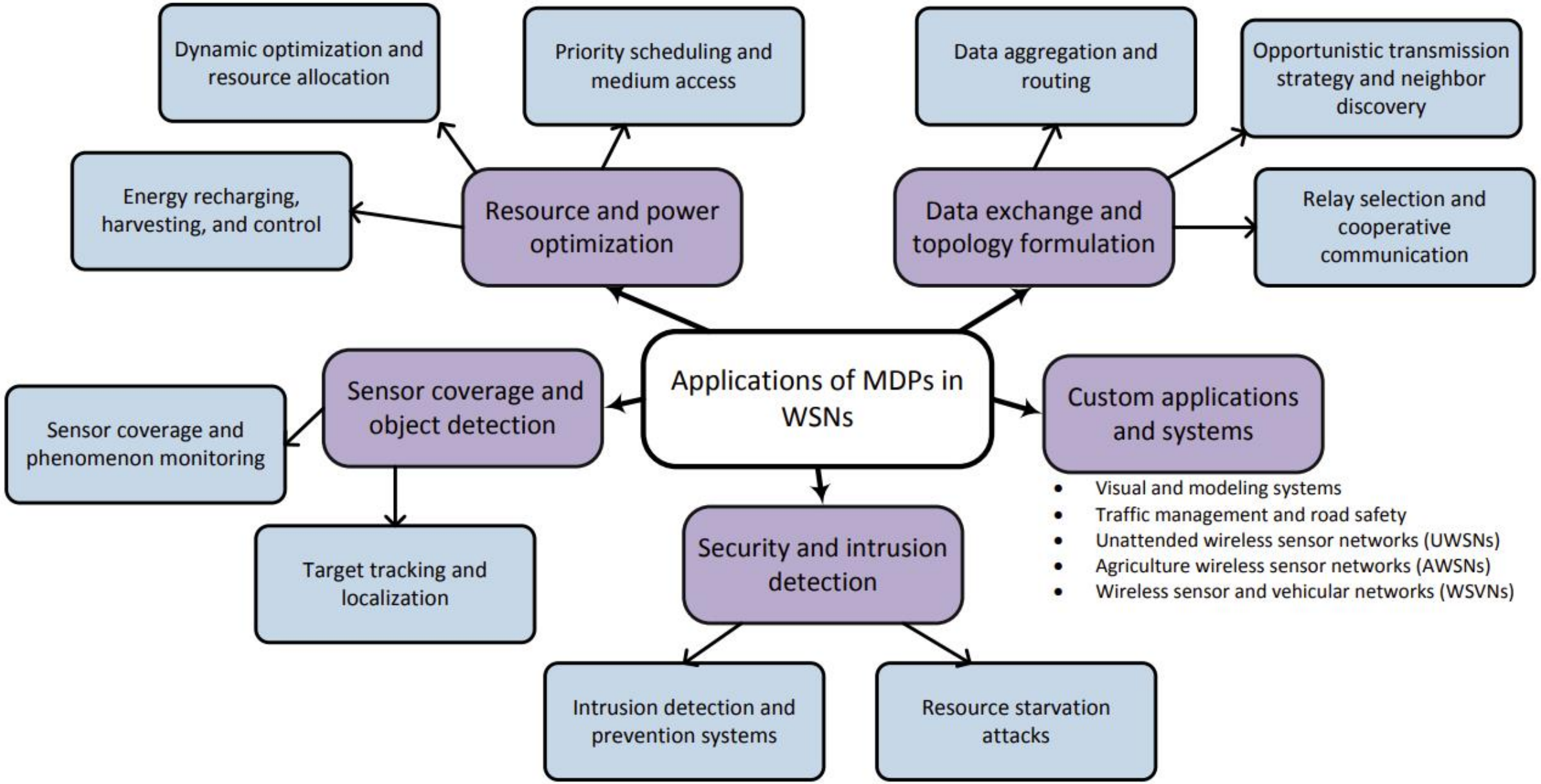


Fig. 3. (a) The proposed RLRS framework, (b) SR1, (c) SR2, (d) SR3. The dashed Encoding module in SR3 means that some models merely use input embeddings as states.

[2101.06286] Reinforcement learning based recommender systems: A survey (arxiv.org)

RLRS	Year	SR	PO	RF	EB	Metrics	Dataset	Application*
WebWatcher [83]	1997	SR1	Q-learning	R1	Offline	Accuracy	CMU	Web
Preda et al. [84]	2005	SR1	Sarsa(λ)	R1	Online	RDR	N/A	Web
MDP [50]	2005	SR1	Policy iteration	R2	Online	RC, ED	Mitos	E-commerce
Taghipour et al. [85]	2007	SR1	Q-learning	R2	Offline	Accuracy, Coverage, SG	DePaul	Web
Mahmood et al. [86]	2007	SR2	Policy iteration	R1	Simulation	N/A	N/A	Trip
Taghipour et al. [87]	2008	SR1	Q-learning	R2	Offline	HR, PA, CR, RQ	DePaul	Web
Mahmood et al. [88]	2009	SR2	Policy iteration	R1	Online	26 variables	N/A	Trip
APG [89]	2010	SR1	Q-learning, Sarsa	R2	Simulation, Online	MR, MH, LTR, UR	N/A	Music
Shortreed et al. [90]	2011	SR2	FQI	R2	Offline	PANSS	CATIE	Health
Zhao et al. [91]	2011	SR2	Fitted Q	R2	Simulation	Survival	N/A	Health
RLradio [49]	2012	SR2	R-learning	R1	Online	Listening Time	N/A	Radio channel
Mahmood et al. [92]	2014	SR2	Q-learning	R1	Simulation, Online	BP, ST, BDV, QE	Amazon	Trip
DJ-MC [93]	2014	SR2	MCTS	R2	Simulation, Online	Reward	Million Song	Music
Theocharous et al. [94]	2015	SR2	FQI	R1	Offline	CTR, LTV	N/A	Ad
POMDP-Rec [95]	2016	SR2	Fitted Q	R2	Offline	RMSE	ML1M, YM	
RLWRec [51]	2017	SR1	Q-learning	R2	Offline	CAC	N/A	Music
Zhang et al. [96]	2017	SR1	GVI	R2	Offline	MRR, P, NDCG	ACM	Collaborator
Choi et al. [44]	2018	SR1	Q-learning, Sarsa	R2	Offline	P, R	ML100K, ML1M	
Intayoad et al. [97]	2018	SR1	Sarsa	R1	Offline	RMSE	N/A	E-learning
RPRMS [98]	2019	SR3	Q-learning	R1	Offline	UR	N/A	Music
CAPR [99]	2019	SR2	MCTS	R2	Offline	P, R, F1	Weeplace	POI
PHRR [100]	2020	SR3	MCTS	R2	Offline	HR, F1	Million Song, TPS, HSP	Music
Kokkodis et al. [101]	2021	SR1	Q-learning	R2	Offline	Market Revenue, WI	N/A	Career path

[2101.06286] Reinforcement learning based recommender systems: A survey (arxiv.org)



1501.00644.pdf (arxiv.org) Wireless Sensor Networks

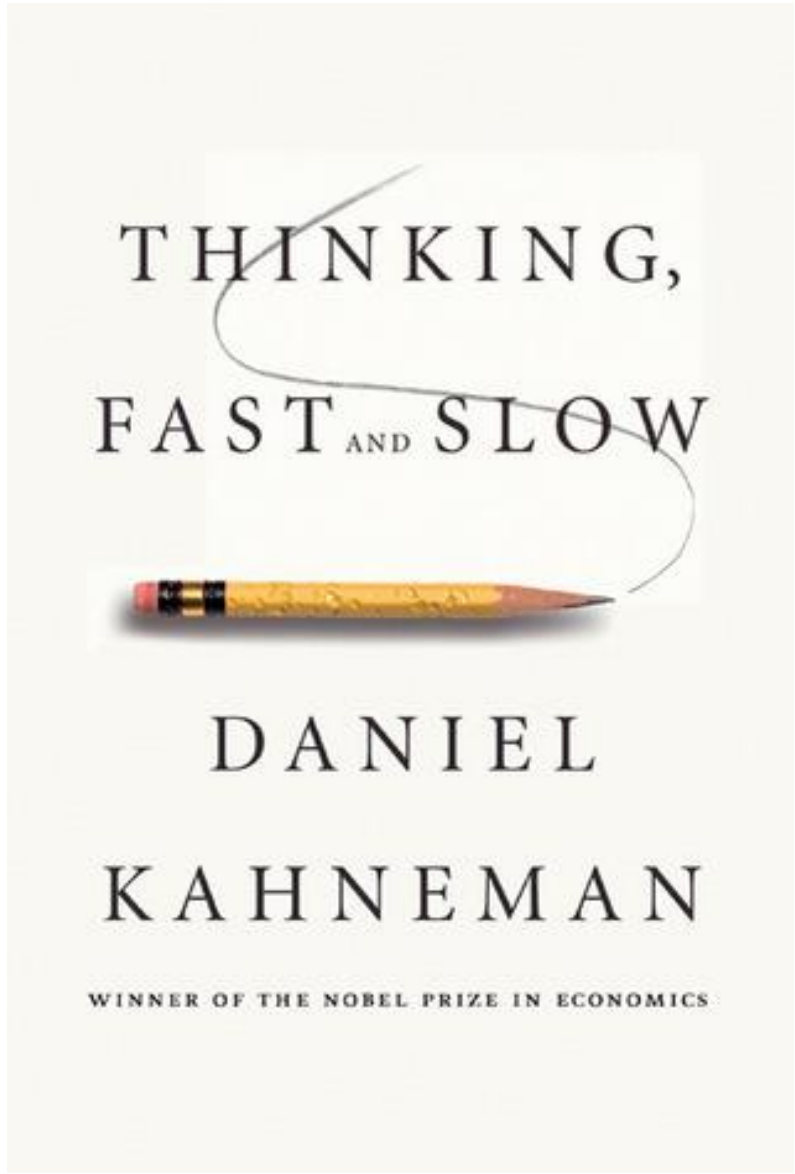
Appendix

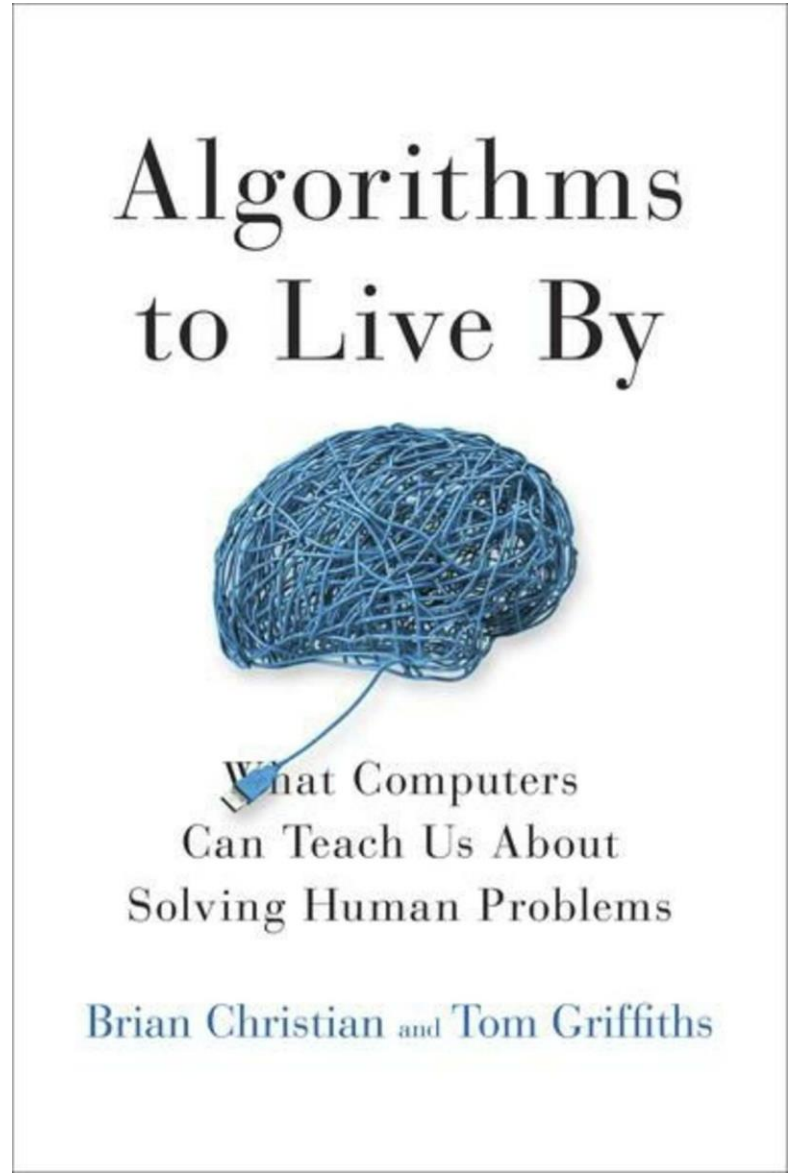
Extra Material

Why this section?

- Decision Theory and Reinforcement Learning can give you general insights useful for general decision-making.

- The field of RL has connections to psychology.


THINKING,
FAST AND SLOW
DANIEL
KAHNEMAN
WINNER OF THE NOBEL PRIZE IN ECONOMICS


Algorithms
to Live By
What Computers
Can Teach Us About
Solving Human Problems
Brian Christian and Tom Griffiths

A History of Reinforcement Learning - Prof. A.G. Barto

https://www.youtube.com/watch?v=ul6B2oFPNDM

Summary:

The talk covers the historical context, including the speaker's early experiences with cellular automata and genetic algorithms. The transition to RL is explored, emphasizing trial and error learning, which had not been extensively studied in AI.

Supervised learning is briefly contrasted with RL, with a focus on learning from evaluations rather than error vectors. The roots of RL are traced back to **psychologist Edward Thorndike's** trial-and-error learning**.** The essence of RL is described as **search plus memory**, with the speaker presenting an early associative search network and later developments like the actor-critic architecture and temporal difference algorithm.

Connections are made to Arthur Samuel's checkers player and dynamic programming. **The TD error is linked to dopamine** in the brain, suggesting a **connection between RL and neuroscience**. Surprising developments, such as TD Gammon and AlphaGo, are discussed, highlighting the role **of Monte Carlo Tree Search**. The speaker addresses challenges, including the **design of reward functions** and the potential for unintended consequences, advocating for "courage ability" in AI systems.

The talk concludes with a mention of model-based RL and the importance of combining different learning approaches, **drawing insights from psychology and neuroscience**.

How a complex system emerges from exploration-exploitation trade-off

https://youtu.be/R33sNqlTFJc

- Complex systems involve interdependent agents, generating emergent phenomena and robustness.
- Fundamental trade-off: agents balance exploration (search for better solutions) and exploitation (utilize known information).
- Balancing exploration and exploitation is crucial in preventing system decline and seizing better opportunities.
- Practical example: Choosing between exploring new delis or exploiting known information in New York.
- Explore-exploit trade-off introduced via the "two-arm bandit" problem, illustrating decision-making in uncertain environments.
- Application of the trade-off to a rugged landscape problem, exemplified by climbing Mount Fuji.
- Introduction of the greedy algorithm (uphill climbing) and its limitations in addressing complex problems.
- Simulated annealing, an interdisciplinary search algorithm originating from physics, balances exploration and exploitation.
- Annealing, used in hardening materials, serves as an analogy for the process in simulated annealing.
- Simulated annealing application in complex systems, including evolutionary processes and market decision-making.
- Proposal to explain the tendency of systems to be complex using the explore-exploit trade-off, emphasizing its role in maintaining dynamism and preventing stasis.